Fakultät Informatik an der Technischen Hochschule Nürnberg Georg Simon Ohm



Bachelorarbeit

Systematische Transformation KI-generierter Web-Architekturen in sichere, lokale Desktop-Anwendungen unter Anwendung des Local-First-Paradigmas

Joshua Ohans

Erstprüfer: Prof. Dr. Matthias Teßmann

Zweitprüfer: Prof. Dr. Ronald Petrlic

© Joshua Ohans 2026

Diese Arbeit, einschließlich ihrer Teile, **ist urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ohne Zustimmung des Urhebers ist untersagt und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmung und die Speicherung und Verarbeitung in elektronischen Systemen.

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Motivation und Relevanz ... 1
1.2 Problemstellung ... 1
1.3 Fallstudie und Zielsetzung ... 2
1.4 Forschungsfrage ... 2
1.5 Aufbau der Arbeit ... 2

2 Theoretischer Hintergrund **4**
2.1 KI-gestützte Anwendungsentwicklung und Softwarequalität ... 4
2.2 Webarchitekturen und Trennung der Verantwortlichkeiten ... 5
2.3 API-Verträge, Typisierung und Codegenerierung ... 5
2.4 Electron als Desktop-Laufzeit ... 6
2.4.1 Navigation über Hashrouting ... 6
2.5 Lokale Persistenz, Schema und Identität ... 7
2.6 Offline-Nutzung und Local-First-Synchronisierung ... 7
2.7 Sicherheitsgrenzen, Härtung und Nachweise ... 8
2.8 Bestehende Ansätze und Einordnung der Arbeit ... 8

3 Methodisches Vorgehen **10**
3.1 Forschungsansatz und Erkenntnisziel ... 10
3.2 Untersuchungsgegenstand und Analyseverfahren ... 10
3.3 Vorversuch und Konsequenzen für das Migrationsvorgehen ... 11
3.4 Ablauf der Entwicklung ... 11
3.4.1 Phase 1: Überführung in eine lokal nutzbare Offline-Anwendung ... 11
3.4.2 Phase 2: Ergänzung um Synchronisierung und weitere Online-Anbindungen ... 12
3.4.3 Phase 3: Konsolidierte Prüfung und Härtung ... 12
3.5 Ableitung des Leitfadens und Bewertung ... 12

4 Analyse und Anforderungsdefinition **13**
4.1 Vom übergeordneten Ziel zur Fallauswahl ... 13
4.2 Funktionen und Architektur des Ausgangsartefakts ... 14
4.2.1 Funktionaler Untersuchungsumfang ... 14
4.2.2 Technische Komponenten und Repository-Struktur ... 14
4.2.3 Lokale Funktionen und exemplarische Online-Anbindungen ... 17
4.3 Zielarchitektur und Sicherheitsbegriff ... 17
4.4 Anforderungskatalog ... 18
4.4.1 A1: Lauffähige Desktop-Anwendung ... 18
4.4.2 A2: Dreischichtenarchitektur ... 18
4.4.3 A3: Begrenzung des Refactoring-Aufwands ... 19

4.4.4 A4: Lokale Kommunikation im Electron-Prozessmodell ... 19
4.4.5 A5: Lokale Datenhaltung und Offline-Kernfunktionen ... 19
4.4.6 A6: Local-First-Kollaboration am Gruppenfeature „Aufgaben“ ... 19
4.4.7 A7: Härtung gemäß Electron Security Guidelines ... 19
4.4.8 A8: Repräsentativer Kernfunktionsumfang und Erweiterbarkeit ... 19
4.4.9 A9: Nachvollziehbarer Migrationsleitfaden ... 19
4.5 Abgrenzung des Untersuchungsumfangs ... 20

5 Entwurf, Migration und Migrationsleitfaden 21
5.1 Struktur des Migrationsleitfadens ... 21
5.2 Phase 1: Von der Webanwendung zur Offline-Kernanwendung ... 22
5.2.1 Initialisierung des Electron-Projekts und Übernahme der Datenbankmodule ... 22
5.2.2 Anpassung der Tabellen und Datenbankabhängigkeiten an SQLite ... 23
5.2.3 Versionierte Schemamigrationen und Datenbankinitialisierung ... 24
5.2.4 Übernahme des API-Vertrags und Anpassung der Codegenerierung ... 25
5.2.5 Funktionsweise der OpenAPI-IPC-Codegenerierung ... 26
5.2.6 IPC-Transport, Preload-Schnittstelle und Dispatcher ... 28
5.2.7 Begründung und Grenzen der vertragsbasierten Generierung ... 28
5.2.8 Übernahme der React-Oberfläche in den Renderer ... 30
5.2.9 Anbindung der Seiten, Datumswerte und Navigation ... 31
5.2.10 Lokale Authentifizierung als erster vollständiger Verarbeitungspfad ... 32
5.2.11 Integration der CRUD-Funktionen am Beispiel der Notenverwaltung ... 33
5.2.12 Lokaler Dateiimport am Beispiel iCal ... 34
5.2.13 Abschluss der Offline-Kernanwendung und Übergang zu Phase 2 ... 36
5.3 Phase 2: Netzwerkanbindungen und Local-First-Synchronisierung ... 36
5.3.1 Auswahlkriterien und betrachtete Synchronisierungslösungen ... 36
5.3.2 Entscheidung für PowerSync und Supabase ... 37
5.3.3 Vorprüfung der Integration und Grenzen der Wiederverwendung ... 37
5.3.4 Architektur mit zwei lokalen Datenhaltungsbereichen ... 38
5.3.5 Technische Integration der Kollaborationsmodule ... 40
5.3.6 Anpassung des Einladungsablaufs und vorausgesetzte Konten ... 43
5.3.7 Schrittweise Überführung weiterer Funktionen ... 44
5.3.8 Verantwortung für Schemaänderungen und Updates ... 46
5.3.9 Grenzen der untersuchten Synchronisierung ... 47
5.3.10 Online-Abruf des Studienkatalogs ... 47
5.4 Phase 3: Prüfung und Härtung der vervollständigten Anwendung ... 48
5.4.1 Prüfrahmen und Sicherheitsmatrix ... 48
5.4.2 Zuordnung der Anforderungen zu den Prüforten ... 49
5.4.3 Umgesetzte Nachbesserungen ... 51
5.4.4 Nachweisverfahren und dokumentierte Grenzen ... 52
5.5 Handlungsempfehlungen des Migrationsleitfadens ... 53
5.5.1 Voraussetzungen und Herstellung der lokalen Datenhaltung ... 53
5.5.2 Schnittstellen und Offline-Kernfunktionen überführen ... 54
5.5.3 Externe Daten und Kollaboration schrittweise ergänzen ... 55
5.5.4 Kontrollpunkte, Härtung und Fortführung des Leitfadens ... 56

6 Evaluation und Demonstration 58
6.1 Prüfgegenstand und Belegbasis ... 58

6.2 Architektur und Wiederverwendung 59
6.3 Kernfunktionen und lokale Nutzbarkeit 60
6.4 Synchronisierung und Fehlerverhalten 62
6.5 Ergebnis der Sicherheitsprüfung 62
6.6 Bewertung der Anforderungen und des Leitfadens 63

7 Diskussion **65**
7.1 Beitrag zur Forschungsfrage 65
7.2 Wiederverwendung und architektonische Anpassung 65
7.3 Einordnung der Local-First-Architektur 67
7.4 Bedeutung der Härtung und des Paketbetriebs 68
7.5 Übertragbarkeit und Grenzen der Ergebnisse 68

8 Fazit und Ausblick **70**
8.1 Fazit 70
8.2 Ausblick 71

A Ursprüngliche Erstellungs-Prompts **81**
A.1 Prompt 1: Persönliche Studienorganisation 81
A.2 Prompt 2: Studiengruppen und Zusammenarbeit 83

Abkürzungsverzeichnis

Kapitel 1

Einleitung

1.1 Motivation und Relevanz

Viele Menschen möchten Anwendungen für den persönlichen Gebrauch erstellen, verfügen jedoch über keine Programmiererfahrung. Durch KI-gestützte Werkzeuge wie Replit oder Lovable ist es ihnen heute möglich, innerhalb kurzer Zeit funktionsfähige Anwendungen zu erzeugen, ohne eine einzige Zeile Code selbst zu schreiben oder zu verstehen. Diese Anwendungen nutzen die Erstellenden im Vertrauen darauf, sie seien sicher und produktionsreif. Tatsächlich betrachten sie den erzeugten Code jedoch nie; sie beurteilen die Anwendung ausschließlich anhand ihrer sichtbaren Funktionen. Der Quellcode bleibt für sie eine Blackbox, deren Qualität und Sicherheit sie nicht einschätzen können.

Die Relevanz dieser Entwicklung reicht über die Erstellung kleiner Hobby-Werkzeuge hinaus: KI-gestützte Entwicklungsplattformen richten sich auch an Unternehmen und werben mit Anwendungen für den produktiven Einsatz, wie etwa die Unternehmensbeispiele auf der Website von Lovable zeigen [1]. Damit gewinnen Anforderungen an Sicherheit, Zuverlässigkeit und Wartbarkeit an Bedeutung. Werden solche Anwendungen zudem als lokale Desktop-Anwendungen mit direktem Zugriff auf Dateisystem und Systemressourcen betrieben, müssen zusätzliche Sicherheitsanforderungen berücksichtigt werden. Eine Anwendung, die oberflächlich funktioniert, aber unbemerkt unsicher oder architektonisch mangelhaft ist, kann in einer solchen lokalen Laufzeitumgebung Schäden verursachen, die über die Zugriffsmöglichkeiten einer im Browser isolierten Anwendung hinausgehen. Genau an dieser Stelle setzt die vorliegende Arbeit an.

1.2 Problemstellung

Von technischen Laien instruierte generative KI-Modelle fokussieren sich nahezu ausschließlich auf die Umsetzung der gewünschten Funktionen. Fundamentale Qualitätsmerkmale des Software Engineerings nach ISO/IEC 25010 — Wartbarkeit, Trennung der Belange, Interoperabilität und Sicherheit — bleiben dabei regelmäßig unberücksichtigt. Auch die Produktdarstellung von Lovable betont die schnelle Umsetzung beschriebener Ideen und deren unmittelbare Bereitstellung. Zwar bewirbt die Plattform zugleich Sicherheitsfunktionen; aus der sichtbaren Funktionalität und den Produktversprechen allein lässt sich jedoch nicht ableiten, ob der erzeugte Code die Qualitätsanforderungen eines konkreten Einsatzszenarios erfüllt [1]. Der entstehende Code ist häufig unstrukturiert und stark gekoppelt, was spätere Wartung und Erweiterung erschwert. Da die Erstellenden nur die Funktionen, nicht aber den Code beurteilen, bleibt diese Diskrepanz für sie unsichtbar: Die Anwendung wirkt einsatzbereit, obwohl sie es nicht ist.

Besonders kritisch wird diese Diskrepanz, sobald eine solche Webanwendung in eine lokale Desktop-Umgebung überführt werden soll. Die Zielumgebung stellt völlig andere Anforderungen als die isolierte Sandbox des KI-Werkzeugs, und die vom Werkzeug getroffenen Architekturentscheidungen lassen sich

nicht unverändert übernehmen. Technisch wäre es mit vergleichsweise geringem Aufwand möglich, eine Webanwendung nur oberflächlich zu verpacken, etwa indem im Main-Prozess einer Electron-Anwendung eine lokale Web-API betrieben und das Frontend unverändert im Renderer ausgeliefert wird. Ein solcher Ansatz ermöglicht zwar die Ausführung auf dem Desktop, berücksichtigt für sich genommen jedoch nicht die zusätzlichen Sicherheitsanforderungen, die durch den Zugriff auf lokale Dateien und Systemressourcen entstehen. Ohne eine gezielte Anpassung der Vertrauensgrenzen und Zugriffsmöglichkeiten können Schwachstellen der ursprünglichen Webanwendung dadurch Auswirkungen auf das lokale System erhalten. Ziel der Arbeit ist deshalb nicht die bloße Ausführbarkeit auf dem Desktop, sondern eine sichere, architektonisch saubere Transformation. Für genau diese Klasse von Anwendungen fehlt bislang ein systematisches Vorgehen, um eine von Laien in Auftrag gegebene, KI-generierte Webanwendung sicher in eine lokale Desktop-Anwendung zu transformieren.

1.3 Fallstudie und Zielsetzung

Um diese Lücke zu schließen, entwickelt die vorliegende Arbeit einen methodischen Migrationspfad für die Überführung KI-generierter Webanwendungen in sichere Local-First-Desktop-Anwendungen. Validierungsobjekt ist eine konkrete Fallstudie: der Studienplaner, eine deutschsprachige Studienverwaltungsanwendung, die als Replit-Webprototyp vorliegt und in eine Electron-Anwendung überführt wird. Entscheidend ist dabei, wie dieser Prototyp entstanden ist: Er wurde bewusst so in Auftrag gegeben, wie es eine Person ohne technischen Hintergrund tun würde. Die beiden ursprünglichen Prompts beschreiben in überwiegend alltagssprachlicher Form gewünschte Funktionen und Nutzungsszenarien, etwa einen persönlichen Stundenplan und die Zusammenarbeit in Studiengruppen (Abschnitt A.1 und Abschnitt A.2). Sie enthalten auch Anforderungen an Anmeldung und Zugriffsschutz, machen jedoch keine Vorgaben zur Wahl eines UI-Frameworks oder einer konkreten Backend- oder Datenbanktechnologie. Der Studienplaner ist damit keine von einem erfahrenen Entwickler gestaltete Anwendung, sondern eine authentisch laienhaft erzeugte, und repräsentiert so genau die Klasse von Artefakten, auf die der Migrationspfad abzielt.

Die Wahl des Local-First-Paradigmas als Zielarchitektur ist dabei nicht beliebig. Local-First bündelt mehrere für die Migration KI-generierter Software zentrale Eigenschaften: Offline-Fähigkeit, Datensouveränität und eine vom Netzwerk entkoppelte, dauerhaft verfügbare Nutzung. Die Arbeit untersucht jedoch nicht sämtliche Aspekte dieses Paradigmas, sondern setzt einen besonderen Schwerpunkt auf die Synchronisierung lokal gespeicherter Daten. Damit soll veranschaulicht werden, dass die Migration in eine Desktop-Umgebung nicht auf den Umbau zu einer ausschließlich offline nutzbaren Anwendung beschränkt ist. Online-Funktionen können erhalten bleiben und mit lokaler Datenhaltung sowie einer auch ohne Netzwerk verfügbaren Kernfunktionalität verbunden werden.

1.4 Forschungsfrage

FF1: Wie können KI-generierte Webarchitekturen mithilfe eines framework-agnostischen Ansatzes systematisch in sichere Local-First-Desktop-Anwendungen überführt werden?

1.5 Aufbau der Arbeit

Der weitere Aufbau der Arbeit orientiert sich am Design-Science-Research-Prozess und gliedert sich in sieben weitere Kapitel. Kapitel 2 legt den theoretischen Hintergrund, indem es KI-gestützte Entwicklungswerkzeuge, die Besonderheiten von Web-zu-Desktop-Architekturen, das Local-First-Paradigma sowie sicherheitsrelevante Grundlagen einordnet und die bestehende Forschungslücke herausarbeitet. Kapitel 3 stellt den methodischen Rahmen der Arbeit vor, begründet die Wahl des Design-Science-Research-Ansatzes und skizziert den eigenen Forschungsprozess. Kapitel 4 widmet sich der Problemanalyse: Zunächst wird die Softwarelandschaft KI-gestützter Entwicklungswerkzeuge, etwa Ansätze wie Replit oder Lovable, betrachtet, bevor in die Tiefe des gewählten Ansatzes, der Replit-Umgebung und des daraus resultierenden Artefakts, eingestiegen wird. Kapitel 5 behandelt Entwurf

und Entwicklung des Artefakts, also die konzeptionelle Architektur und die praktische Umsetzung der Electron-Anwendung. Kapitel 6 stellt die Evaluation und Demonstration des entwickelten Migrationspfads anhand der Fallstudie in den Mittelpunkt. Kapitel 7 diskutiert die gewonnenen Erkenntnisse, reflektiert die verfolgten Lösungswege und ordnet die Ergebnisse im Vergleich zu bestehenden Ansätzen ein. Kapitel 8 fasst die zentralen Erkenntnisse zusammen und gibt einen Ausblick auf offene Fragestellungen und zukünftigen Forschungsbedarf.

Kapitel 2

Theoretischer Hintergrund

Dieses Kapitel erläutert die Begriffe und technischen Prinzipien, auf denen die Analyse und Migration aufbauen. Es verbindet Softwarequalität und Schichtentrennung mit vertragsbasierten Schnittstellen, dem Electron-Prozessmodell sowie lokaler Datenhaltung und Synchronisierung. Die konkrete Ausgangsarchitektur, Werkzeugwahl und Umsetzung werden in den nachfolgenden Kapiteln untersucht.

2.1 KI-gestützte Anwendungsentwicklung und Softwarequalität

Unter KI-gestützter Anwendungsentwicklung wird hier die Erzeugung und Überarbeitung von Anwendungscode anhand sprachlicher Anforderungen verstanden. Das resultierende Ausgangsartefakt umfasst ausführbaren Code, Konfigurationen und Abhängigkeiten. Seine sichtbare Funktionalität und seine technischen Eigenschaften sind getrennt zu betrachten: Ein erfolgreicher Oberflächenablauf zeigt zunächst, dass eine bestimmte Interaktion funktioniert. Ob Datenzugriffe angemessen begrenzt sind, Fehler behandelt werden und die Struktur spätere Änderungen ermöglicht, erfordert weitere Prüfungen.

Perry et al. untersuchten in einer Studie mit 47 Teilnehmenden fünf sicherheitsbezogene Programmieraufgaben. Teilnehmende mit Zugang zu einem auf `codex-davinci-002` basierenden KI-Assistenten erstellten in vier der fünf Aufgaben häufiger unsichere Lösungen als die Vergleichsgruppe. Zugleich schätzten sie ihre Lösungen häufiger als sicher ein [2]. Die Ergebnisse betreffen das untersuchte Modell, die Aufgaben und die damaligen Versuchsbedingungen. Sie begründen die Relevanz einer eigenständigen Sicherheitsprüfung, erlauben aber keine unmittelbare Qualitätsbewertung heutiger Replit- oder Lovable-Anwendungen.

Zur Beschreibung der benötigten Eigenschaften unterscheidet Requirements Engineering funktionale Anforderungen, Qualitätsanforderungen und Randbedingungen. Funktionale Anforderungen betreffen das verlangte Verhalten, etwa das Speichern eines Termins. Qualitätsanforderungen beziehen sich auf Eigenschaften wie Zuverlässigkeit oder Änderbarkeit. Randbedingungen begrenzen den Lösungsraum, beispielsweise durch eine vorgegebene Zieltechnologie [3]. Diese Unterscheidung hilft, gewünschte Funktionen von den Bedingungen ihrer Umsetzung und Nutzung zu trennen.

ISO/IEC 25010:2023 stellt ein Produktqualitätsmodell bereit, dessen Merkmale und Untermerkmale zur Spezifikation, Messung und Bewertung von Qualität verwendet werden können [4]. Für die vorliegende Arbeit stehen insbesondere Wartbarkeit, Testbarkeit und Sicherheit im Mittelpunkt. Die Trennung von Verantwortlichkeiten ist ein Entwurfsprinzip, das solche Qualitätsziele unterstützen soll. Die Wahl einer Schichtenarchitektur ersetzt deshalb nicht die Bewertung der tatsächlich entstandenen Anwendung.

2.2 Webarchitekturen und Trennung der Verantwortlichkeiten

In einer Client-Server-Webarchitektur fordert der Browser Ressourcen und Anwendungsdaten über HTTP an. Der Server verarbeitet die Anfrage und liefert eine Antwort zurück; bei dynamischen Anwendungen können dabei Fachlogik und Datenbankzugriffe ausgeführt werden [5]. Bei einer getrennten Weboberfläche übernimmt das Frontend insbesondere Darstellung und Benutzerinteraktion, während ein Backend fachliche Operationen und Datenzugriffe bereitstellt. Entscheidend für die Migration ist, welche Aufgaben tatsächlich an diese entfernte Laufzeit gebunden sind.

Eine Schichtenarchitektur ordnet Verantwortlichkeiten und Abhängigkeiten innerhalb der Anwendung. Eine verbreitete Aufteilung unterscheidet Darstellung, fachliche Verarbeitung und Datenzugriff. Logische Schichten sind dabei von physisch getrennten Ausführungseinheiten zu unterscheiden: Mehrere Schichten können innerhalb derselben Laufzeit liegen [6]. Ebenso belegt die Ablage in unterschiedlichen Projektordnern für sich genommen keine fachliche Schichtentrennung. Maßgeblich ist, welche Aufgaben die Module ausführen und welche anderen Module sie dafür aufrufen.

Für die Verarbeitung fachlicher Aufrufe werden in dieser Arbeit die Rollen Controller, Service und Repository verwendet. Ein Controller beziehungsweise Handler nimmt den Aufruf entgegen, ordnet Eingaben zu und übersetzt Ergebnis oder Fehler in die jeweilige Schnittstellenform. Ein Service stellt fachliche Operationen bereit und koordiniert ihre Verarbeitung. Das Service-Layer-Muster beschreibt hierfür eine Anwendungsgrenze, hinter der Geschäftslogik und zusammengehörige Aktionen organisiert werden [7]. Ein Repository kapselt den Zugriff auf gespeicherte Daten gegenüber der Fachlogik. Das Repository-Muster vermittelt zwischen fachlichen Objekten und ihrer Speicherung und konzentriert die zugehörigen Zugriffsoperationen [8].

Diese Rollen bilden keine zusätzliche Aufteilung in drei Betriebssystemprozesse. Der Controller gehört zur Verarbeitung einer Schnittstelle; Services und Repositories können gemeinsam in einer privilegierten Laufzeit ausgeführt werden. Ihre Trennung soll ermöglichen, den Transport oder die konkrete Datenbankanbindung gezielt anzupassen, ohne sämtliche fachlichen Regeln an diesen Stellen erneut zu implementieren. Werden Datenzugriffe und externe Dienste über austauschbare Abhängigkeiten eingebunden, lässt sich die Fachlogik mit Testimplementierungen prüfen. Ob diese Austauschbarkeit im konkreten Projekt erreicht wird, ist an seinen Modulen und Aufrufwegen zu untersuchen.

2.3 API-Verträge, Typisierung und Codegenerierung

Ein API-Vertrag beschreibt die angebotenen Operationen und die erwarteten Strukturen ihrer Eingaben und Ergebnisse. OpenAPI definiert hierfür eine programmiersprachenunabhängige Beschreibung von HTTP-APIs. Ein OpenAPI-Dokument kann unter anderem Pfade, Operationen, Parameter, Antwortformen und wiederverwendbare Schemata festlegen. Es kann als Grundlage für Dokumentation und die Erzeugung abhängiger Programmteile dienen [9]. Die Definition schafft eine gemeinsame Schnittstellengrundlage für die beteiligten Komponenten.

Codegenerierung leitet aus dieser Grundlage beispielsweise Typdefinitionen, Aufrufhilfen oder Validierungsschemata ab. Die Übersetzung besitzt dabei eigene Regeln: Der Generator muss bestimmen, wie Operationen benannt, Eingaben zugeordnet und Datentypen in der Zielsprache dargestellt werden. Änderungen am Vertrag können dadurch abgestimmt in mehreren abhängigen Dateien erscheinen. Fachliche Verarbeitung und Zugriffskontrollen bleiben weiterhin zu implementieren; der Generator erzeugt zunächst die durch seine Übersetzungsregeln festgelegten Bestandteile.

Statische Typisierung und Laufzeitvalidierung erfüllen unterschiedliche Aufgaben. TypeScript prüft Typbeziehungen während der Entwicklung und Übersetzung; seine Typannotationen werden bei der Erzeugung von JavaScript entfernt und verändern das Laufzeitverhalten nicht [10]. Eine Typdefinition gewährleistet daher nicht, dass eine tatsächlich empfangene Eingabe ihr entspricht. Bibliotheken wie

Zod prüfen Werte anhand ausführbarer Schemata und liefern bei ungültigen Eingaben strukturierte Validierungsfehler [11]. Auch eine strukturell gültige Eingabe muss anschließend fachlich zulässig sein, etwa hinsichtlich Wertebereich, Benutzerzuordnung und Berechtigung.

Für einen Transportwechsel ist deshalb zwischen der Bedeutung einer Operation und ihrer bisherigen HTTP-Abbildung zu unterscheiden. Ein vorhandener OpenAPI-Vertrag kann als Ausgangspunkt einer angepassten Generierung dienen. Seine Übertragung auf IPC ist jedoch eine projektspezifische Interpretation und keine von OpenAPI standardisierte IPC-Schnittstelle. Zu klären sind dabei insbesondere Datendarstellung, Fehlerformen und die Zuordnung der Operationen zum neuen Aufrufweg.

2.4 Electron als Desktop-Laufzeit

Electron übernimmt ein Mehrprozessmodell aus Chromium und verbindet die Darstellung von Webinhalten mit einer Node.js-Laufzeit. Der Main-Prozess bildet den Einstiegspunkt der Anwendung, verwaltet ihren Lebenszyklus und erzeugt Fenster. Er kann Node.js- sowie native Electron-APIs verwenden. Renderer-Prozesse stellen die Webinhalte der Fenster dar und verarbeiten die zugehörige Oberfläche [12]. Dadurch kann eine vorhandene Weboberfläche die Grundlage einer Desktop-Anwendung bilden, während privilegierte Aufgaben gesondert zugeordnet werden.

Ein Preload-Skript wird im Renderer-Prozess vor den eigentlichen Webinhalten ausgeführt. Es ist kein eigenständiger dritter Prozess. Bei aktivierter Context Isolation besitzen Preload und geladene Oberfläche getrennte JavaScript-Kontexte. Über `contextBridge` können ausgewählte Funktionen für die Oberfläche bereitgestellt werden. Die Electron-Dokumentation empfiehlt begrenzte Methoden statt einer ungefilterten Weitergabe mächtiger Schnittstellen wie `ipcRenderer.send` [13]. Die Bridge bestimmt damit, welche Fähigkeiten die Oberfläche aufrufen kann.

Interprozesskommunikation (IPC) vermittelt Nachrichten zwischen den beteiligten Prozessen. Electron unterstützt unter anderem einen Anfrage-Antwort-Aufruf, bei dem der Renderer eine Operation anstößt und ein Ergebnis aus dem Main-Prozess erhält [14]. Für fachliche Schnittstellen können solche Aufrufe mit Operationskennungen, Eingaben und definierten Ergebnissen versehen werden. Der Main-Prozess muss die erlaubten Absender, Operationen und Daten prüfen, bevor privilegierte Verarbeitung erfolgt.

Die technische Prozessstruktur und die fachlichen Schichten sind somit unabhängig voneinander zu beschreiben. Der Renderer kann die Darstellung übernehmen, während Controller, Services und Repositories im Main-Prozess liegen. Preload und IPC bilden den Kommunikationsweg zwischen diesen Zuständigkeiten. Eine solche Zuordnung macht die privilegierten Eingangsstellen sichtbar; ihre Begrenzung und Absicherung sind eigene Entwurfs- und Prüfaufgaben.

2.4.1 Navigation über Hashrouting

Bei einer Single-Page-Anwendung bestimmt ein clientseitiger Router, welche Ansicht innerhalb des geladenen Dokuments dargestellt wird. Hashrouting legt die Anwendungsroute im Fragment der Adresse hinter `#` ab, beispielsweise als `#/kalender`. Das Fragment wird clientseitig verarbeitet und bei einer HTTP-Anfrage nicht an den Server übermittelt [15]. Der Ressourcenpfad des Dokuments bleibt dadurch unverändert, während sich die ausgewählte Ansicht ändern kann.

Dies ist für eine lokal geladene Electron-Oberfläche eine mögliche Navigationsstrategie: Eine fachliche Route muss nicht als eigener Dateipfad aufgelöst werden. Wouter stellt dafür den Hook `useHashLocation` bereit [16]. Hashrouting ist keine zwingende Vorgabe von Electron; seine Eignung hängt vom Laden der Renderer-Ressourcen und der gewählten Navigation ab.

2.5 Lokale Persistenz, Schema und Identität

Persistenz bezeichnet den Erhalt von Daten über die aktuelle Programmausführung hinaus. Eine lokal ausgeführte Funktion ist daher erst dann dauerhaft nutzbar, wenn ihre Ergebnisse gespeichert und nach einem Neustart wieder gelesen werden können. SQLite stellt hierfür eine eingebettete SQL-Datenbank bereit. Die Bibliothek benötigt keinen eigenen Datenbankserver und liest beziehungsweise schreibt Datenbankdateien direkt [17]. Das Öffnen, Initialisieren und Aktualisieren dieses Datenbestands wird damit Teil der Anwendungszuständigkeit.

Eine Datenzugriffsbibliothek beziehungsweise ein Object-Relational Mapper (ORM) vermittelt zwischen Programmstrukturen und Datenbankoperationen. Seine Beibehaltung kann die Weiterverwendung von Abfragen und Schemadefinitionen unterstützen. Ein Wechsel des Datenbanksystems erfordert trotzdem einen Abgleich von Datentypen, Schlüsseln, Beziehungen und Prüfbedingungen. Gleiche Tabellen- oder Typnamen gewährleisten keine unveränderte Bedeutung ihrer Speicherung.

Versionierte Schemamigrationen beschreiben die aufeinanderfolgenden Änderungen am Datenbankschema. Drizzle Kit kann dazu aus Schemadefinitionen SQL-Migrationsdateien erzeugen [18]. Die Aktualisierung einer Anwendung muss unterscheiden, ob ein neuer Datenbestand eingerichtet oder ein bereits vorhandener Bestand weiterentwickelt wird. Programmdateien und veränderliche Benutzerdaten besitzen entsprechend unterschiedliche Lebenszyklen.

Mit lokaler Datenhaltung muss außerdem die Zuordnung der Datensätze zu einer Identität festgelegt werden. Eine lokale Benutzerkennung, eine aktive Sitzung und ein entferntes Konto erfüllen unterschiedliche Aufgaben. Die lokale Zuordnung hält persönliche Datensätze konsistent; eine entfernte Authentifizierung ermöglicht den Zugriff auf einen gemeinsamen Dienst. Für synchronisierte Daten müssen Kennungen auch bei zunächst getrennten lokalen Änderungen eindeutig erzeugt und erhalten werden können. Diese Identitätsentscheidungen betreffen Beziehungen und Zugriffskontrollen und sind deshalb Bestandteil der Architektur.

2.6 Offline-Nutzung und Local-First-Synchronisierung

Offline-Fähigkeit beschreibt hier die Ausführbarkeit der ausgewählten Funktionen ohne aktuelle Netzwerkverbindung. Dafür müssen benötigter Anwendungscode und Daten lokal verfügbar sein. Ein System kann solche Funktionen anbieten und zugleich andere Operationen besitzen, die unmittelbar eine Verbindung benötigen. Der Begriff legt für sich genommen noch keine Strategie für gemeinsame Daten oder konkurrierende Änderungen fest.

Kleppmann et al. beschreiben Local-First als Zielbild, das Zusammenarbeit mit lokaler Nutzbarkeit und Kontrolle über die eigenen Daten verbindet. Zu seinen Idealen gehören unter anderem Offline-Nutzung, geräteübergreifender Zugriff, langfristige Verfügbarkeit und Sicherheit beziehungsweise Privatheit. Die lokale Kopie nimmt dabei eine zentrale Rolle ein; Server können die Verteilung unterstützen [19]. Local-First umfasst damit weitergehende Ziele als die Verwendung einer lokalen Datenbank. Welche Eigenschaften eine konkrete Anwendung umsetzt, ist jeweils auszuweisen.

Für externe Datenflüsse ist zwischen Übernahme und Synchronisierung zu unterscheiden. Bei einer Übernahme in einer Richtung werden Referenzdaten heruntergeladen, geprüft und lokal gespeichert. Eine Zusammenarbeit mit gemeinsamen Daten benötigt zusätzlich die Übermittlung lokaler Änderungen und die Verarbeitung entfernter Ergebnisse. Lokaler Schreibabschluss und entfernter Abgleich finden dabei zu unterschiedlichen Zeitpunkten statt.

PowerSync stellt für eine solche Anbindung einen lokalen Datenbestand und eine Upload-Warteschlange bereit. Die Anwendung übermittelt aufgezeichnete Änderungen über einen Connector an ihr Backend. PowerSync verwendet ein serverautoritäres Modell: Die entfernte Verarbeitung bestimmt den anschließend bestätigten Datenbestand [20]. Eine lokal abgeschlossene Änderung ist deshalb von ihrer

Annahme durch das Backend zu unterscheiden. Der Einsatz dieser Lösung kann lokale Verarbeitung mit ergänzendem Abgleich verbinden, ohne automatisch sämtliche Ideale des weitergehenden Local-First-Zielbilds zu erfüllen.

Die zeitliche Trennung erfordert ein geregeltes Fehlerverhalten. Vorübergehende Verbindungsprobleme, endgültige Ablehnungen und unklare Ergebnisse nach einer verlorenen Antwort benötigen unterschiedliche Behandlung. Für wiederholbare Aufrufe ist Idempotenz relevant: RFC 9110 beschreibt sie für HTTP-Methoden als gleiche beabsichtigte Wirkung mehrerer identischer Anfragen und einer einzelnen Anfrage [21]. Dieses Prinzip ist bei der Gestaltung wiederholbarer Upload-Operationen zu berücksichtigen. Davon zu unterscheiden sind fachliche Konflikte zwischen neuen konkurrierenden Bearbeitungen, für die die Anwendung eigene Regeln benötigt. Die Wahl einer Synchronisierungslösung beantwortet diese Fragen nur im Umfang ihres jeweiligen Verarbeitungsmodells.

2.7 Sicherheitsgrenzen, Härtung und Nachweise

Ein Sicherheitsentwurf muss festlegen, welche Komponenten welchen Eingaben vertrauen dürfen und welche Fähigkeiten ihnen zur Verfügung stehen. Saltzer und Schroeder formulieren dafür unter anderem das Prinzip geringster Privilegien: Programme und Benutzer sollen nur die für ihre Aufgaben benötigten Rechte besitzen. Hinzu kommen sichere Vorgaben und die vollständige Prüfung von Zugriffen [22]. Für eine Desktop-Anwendung betrifft dies insbesondere die Stellen, an denen Oberflächenaktionen privilegierte Datei-, Datenbank- oder Dienstoperationen auslösen.

Die Electron Security Guidelines konkretisieren solche Schutzaufgaben für die Laufzeit. Sie behandeln unter anderem die Begrenzung von Node.js-Zugriffen im Renderer, Context Isolation, Sandboxing, Berechtigungen, Inhalts- und Navigationsbeschränkungen sowie die Validierung von IPC-Absendern. Die Dokumentation weist ausdrücklich auf die erhöhten Folgen hin, wenn fremder JavaScript-Code Zugriff auf lokale Systemfähigkeiten erhält [23]. Die Übernahme einer Weboberfläche erfordert daher eine Prüfung der verfügbaren Fähigkeiten und ihrer Schnittstellen.

Härtung bezeichnet in dieser Arbeit die gezielte Begrenzung dieser Zugriffsmöglichkeiten und die Umsetzung der anwendbaren Schutzmaßnahmen. Eingaben sind an den jeweiligen Grenzen zu prüfen: Eine IPC-Nachricht, eine importierte Datei und eine Netzwerkantwort besitzen unterschiedliche Herkunft und Verarbeitungspfade. Formatprüfung, fachliche Validierung und Berechtigungsentscheidung ergänzen sich. Die zu prüfenden Eigenschaften müssen auf die tatsächliche Anwendung und ihren festgelegten Sicherheitsumfang bezogen werden.

Auch die Art des Nachweises ist zu unterscheiden. Codeinspektion zeigt eine vorhandene Implementierung oder Konfiguration. Ein Unit-Test prüft ausgewählte Entscheidungen unter festgelegten Bedingungen. Ein Laufzeitversuch beobachtet einen konkreten Ablauf; ein Paketnachweis bezieht zusätzlich die ausgelieferten Ressourcen und Abhängigkeiten ein. Für eine nachvollziehbare Bewertung müssen Gegenstand, Prüfstand und Reichweite dieser Belege angegeben werden. Aus einem erfolgreichen Einzeltest folgt keine allgemeine Aussage über die Abwesenheit von Schwachstellen.

2.8 Bestehende Ansätze und Einordnung der Arbeit

Die beschriebenen Ansätze liefern unterschiedliche Grundlagen für die Migration. Schichtenarchitekturen und die Muster Service Layer und Repository ordnen fachliche Verantwortlichkeiten. OpenAPI unterstützt die Beschreibung und abgestimmte Ableitung von Schnittstellenbestandteilen. Die Electron-Dokumentation behandelt Prozesskommunikation und laufzeitspezifische Schutzmaßnahmen; die Local-First-Literatur beschreibt Ziele für lokale Nutzung und Zusammenarbeit.

Für die Veränderung bestehender Systeme beschreibt Fowler mit Strangler Fig eine schrittweise Modernisierung, bei der abgegrenzte Funktionen nach und nach in eine neue Struktur überführt werden. Übergangsarchitekturen können die parallele Nutzung alter und neuer Komponenten ermöglichen [24].

Refactoring ist davon begrifflich zu unterscheiden: Es verbessert die interne Struktur von Code bei unverändertem beobachtbarem Verhalten [3]. Eine Migration kann solche Strukturänderungen mit Portierungen, Datenüberführungen und zusätzlichen Funktionsanpassungen verbinden.

Für die vorliegende Fragestellung werden diese Grundlagen zu einem fallbezogenen Vorgehen zusammengeführt. Zu untersuchen ist, welche Bestandteile eines KI-generierten Webartefakts erhalten bleiben können, welche Zuständigkeiten die lokale Anwendung übernimmt und wie zusätzliche Netzwerkanbindungen begrenzt integriert werden. Der frameworkagnostische Anspruch betrifft dabei die Einordnung der Aufgaben und ihrer Schnittstellen. Die konkrete Umsetzung und die Grenzen ihrer Übertragbarkeit werden anhand der Fallstudie untersucht. Kapitel 3 erläutert das methodische Vorgehen; Kapitel 4 konkretisiert Ausgangsartefakt, Ziele und Anforderungen.

Kapitel 3

Methodisches Vorgehen

3.1 Forschungsansatz und Erkenntnisziel

Die Arbeit verfolgt einen gestaltungsorientierten Forschungsansatz. Im Mittelpunkt stehen die Entwicklung eines Migrationsleitfadens und dessen Anwendung auf den KI-generierten Webprototyp Studienplaner. Als Forschungsrahmen dient Design Science Research. Das Prozessmodell nach Peffers et al. unterscheidet Problemidentifikation und Motivation, Definition der Lösungsziele, Entwurf und Entwicklung, Demonstration, Evaluation sowie Kommunikation [25].

Die Zuordnung zu diesem Rahmen strukturiert die wissenschaftliche Darstellung des tatsächlich durchgeführten Vorgehens. Sie bedeutet nicht, dass sämtliche technischen Schritte bereits zu Beginn festgelegt waren. Die Migration wurde schrittweise entwickelt; Erkenntnisse aus der Analyse und Umsetzung führten zu Anpassungen der Architektur und des weiteren Vorgehens. Die nachfolgend beschriebenen Phasen fassen diesen Prozess rückblickend zusammen.

Das Erkenntnisziel besteht darin, die für die Migration notwendigen Veränderungen nachvollziehbar zu beschreiben und daraus einen fallbezogenen Leitfaden abzuleiten. Die Electron-Anwendung dient als technischer Demonstrator. Der Leitfaden dokumentiert die Voraussetzungen, Entscheidungen, Transformationsschritte und Kontrollpunkte der Migration. Aussagen über seine Übertragbarkeit werden auf den untersuchten Fall begrenzt.

3.2 Untersuchungsgegenstand und Analyseverfahren

Ausgangspunkt war der auf Replit erzeugte Studienplaner. Wie in Kapitel 1 erläutert, erfolgte dessen Erstellung anhand funktionaler Beschreibungen ohne verbindliche Vorgaben zur technischen Architektur. Untersucht wurde damit ein KI-generiertes Ausgangsartefakt, dessen Komponenten für eine lokale Desktop-Laufzeit neu eingeordnet werden mussten.

Die beiden ursprünglichen Erstellungs-Prompts sind in Abschnitt A.1 und Abschnitt A.2 vollständig dokumentiert. Sie beschreiben in überwiegend alltagssprachlicher Form zunächst die persönliche Studienorganisation und anschließend die Zusammenarbeit in Studiengruppen. Neben funktionalen Wünschen nennen sie Anforderungen an Anmeldung und Zugriffsschutz, legen jedoch weder ein UI-Framework noch eine konkrete Backend- oder Datenbanktechnologie fest.

Die Untersuchung erfolgte anhand des Quellcodes, der Projektkonfiguration und der vorhandenen technischen Dokumentation. Dabei wurden Oberfläche, Schnittstellen, Geschäftslogik, Datenhaltung und externe Abhängigkeiten erfasst. Für die Migration war insbesondere zu klären, welche Funktionen unmittelbar von einem Backend oder einer Netzwerkverbindung abhingen, welche Bestandteile lokal weiterverwendet werden konnten und an welchen Stellen eine Anpassung der Verantwortlichkeiten erforderlich war.

Ergänzend wurde die offizielle Plattformdokumentation herangezogen, um die Wahl des Untersuchungsgegenstands einzuordnen. Replit und Lovable wurden hierzu beispielhaft hinsichtlich ihrer dokumentierten Backend-Ansätze betrachtet. Diese Betrachtung dient der Begründung des Untersuchungsumfangs; sie ist keine empirische Vergleichsstudie der Plattformen. Die Ergebnisse und der daraus abgeleitete Anforderungskatalog werden in Kapitel 4 dargestellt.

3.3 Vorversuch und Konsequenzen für das Migrationsvorgehen

Vor der Entwicklung des in dieser Arbeit untersuchten Demonstrators wurde in einem separaten Repository ein erster Migrationsversuch mit LLM-Unterstützung durchgeführt. Ausgangs- und Zielprojekt wurden dem LLM als Arbeitsgrundlage bereitgestellt. Die Überführung sollte schrittweise erfolgen, beginnend mit der Datenbank und anschließend den Schnittstellen und der Oberfläche. Die Datenbankübertragung ließ sich dabei ohne wesentliche Schwierigkeiten durchführen. Die Schnittstellen wurden zunächst manuell angepasst, ohne den vorhandenen API-Vertrag als gemeinsame Grundlage für die Generierung der Schnittstellenbausteine weiterzuführen.

Bei der weiteren Bearbeitung wurden vorhandene Dateien und ihre Fachlogik durch das LLM gelesen und in der Zielstruktur neu geschrieben. Dabei entstanden auch kleinere Änderungen, deren Notwendigkeit für die Migration nicht erkennbar war. Beobachtet wurden unter anderem Änderungen von Texteingaben zu numerischen Formularfeldern, Anpassungen der Datumsverarbeitung und zusätzliche Platzhalter in Formularen. Solche Eingriffe erschwerten es, notwendige Anpassungen an die Desktop-Umgebung von unbeabsichtigten Veränderungen des Anwendungsverhaltens zu unterscheiden. Die manuelle Bearbeitung mehrerer Schnittstellenmodule erschwerte außerdem die Nachverfolgung von Datentypen und Konvertierungen über den gesamten Verarbeitungspfad. Dadurch war weniger übersichtlich, ob der übernommene Funktionsumfang und das Verhalten weiterhin dem Ausgangsartefakt entsprachen.

Diese Erfahrungen flossen in das überarbeitete Vorgehen ein. Vorhandene Oberflächenkomponenten und fachliche Regeln sollten gezielt weiterverwendet und Änderungen auf die Anforderungen der Zielumgebung zurückgeführt werden. Für die Kernschnittstellen wurde der vorhandene OpenAPI-Vertrag beibehalten; Typen, Aufrufhilfen und Laufzeitschemata wurden daraus mit einer auf IPC angepassten Generierung abgeleitet. Die architektonische Trennung sollte zugleich die Zuordnung und Prüfung der erforderlichen Änderungen erleichtern. Der Vorversuch erklärt damit die Entwicklung dieser Entscheidungen. Seine Darstellung beruht auf rückblickend eingeordneten Beobachtungen des Entwicklungsverlaufs und stellt keine kontrollierte Vergleichsevaluation der beiden Vorgehensweisen dar.

3.4 Ablauf der Entwicklung

Die technische Entwicklung gliederte sich in drei aufeinander aufbauende Phasen. Diese sind Arbeitsschritte innerhalb der Entwurfs- und Entwicklungsaktivität des Forschungsrahmens und bilden kein eigenständiges Forschungsmodell.

3.4.1 Phase 1: Überführung in eine lokal nutzbare Offline-Anwendung

Zunächst wurden die eingeschlossenen Kernfunktionen von der ursprünglichen Server-Laufzeit entkoppelt. Hierzu wurden die Komponenten auf eine Desktop-Architektur mit getrennten Verantwortlichkeiten für Darstellung, Geschäftslogik und Datenzugriff abgebildet. Die Datenhaltung wurde lokal bereitgestellt. Aufrufe der bisherigen Webschnittstellen wurden in einen begrenzten Kommunikationsweg zwischen Renderer, Preload und Main-Prozess überführt.

Ziel dieser Phase war die Entwicklung einer Electron-Anwendung, deren lokale Kernfunktionen ohne Netzwerkverbindung ausführbar sind und deren Architektur zugleich die Grundlage für die nachfolgenden Erweiterungen bildet. Die Verantwortlichkeiten und Schnittstellen wurden so gestaltet,

dass Synchronisierung und weitere externe Dienste später integriert werden können, ohne die lokale Kernarchitektur erneut grundlegend umzustrukturieren. Damit verbindet diese Phase die Herstellung lokaler Nutzbarkeit mit der Vorbereitung einer schrittweise erweiterbaren Anwendung.

3.4.2 Phase 2: Ergänzung um Synchronisierung und weitere Online-Anbindungen

Aufbauend auf der lokal ausführbaren Kernanwendung wurden in der zweiten Phase die netzwerkabhängigen Funktionen betrachtet. Diese waren bei der Herstellung der Offline-Kernanwendung zunächst ausgeklammert worden. Ziel war es, externe Datenquellen und Dienste schrittweise in die vorbereitete Desktop-Architektur einzubinden und dadurch den angestrebten Funktionsumfang zu vervollständigen.

Die Wiederanbindung wurde anhand zweier unterschiedlich komplexer Fälle untersucht. Der Abruf eines Studiengangkatalogs dient als Beispiel für das Herunterladen externer Daten und deren Übernahme in die lokale Anwendung. Dieser Vorgang umfasst keine Synchronisierung und veranschaulicht zunächst einen einfachen Netzwerkzugriff. Darauf aufbauend wurde mit der gemeinsamen Aufgabenverwaltung in Studiengruppen ein Kollaborationsfall betrachtet. Hier werden lokal verarbeitete Änderungen über eine zusätzliche Synchronisierungsinfrastruktur zwischen den Beteiligten ausgetauscht. Dieser zweite Fall greift gezielt Eigenschaften des Local-First-Paradigmas auf.

3.4.3 Phase 3: Konsolidierte Prüfung und Härtung

Nach der funktionalen Umsetzung wurden die entstandenen lokalen und netzwerkbezogenen Schnittstellen systematisch hinsichtlich der Electron-Sicherheitsrichtlinien geprüft und erforderliche Anpassungen vorgenommen. Der Sicherheitsbegriff wird für diese Arbeit über die Umsetzung der anwendbaren Electron Security Guidelines einschließlich Eingabevalidierung und Maßnahmen gegen die Ausführung fremden Codes operationalisiert [23].

Diese abschließende Phase bündelt die Prüfung der fertigen Anwendung. Sicherheitsrelevante Architekturentscheidungen, insbesondere die Trennung privilegierter Funktionen vom Renderer und die Begrenzung der Preload-Schnittstellen, betreffen bereits die vorausgehenden Phasen. Die Darstellung einer abschließenden Härtung bedeutet daher nicht, dass Sicherheit erst nach Abschluss der Entwicklung berücksichtigt wird.

3.5 Ableitung des Leitfadens und Bewertung

Aus dem dokumentierten Migrationsprozess werden die Schritte des Leitfadens abgeleitet. Für jeden Schritt werden Ausgangszustand, Ziel, notwendige Änderungen und ein Kontrollpunkt beschrieben. Dabei werden architektonische Prinzipien von Anpassungen unterschieden, die an den konkreten Technologieeinsatz der Fallstudie gebunden sind. Die technischen Schritte und ihre Zuordnung zum Demonstrator werden in Kapitel 5 erläutert.

Die Bewertung orientiert sich am Anforderungskatalog aus Kapitel 4. Architektur- und Quellcodeinspektionen, vorhandene Service-Unit-Tests sowie begrenzte manuelle Demonstrationen bilden die vorgesehenen Nachweisformen. Für den Sicherheitsnachweis werden Anwendbarkeit, Umsetzung und Beleg jeder Electron-Empfehlung getrennt erfasst. Bereits vorliegende Sicherheitsberichte werden mit ihrem jeweiligen Prüfstand und ihren Nachweisgrenzen berücksichtigt.

Die Festlegung der Nachweise ist von den Ergebnissen ihrer Durchführung zu unterscheiden. Aus der Beschreibung des Verfahrens folgt keine pauschale Erfüllung der Anforderungen. Die Ergebnisdarstellung erfolgt in Kapitel 6. Ein weiterer Migrationsdurchlauf zur unabhängigen Erprobung des Leitfadens ist nicht Bestandteil der Arbeit. Die Bewertung stützt sich auf den dokumentierten Prozess und den vorhandenen Demonstrator; eine empirisch bestätigte Übertragbarkeit auf weitere Plattformen wird daraus nicht abgeleitet.

Kapitel 4

Analyse und Anforderungsdefinition

4.1 Vom übergeordneten Ziel zur Fallauswahl

Die in Kapitel 1 formulierte Forschungsfrage nimmt einen frameworkagnostischen Migrationsansatz als Ausgangsziel auf. Für dessen Bearbeitung ist zunächst zu bestimmen, welche Architekturmerkmale des Ausgangsartefakts die Transformation beeinflussen. Entscheidend ist nicht allein das verwendete Frontend-Framework, sondern auch, wo Geschäftslogik, Datenhaltung und Authentifizierung ausgeführt werden und welche dieser Bestandteile im verfügbaren Quellcode enthalten sind.

Zur Einordnung werden Replit und Lovable anhand ihrer offiziellen Dokumentation beispielhaft betrachtet. Replit beschreibt Webanwendungen als standardmäßig erzeugte Full-Stack-Anwendungen mit Frontend und Backend. API-Routen, Datenbank und serverseitige Logik werden nach dem Bedarf der Anwendung eingerichtet. Diese Dokumentation beschreibt das Plattformangebot; den konkreten Aufbau des Studienplaners belegt erst die Analyse des vorliegenden Artefakts [26].

Lovable dokumentiert dagegen einen standardmäßig aktivierten verwalteten Cloud-Backend-Dienst auf Grundlage der Open-Source-Technologie von Supabase. Alternativ kann ein eigenes Supabase-Projekt angebunden werden. Datenbank, Authentifizierung, Dateispeicherung und serverseitige Funktionen werden damit als Backend-Dienste bereitgestellt. Diese dokumentierte Dienststruktur wird hier als Backend-as-a-Service eingeordnet. Daraus folgt jedoch nicht, dass jedes Lovable-Projekt dieselbe Architektur aufweist [27].

Für die Migration ergibt sich daraus eine unterschiedliche Ausgangslage. Liegt die serverseitige Anwendungslogik als eigener Quellcode vor, kann ihre Übernahme und Anpassung an die lokale Laufzeit untersucht werden. Bei einer dienstgebundenen Architektur müssen dagegen auch die vom Dienst übernommenen Verantwortlichkeiten identifiziert und gegebenenfalls lokal nachgebildet werden. Eine reine Ersetzung des Frontend-Transports deckt diese zweite Ausgangslage nicht vollständig ab.

Die Entwicklung und Prüfung eines allgemeinen Leitfadens für beide Fälle würde den Umfang dieser Arbeit überschreiten. Die praktische Untersuchung wird daher auf den Replit-Prototyp Studienplaner begrenzt. Lovable dient zur dokumentationsgestützten Begründung dieser Eingrenzung; eine Lovable-Anwendung wird nicht migriert. Der resultierende Leitfaden kann potenziell übertragbare Prinzipien aufzeigen, wird jedoch nicht als validierte Lösung für beliebige Frameworks oder Plattformen ausgegeben.

4.2 Funktionen und Architektur des Ausgangsartefakts

4.2.1 Funktionaler Untersuchungsumfang

Der funktionale Ausgangspunkt ergibt sich aus den ursprünglichen Erstellungs-Prompts zur persönlichen Studienorganisation und zur Zusammenarbeit in Studiengruppen. Beide Prompts sind in Anhang A dokumentiert. Für die Migration werden sämtliche persönlichen Funktionsbereiche sowie Authentifizierung und ausgewählte Gruppenfunktionen betrachtet. Tabelle 4.1 konkretisiert diesen Untersuchungsumfang. Die Aufstellung bildet den funktionalen Bezugsrahmen; maßgeblich für den Demonstrator ist der in Abschnitt 4.3 definierte repräsentative Kernfunktionsumfang.

Bereich	**Betrachtete Funktionen**
Dashboard	Notendurchschnitt, gesammelte ECTS-Punkte, anstehende Fristen, Schnellzugriffe sowie letzte Aktivitäten und nächste Termine.
Kalender	Monatsansicht mit Tages-Popover, Termindetails und Löschen, farbliche Kennzeichnung der Terminarten, iCal-/ICS-Import, mehrtägige Termine und nach Priorität filterbare Listenansicht.
Noten	Semesterweise gruppierte Kurs-, Noten- und ECTS-Tabelle; Anlegen, Lesen, Bearbeiten und Löschen; CSV-Import mit Spaltenerkennung; ECTS-gewichteter Notendurchschnitt und Prüfung der Kurszugehörigkeit zum Studiengang.
Stundenplan	Raster mit fünf Wochentagen und sechs festen Hochschulzeitblöcken, Auswahl vorbereiteter Hochschulpläne, Hinzufügen und Entfernen von Kursen, visuelle Unterscheidung vorgeschlagener und eigener Einträge sowie Leeren des Plans mit Bestätigung.
Authentifizierung	Registrierung, Anmeldung und Abmeldung sowie die Zuordnung persönlicher Daten zur jeweiligen Identität.
Studiengruppen	Anlegen benannter Gruppen und Einladen weiterer Mitglieder; gemeinsame Aufgaben mit Fälligkeit, zuständiger Person und Status „offen“ beziehungsweise „erledigt“.

Tabelle 4.1: Funktionaler Untersuchungsumfang auf Grundlage der ursprünglichen Erstellungs-Prompts.

Im Webprototyp wird die angemeldete Sitzung über Session-Cookies vermittelt. Bei der Migration sind die lokale Zuordnung persönlicher Daten und die für die Zusammenarbeit benötigte entfernte Authentifizierung gesondert zu betrachten. Die konkrete Umsetzung der Identitäts- und Sitzungsverwaltung wird in Kapitel 5 beschrieben.

Der Kollaborationsumfang wird auf Gruppenerstellung, Einladungen und gemeinsame Aufgaben begrenzt. Kommentare, gemeinsame Kalendertermine sowie zusätzliche Gruppeninhalte wie Links und Notizen werden nicht untersucht. Die Sichtbarkeit gemeinsamer Aufgaben bleibt auf die jeweiligen Gruppenmitglieder begrenzt; persönliche Noten und Termine werden durch die Gruppenzugehörigkeit nicht automatisch geteilt.

4.2.2 Technische Komponenten und Repository-Struktur

Der vorliegende Replit-Prototyp enthält eine React-Oberfläche, einen eigenständigen Express-API-Server, einen durch OpenAPI beschriebenen API-Vertrag sowie eine über Drizzle angebundene

PostgreSQL-Datenhaltung. Oberfläche, Backend und gemeinsam genutzte Bibliotheken sind im Projekt getrennt abgelegt. Die Datenbankanbindung erwartet eine konfigurierte entfernte Datenbankverbindung. Damit liegt im untersuchten Fall serverseitiger Quellcode vor, der für die Migration analysiert und angepasst werden kann [28].

Die Schnittstellen zwischen Frontend und Backend werden in der Datei `lib/api-spec/openapi.yaml` als gemeinsamer API-Vertrag beschrieben. Dieser legt die Endpunkte sowie die Strukturen der Anfragen und Antworten fest. Auf dieser Grundlage erzeugt Orval API-Aufrufhilfen, React-Query-Hooks und Typdefinitionen für das Frontend sowie Zod-Schemata für die Validierung im Backend. Die generierten Bestandteile werden über gemeinsam genutzte Projektbibliotheken eingebunden. Der OpenAPI-Vertrag bildet damit die gemeinsame Grundlage für die Schnittstellendefinition auf beiden Seiten und wird bei der späteren Anpassung des Kommunikationswegs erneut aufgegriffen [28].

Die Oberfläche in `artifacts/studienplaner/` verwendet React und Vite sowie Tailwind CSS und shadcn/ui für die Darstellung. Der Express-Server liegt in `artifacts/api-server/` und wird mit esbuild als ESM-Anwendung gebündelt. Die gemeinsam genutzten Bibliotheken sind unter `lib/` zusammengefasst: `db/` enthält das Drizzle-Schema und den PostgreSQL-Client, `api-spec/` den OpenAPI-Vertrag, `api-zod/` die daraus erzeugten Validierungsschemata und `api-client-react/` die mit Orval generierten React-Query-Aufrufhilfen. `artifacts/mockup-sandbox/` dient der Gestaltung und Erprobung von Oberflächen und wird nicht als fachlicher Bestandteil der Migration betrachtet [28].

Abbildung 4.1 stellt die Komponenten und ihre Beziehungen dar. Die durchgezogenen Pfeile zeigen den Kommunikationsweg zur Laufzeit: Die React-Oberfläche ruft den Express-API-Server über HTTP auf; dessen Route-Handler greifen über Drizzle und den PostgreSQL-Client auf die Datenbank zu. Die gestrichelten Pfeile zeigen die Codegenerierung und Einbindung der aus dem OpenAPI-Vertrag erzeugten Bestandteile. Der Vertrag selbst ist damit eine Grundlage der Entwicklung und kein zusätzlicher Vermittler im Laufzeitpfad.

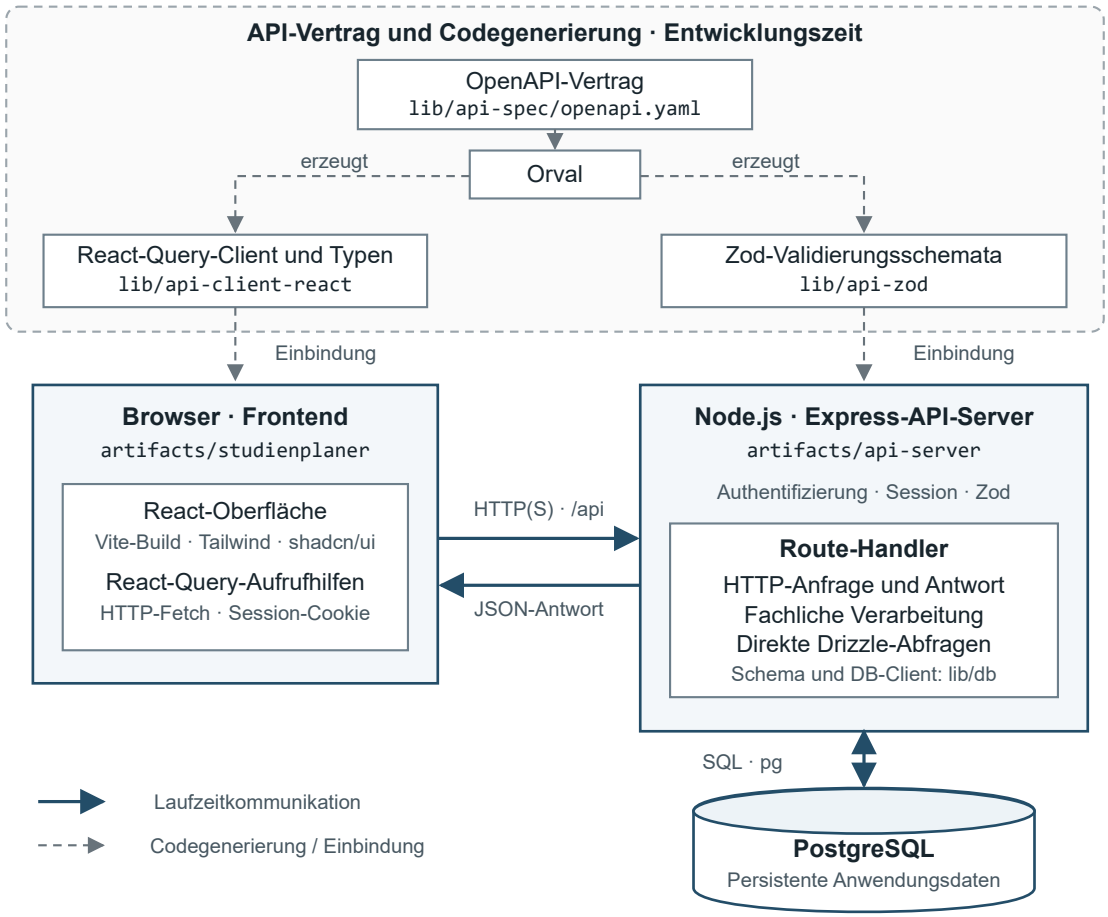


Abbildung 4.1: Ursprüngliche Webarchitektur des Studienplaners mit Laufzeitkommunikation und OpenAPI-basierter Codegenerierung; eigene Darstellung auf Grundlage des Ausgangsartefakts.

Diese Projektaufteilung ist noch kein Nachweis einer durchgängig getrennten Dreischichtenarchitektur. Für die Transformation sind die tatsächlichen Verantwortlichkeiten der Module und ihre Abhängigkeiten maßgeblich. Das in Quellcode 4.1 gezeigte Beispiel veranschaulicht das im API-Server verwendete Verarbeitungsmuster innerhalb eines Route-Handlers.

```
router.get("/dashboard/upcoming", requireAuth, async (req, res): Promise<void> => {
  const userId = req.session.userId!;
  const now = new Date();
  const in7Days = new Date(now.getTime() + 7 * 24 * 60 * 60 * 1000);

  const events = await db
    .select()
    .from(eventsTable)
    .where(
      and(
        eq(eventsTable.userId, userId),
        gte(eventsTable.startDate, now),
        lte(eventsTable.startDate, in7Days),
      ),
    )
    .orderBy(eventsTable.startDate);

  res.json(events);
});
```

Quellcode 4.1: Exemplarischer Express-Route-Handler für die Abfrage anstehender Termine im Ausgangsartefakt.

Der exemplarische Route-Handler verbindet HTTP-Verarbeitung, fachliche Auswahl und Datenzugriff innerhalb einer Funktion. Die Projektaufteilung in Frontend, API-Server und gemeinsame Bibliotheken entspricht daher noch keiner durchgängigen Trennung fachlicher Verantwortlichkeiten.

Das zentrale Migrationsproblem besteht darin, die bisher servergebundenen Funktionen in die lokale Desktop-Laufzeit zu überführen. Dabei sind insbesondere der Kommunikationsweg, die Datenhaltung, die Sitzungsverwaltung und die Zuständigkeiten der beteiligten Komponenten neu einzuordnen.

4.2.3 Lokale Funktionen und exemplarische Online-Anbindungen

Die ausgewählten Funktionen decken unterschiedliche Verarbeitungspfade ab. Noten-, Termin- und Stundenplanverwaltung stehen für die Verarbeitung von Benutzereingaben und persistenten Anwendungsdaten. CSV- und iCal-/ICS-Importe ergänzen diesen Umfang um das Einlesen, Parsen und Validieren externer Dateien.

Für die persönlichen Funktionen wird eine lokale Verarbeitung vorgesehen. Die benötigten Katalogdaten müssen dafür bereits lokal verfügbar sein. Der Studienkatalog und die Studiengruppen bilden die betrachteten Online-Anbindungen: Beim Katalog werden externe Referenzdaten heruntergeladen und lokal übernommen; im Gruppenbereich werden gemeinsame Daten zwischen Beteiligten abgeglichen. Gruppenerstellung und Einladungen werden dabei als unmittelbar netzwerkabhängige Verwaltungsoperationen eingeordnet.

Diese Auswahl bildet den funktionalen Bezugsrahmen für die Zielarchitektur und den Anforderungskatalog.

4.3 Zielarchitektur und Sicherheitsbegriff

Ziel der Migration ist eine lauffähige Desktop-Anwendung, die einen sinnvoll nutzbaren Kernumfang des Ausgangsartefakts erhält. Vorhandene Oberflächenbestandteile und fachliche Regeln sollen soweit möglich weiterverwendet werden. Dadurch bleibt der Prototyp die Grundlage der Transformation, und die Änderungen können auf die Anforderungen der neuen Umgebung konzentriert werden. Architektur- und Sicherheitsanforderungen haben dabei Vorrang vor der unveränderten Übernahme bestehender Komponenten.

Für die fachliche Struktur wird eine Dreischichtenarchitektur gewählt. Darstellung, Geschäftslogik und Datenzugriff sollen getrennte Verantwortlichkeiten besitzen; fachliche Aufrufe werden entsprechend in Controller, Services und Repositories gegliedert. Damit lassen sich Änderungen am Transport und an der Persistenz gezielt zuordnen. Fachliche Services sollen unabhängig von Oberfläche und Transport geprüft werden können, wobei Datenzugriffe und externe Dienste durch Testimplementierungen ersetzbar sind.

Die Electron-Prozesse setzen diese fachlichen Schichten nicht unmittelbar abbildend um. Der Renderer stellt die Oberfläche bereit; Geschäftslogik und Datenzugriff können beide im Main-Prozess liegen. Privilegierte Operationen werden über begrenzte Preload-Schnittstellen und IPC aufgerufen. Dieser Kommunikationsweg organisiert lokale Fachoperationen innerhalb der Electron-Prozessstruktur, ohne das HTTP-Routing des ursprünglichen Backends als zusätzliche Vermittlungsschicht weiterzuführen.

Die lokale Datenhaltung bildet die Grundlage der Offline-Nutzung. Persönliche Änderungen sollen ohne Verbindung verarbeitet werden können und nach einem Neustart erhalten bleiben. Ergänzende Online-Anbindungen übernehmen zwei unterschiedliche Aufgaben: Der Katalogabruf aktualisiert lokal verfügbare Referenzdaten; die Gruppensynchronisierung übermittelt lokal verarbeitete Aufgabenänderungen bei vorhandener Verbindung. Damit wird die Offline-Kernanwendung um den ausgewählten Kollaborationsumfang erweitert [19].

Der Demonstrator soll vollständige Beispiele für die betrachteten Interaktionsformen bereitstellen. Dazu gehören Datenzuordnung, Validierung, Berechtigungsprüfung und Fehlerbehandlung. Die vollständige Übernahme jeder CRUD-Variante ist kein eigenständiges Ziel. Entscheidend ist, dass die vorhandenen Verarbeitungspfade einheitliche Muster zeigen, auf denen weitere Funktionen und Tests mit vergleichbaren Anforderungen aufbauen können.

Der Wechsel zur lokalen Laufzeit erweitert die Zugriffsmöglichkeiten auf Dateien und Systemressourcen. Unter einer gehärteten Electron-Anwendung versteht diese Arbeit deshalb die Umsetzung der anwendbaren Electron Security Guidelines im festgelegten Prüfumfang. Dazu zählen insbesondere Renderer-Isolation, begrenzte privilegierte Schnittstellen, die Prüfung von IPC-Absendern und Eingaben sowie Beschränkungen für nachgeladene Inhalte, Navigation und zusätzliche Fenster [23].

Aus der Umsetzung wird ein Migrationsleitfaden abgeleitet. Dieser soll die Voraussetzungen, Entscheidungen, Änderungen und Kontrollpunkte so dokumentieren, dass das Vorgehen am Demonstrator nachvollzogen werden kann. Projektspezifische Anpassungen und möglicherweise übertragbare Prinzipien werden dabei unterschieden.

4.4 Anforderungskatalog

Die Anforderungen konkretisieren die beschriebenen Ziele und Entwurfsentscheidungen. Ihre Kennungen dienen der Zuordnung zu den Migrationsschritten und der späteren Evaluation. Die folgenden Bewertungsansätze bestimmen die erforderlichen Nachweise; die Ergebnisse werden in Kapitel 6 dargestellt.

4.4.1 A1: Lauffähige Desktop-Anwendung

Das Ausgangsartefakt muss in eine lauffähige Electron-Anwendung überführt werden. Bewertet werden der Start eines gebauten Pakets und die Ausführbarkeit der eingeschlossenen Kernfunktionen. Version und Laufzeitumgebung sind im Nachweis festzuhalten.

4.4.2 A2: Dreischichtenarchitektur

Darstellung, Geschäftslogik und Datenzugriff müssen getrennte Verantwortlichkeiten besitzen. Die Fachlogik muss unabhängig von Oberfläche und Transport prüfbar sein. Der Nachweis erfolgt durch

den Abgleich der tatsächlichen Module mit der Zielarchitektur und einen beispielhaften fachlichen Verarbeitungspfad.

4.4.3 A3: Begrenzung des Refactoring-Aufwands

Bestandteile des Ausgangsartefakts sollen soweit möglich weiterverwendet werden. Notwendige Anpassungen und Ersetzungen sind nach Komponenten beziehungsweise Schichten zu dokumentieren und zu begründen. Bewertet wird die Nachvollziehbarkeit der Eingriffe; ein global minimaler Entwicklungsaufwand oder eine gemessene Zeitersparnis wird nicht verlangt.

4.4.4 A4: Lokale Kommunikation im Electron-Prozessmodell

Lokale Fachoperationen müssen über begrenzte Preload-Schnittstellen und IPC ausgeführt werden. Dies nutzt den in Abschnitt 2.4 beschriebenen Kommunikationsweg des Electron-Prozessmodells. Die zusätzliche Entwurfsentscheidung dieser Arbeit besteht darin, das ursprüngliche HTTP-Routing nicht als lokalen Server weiterzuführen. Der Nachweis beschreibt den tatsächlichen Aufrufweg und zeigt, dass hierfür kein lokaler HTTP-Server erforderlich ist. Externe HTTPS-Anbindungen werden gesondert betrachtet.

4.4.5 A5: Lokale Datenhaltung und Offline-Kernfunktionen

Die ausgewählten persönlichen Kernfunktionen und die lokale Identitätsverwaltung müssen ohne Netzwerkverbindung nutzbar sein. Änderungen müssen einen Anwendungsneustart überdauern. Damit wird die lokale Nutzbarkeit als Grundlage des in Abschnitt 2.6 beschriebenen Local-First-Anspruchs konkretisiert; A6 ergänzt die Zusammenarbeit. Für katalogabhängige Funktionen wird ein bereits lokal verfügbarer Datenbestand vorausgesetzt. Der Nachweis umfasst lokale Verarbeitung und anschließendes erneutes Lesen der gespeicherten Daten.

4.4.6 A6: Local-First-Kollaboration am Gruppenfeature „Aufgaben“

Aufgabenänderungen müssen lokal verarbeitet und bei vorhandener Verbindung synchronisiert werden können. Lokale Speicherung und anschließende Übermittlung sind getrennt nachzuweisen. Gruppenerstellung und Einladungen gehören zum Kernumfang; ihre Online-Voraussetzungen sind auszuweisen.

4.4.7 A7: Härtung gemäß Electron Security Guidelines

Die Sicherheitscheckliste muss vollständig auf Anwendbarkeit geprüft werden. Für anwendbare Empfehlungen sind Umsetzung, Beleg und Nachweisebene zu dokumentieren. Vorhandene Tests und ausgewählte negative Laufzeitfälle ergänzen die Prüfung. Fehlende oder begrenzte Nachweise müssen ausdrücklich erkennbar bleiben.

4.4.8 A8: Repräsentativer Kernfunktionsumfang und Erweiterbarkeit

Der Demonstrator muss zusammenhängende Verarbeitungspfade für CRUD-Oberflächeninteraktionen, Dateiimporte und Netzwerkanbindungen bereitstellen. Der Nachweis benennt die übernommenen Funktionen, vergleicht ihr Verhalten mit dem Ausgangsartefakt und ordnet ihnen Architektur- und Sicherheitsmuster zu. Er muss zeigen, wie weitere Funktionen mit vergleichbaren Anforderungen auf diesen Mustern aufbauen können.

4.4.9 A9: Nachvollziehbarer Migrationsleitfaden

Der Leitfaden muss Voraussetzungen, Reihenfolge, Änderungen, Entscheidungen und Kontrollpunkte beschreiben. Seine Schritte müssen dem Demonstrator zugeordnet werden können. Bewertet werden die Nachvollziehbarkeit dieser Zuordnung und die Kennzeichnung des Geltungsbereichs der Empfehlungen.

4.5 Abgrenzung des Untersuchungsumfangs

Die interne Sicherheit und der Betrieb entfernter Dienste für Synchronisierung oder Datenbereitstellung liegen außerhalb des Umfangs. Ebenso ausgeschlossen sind die Verschlüsselung der lokalen Datenbankdatei und ihr Schutz gegenüber anderen Betriebssystemprozessen oder Benutzern.

Kommentare, weitere gemeinsame Gruppeninhalte, die Synchronisierung persönlicher Daten über mehrere Endgeräte und gemeinsame Echtzeitbearbeitung von Dokumenten werden nicht untersucht. Auch die Entwicklung eines vollständigen entfernten Kontolebenszyklus mit Registrierung und Passwortzurücksetzung sowie die Übertragung von Dateien über entfernte Speicherbereiche gehören nicht zum Umfang.

Frontend-Unit-Tests, automatisierte End-to-End-Tests, ein umfassender Sicherheitsvergleich zwischen Web- und Desktop-Laufzeit sowie ein unabhängiger Reproduktionsdurchlauf des Leitfadens sind ausgeschlossen. Aussagen zur Sicherheit und Übertragbarkeit bleiben auf die ausgewiesenen Verfahren und Nachweise begrenzt.

Kapitel 5

Entwurf, Migration und Migrationsleitfaden

Dieses Kapitel dokumentiert den durchgeführten Migrationsprozess und die dabei getroffenen Entscheidungen. Die Darstellung beginnt mit der Vorbereitung der lokalen Datenhaltung und folgt anschließend den weiteren Transformationsschritten. Auf dieser Grundlage werden die Handlungsempfehlungen des Migrationsleitfadens abgeleitet. Die Beschreibung der konkreten Umsetzung und die daraus gewonnenen Empfehlungen sind dabei zu unterscheiden.

5.1 Struktur des Migrationsleitfadens

Der Leitfaden ordnet die Migration in drei Phasen: die Überführung in eine lokal nutzbare Offline-Anwendung, die Ergänzung um Synchronisierung und weitere Online-Anbindungen sowie die konsolidierte Härtung. Innerhalb dieser Phasen werden zusammengehörige Änderungen nach ihrer Aufgabe erläutert, etwa Datenhaltung, Schnittstellen, CRUD-Oberflächeninteraktionen, Dateiimporte und Netzwerkanbindungen. Die Darstellung bündelt die tatsächlich vorgenommenen Anpassungen und erklärt exemplarisch ihre Entscheidungen und Prinzipien. Sie beschreibt daher nicht jede migrierte Funktion einzeln. Die Analyse aus Kapitel 4 liefert den architektonischen Ausgangspunkt und die Kriterien zur Bewertung des erreichten Zustands.

Die Phasen unterscheiden Zielzustände der Entwicklung. Sie behaupten weder eine von Beginn an vollständig festgelegte Projektplanung noch eine vollständige Trennung aller Arbeitsschritte. Insbesondere beeinflussen die Sicherheitsanforderungen bereits den ersten Architekturentwurf. Jede Phase beschreibt deshalb neben ihrem funktionalen Ziel auch die relevanten Schnittstellen und einen Kontrollpunkt für den erreichten Stand.

Phase	Zielzustand	Kontrollpunkt
1	Lokale Kernfunktionen mit lokaler Datenhaltung und begrenzter IPC-Kommunikation.	Kernabläufe ohne Netzwerk ausführen; Änderungen nach Neustart erneut lesen.
2	Synchronisierte Aufgaben und gesonderte Online-Anbindungen.	Lokale Speicherung, spätere Übermittlung und Verhalten bei Netzwerkausfall unterscheiden.
3	Geprüfte Härtung im festgelegten Electron-Sicherheitsrahmen.	Anwendbarkeit, Umsetzung und Nachweis jeder Empfehlung dokumentieren.

Tabelle 5.1: Phasen und Kontrollpunkte des fallbezogenen Migrationsleitfadens.

5.2 Phase 1: Von der Webanwendung zur Offline-Kernanwendung

Die Umsetzung beginnt mit der lokalen Datenbank und der Schnittstelle, auf die anschließend die vorhandene Oberfläche und die fachlichen Verarbeitungspfade aufbauen. Katalogabhängige Funktionen verwenden in dieser Phase einen lokal bereitgestellten Datenbestand.

5.2.1 Initialisierung des Electron-Projekts und Übernahme der Datenbankmodule

Electron wurde im Sinne von A3 gewählt, um vorhandene Komponenten weiterzuverwenden. Das Backend des Ausgangsartefakts verwendete TypeScript und Node.js [28]. Da der Electron-Main-Prozess ebenfalls eine Node.js-Laufzeit bereitstellt, konnten fachliche Logik und Bibliotheken als Grundlage der lokalen Umsetzung dienen [12].

Als Alternative wurde Tauri erwogen, dessen privilegierter Core-Prozess auf Rust basiert [29]. Eine Portierung des Backends in diesen Core hätte neben der Sprache auch Datentypen, Fehlerbehandlung und Bibliotheksanbindungen betroffen, insbesondere den Drizzle-Datenzugriff. Tauri erlaubt allerdings auch die Einbindung einer Node.js-Anwendung als Sidecar [30]. Dieser Weg hätte einen zusätzlichen auszuliefernden Prozess und dessen Kommunikation erfordert. Für Electron sprach die unmittelbare Ausführung der übernommenen Backend-Komponenten im Main-Prozess.

Das Zielprojekt wurde mit der `react-ts`-Vorlage von electron-vite initialisiert [31]. Damit blieb die im Replit-Frontend verwendete Kombination aus React, TypeScript und Vite erhalten [28]. Die betrachtete React-TypeScript-Anleitung von Electron Forge verwendet `webpack-typescript`; Forge bietet daneben auch Vite-Vorlagen an [32], [33]. Die Wahl von electron-vite beruhte auf der erwarteten Begrenzung der Anpassungen durch kompatible Technologien. Ein geringerer Migrationsaufwand gegenüber Tauri oder Electron Forge wurde nicht vergleichend evaluiert.

Anschließend wurden die Drizzle-Schemadefinitionen für Benutzer, Stundenpläne, Studiengänge, Gruppen, Noten, Termine und das Aktivitätsprotokoll samt Exportmodul nach `src/main/db/schema` übernommen. Die angepasste `drizzle.config.ts` liegt im Projektstamm. Für die erste Entwicklung ersetzte eine lokale SQLite-Datei die PostgreSQL-Verbindung: Der Dialekt wechselte zu `sqlite`, und `DATABASE_URL` entfiel für die lokale Datenhaltung [28], [34].

Mit dem Umgebungswechsel übernahm die Desktop-Anwendung im Main-Prozess das Anlegen, Öffnen und Migrieren der Benutzerdatenbank. Die Datei wird zur Laufzeit unter dem von Electron bereitgestellten Pfad `userData` angelegt [35]. Dieser liegt außerhalb des Projekt- und Installationsverzeichnisses, trennt veränderliche Daten von Programmdateien und vermeidet Schreibzugriffe auf das Installationsverzeichnis. Ein Update kann so Programmdateien ersetzen, ohne die Benutzerdatenbank

auszutauschen. Ausgeliefert werden die Migrationen zur Einrichtung und Aktualisierung ihres Schemas [34].

5.2.2 Anpassung der Tabellen und Datenbankabhängigkeiten an SQLite

Die Übernahme der Dateien genügte noch nicht, da die bestehenden Definitionen PostgreSQL-spezifische Datentypen und Tabellenkonstruktoren verwendeten. Diese wurden durch die entsprechenden SQLite-Konstrukte ersetzt. Die fachlichen Tabellen- und Spaltennamen sowie die vorhandenen Pflichtfelder und Standardwerte wurden dabei möglichst beibehalten. Dieser Ansatz begrenzte die Änderungen am Datenmodell und entsprach dem in Kapitel 4 festgelegten Ziel, vorhandene Komponenten mit möglichst geringem Refactoring-Aufwand weiterzuverwenden.

Tabelle 5.2 fasst die im Demonstrator verwendeten Abbildungen zusammen. Das Stundenplanschema liefert Beispiele für Kennungen, boolesche Werte und Enums; das Terminschema ergänzt die Datumsabbildung [28], [34].

PostgreSQL / Drizzle	**SQLite / Drizzle**	**Bedeutung der Anpassung**
`pgTable`	`sqliteTable`	Tabellenkonstruktor für den neuen Dialekt.
`serial` als Primärschlüssel	`integer` mit `autoIncrement`	Numerische, automatisch erzeugte Kennung bleibt erhalten.
`boolean`	`integer` im Modus `boolean`	Integer-Speicherung mit booleschen TypeScript-Werten.
`pgEnum`	`text` mit `enum`-Werteliste	Typableitung bleibt erhalten; keine gleichwertige Datenbankprüfung.
`timestamp` mit Zeitzone	`integer` im Modus `timestamp`	Zeitpunkt wird als Unix-Zeit gespeichert und im Code als `Date` verwendet.

Tabelle 5.2: Ausgewählte Abbildungen der PostgreSQL-Schemadefinitionen auf SQLite im Demonstrator.

Die PostgreSQL-Enums für Wochentage und Zeitblöcke wurden durch konstante Wertelisten und daraus abgeleitete TypeScript-Union-Typen ersetzt. Die `enum`-Option des Text-Konstruktors unterstützt diese Typableitung, erzwingt jedoch zur Laufzeit keinen zulässigen Wertebereich in der Datenbank [36]. Eingehende Werte müssen deshalb weiterhin geprüft werden. Auch die Datumsabbildung verlangt einen konsistenten Umgang mit Zeiteinheit und Zeitzonen; im gewählten Modus wird der Zeitpunkt in Sekunden gespeichert. Die aus den Tabellen abgeleiteten Insert- und Select-Typen sowie die Erzeugung der Zod-Schemata konnten weiterhin nach dem vorhandenen Muster verwendet werden [34].

Für die benötigten Pakete wurde zunächst die Abhängigkeitskonfiguration des Ausgangsartefakts herangezogen. Die angegebenen Versionsbereiche für `drizzle-orm` (`^0.45.2`), `drizzle-kit` (`^0.31.10`) und `drizzle-zod` (`^0.8.3`) wurden übernommen. Der PostgreSQL-Treiber `pg` wurde für die lokale Datenhaltung durch `@libsql/client` ersetzt; im Demonstrator ist hierfür der Versionsbereich `^0.18.0` angegeben [28], [34]. Damit blieb Drizzle als Abstraktion für Schema und Datenzugriff erhalten, während der verwendete Treiber wechselte.

Die Verantwortlichkeiten der Pakete sind dabei zu unterscheiden: Der libSQL-Client öffnet die lokale Datenbankdatei, Drizzle ORM führt die Datenbankoperationen aus und Drizzle Kit erzeugt die SQL-Migrationsdateien. Das Anwenden dieser Dateien beim Anwendungsstart übernimmt der Migrator aus `drizzle-orm/libsql/migrator`. Die automatische Anwendung vorbereiteter Migrationen ist somit Bestandteil der eingerichteten Drizzle-Initialisierung und keine alleinige Eigenschaft des Datenbanktreibers [34].

5.2.3 Versionierte Schemamigrationen und Datenbankinitialisierung

Nach der Anpassung der Tabellen wurden zwei Wege zur Einrichtung des Datenbankschemas unterschieden. Mit `drizzle-kit push` können die Schemadefinitionen unmittelbar mit einer angebundenen Datenbank abgeglichen und die Änderungen ohne gesondert gespeicherte SQL-Migrationsdateien angewandt werden [37]. Alternativ erzeugt `drizzle-kit generate` SQL-Dateien anhand der aktuellen Definitionen und der bisherigen Schema-Snapshots. Diese Dateien können anschließend durch den Drizzle-Migrator angewandt werden [18].

Für den Demonstrator wurde der zweite Weg gewählt. Die Migrationsdateien liegen im Verzeichnis `drizzle` und können gemeinsam mit dem Anwendungscode versioniert und ausgeliefert werden. Beim Start verarbeitet der Migrator die noch nicht angewandten Migrationen. Damit steht ein nachvollziehbarer Ablauf für spätere Schemaänderungen bereit, ohne die vorhandene Benutzerdatenbank bei jedem Update neu anzulegen. Die Verträglichkeit einer konkreten Schemaänderung mit bestehenden Daten muss dennoch anhand der jeweiligen Migration geprüft werden.

Die resultierende Konfiguration beschreibt deshalb nur den SQLite-Dialekt, den Einstiegspunkt der Schemadefinitionen und das Ausgabeverzeichnis der Migrationen. Ein Verbindungsparameter zur Entwicklungsdatenbank ist für die Generierung nicht mehr erforderlich [18]. Der Laufzeitpfad der Benutzerdatenbank wird separat im Main-Prozess festgelegt. Quellcode 5.1 zeigt diese Konfiguration des Demonstrators [34].

```ts
import { defineConfig } from 'drizzle-kit'

export default defineConfig({
  schema: './src/main/db/schema/index.ts',
  out: './drizzle',
  dialect: 'sqlite'
})
```

Quellcode 5.1: Drizzle-Kit-Konfiguration zur Erzeugung der SQLite-Migrationen.

In `package.json` wurden `db:generate` für `drizzle-kit generate` und `db:check` für `drizzle-kit check` eingerichtet. Zur Laufzeit verwendet die Anwendung die erzeugten Dateien über den ORM-Migrator [34].

Die Initialisierung wurde in den Startablauf des Main-Prozesses eingebunden. Innerhalb des mit `app.whenReady()` begonnenen Ablaufs in `src/main/index.ts` wird `initDatabase()` abgewartet, bevor die lokale Datenbank an Repositories und Dienste übergeben und das Anwendungsfenster geöffnet wird. Scheitert die Vorbereitung, zeigt die Anwendung eine Fehlermeldung an und beendet den Start. Die Oberfläche wird dadurch nicht mit einer unvollständig vorbereiteten Datenbank geöffnet [34].

In `src/main/db/index.ts` wird der Speicherort aus `app.getPath('userData')` und dem Unterverzeichnis `data` gebildet. Die paketierte Anwendung verwendet `studyplan.db`, die Entwicklung `studyplan-dev.db`. Unter Windows ergibt sich beispielsweise `%APPDATA%\studyplan\data\studyplan.db` innerhalb von `AppData\Roaming`. Der Unterordner wird vor dem Öffnen der Datei angelegt [34].

Danach öffnet der libSQL-Client die Datei über eine lokale `file:`-URL. Die Initialisierung setzt ausdrücklich `PRAGMA foreign_keys = ON` und `PRAGMA journal_mode = WAL`. Die erste Einstellung aktiviert die Prüfung der im Schema tatsächlich definierten Fremdschlüsselbedingungen; sie ergänzt selbst keine solchen Bedingungen. Die zweite wählt den Write-Ahead-Logging-Modus für die Datenbank [38]. Anschließend werden die ausstehenden Migrationen angewandt und die Verbindung mit einer einfachen Abfrage geprüft. Die konkrete Implementierung legt außerdem die SQLite-kompatible Migrationstabelle `__drizzle_migrations` vor dem Aufruf des Migrators an [34].

Damit die Migrationen auch in der installierten Anwendung verfügbar sind, wurde das Verzeichnis `drizzle` in `electron-builder.yml` unter `extraResources` aufgenommen. Quellcode 5.2 zeigt diesen Eintrag. In der Entwicklung liest die Anwendung die Dateien aus dem Projektverzeichnis; im paketierten Zustand verwendet sie `process.resourcesPath` und den dort enthaltenen Unterordner `drizzle` [34].

```yaml
extraResources:
  - from: drizzle
    to: drizzle
    filter:
      - '**/*'
```

Quellcode 5.2: Auslieferung der Migrationsdateien über zusätzliche Paketressourcen (Ausschnitt).

Für Funktionen mit Referenzdaten wurden außerdem Studiengänge, Lehrveranstaltungen und Stundenplanvorlagen über die Datenmigrationen `0001_seed-study-programs.sql` und `0002_seed-timetable-presets.sql` bereitgestellt. Sie werden beim Einrichten der Datenbank lokal eingefügt und stehen damit auch vor dem in Phase 2 ergänzten Katalogabruf zur Verfügung. Dies ist für die Offline-Nutzung entscheidend: Ein lokal ausführbarer Handler kann nur auf den tatsächlich vorhandenen Datenbestand zugreifen [34].

5.2.4 Übernahme des API-Vertrags und Anpassung der Codegenerierung

Die Schnittstellenmigration ging vom OpenAPI-Vertrag unter `lib/api-spec/openapi.yaml` aus, der nach `api/openapi.yaml` übernommen wurde. Seine Operationskennungen, Parameter und Datenschemata dienten bereits im Web-Prototyp zur Erzeugung von Aufrufhilfen und Zod-Schemata [28].

Im Ausgangsartefakt verwendete Orval dafür `orval.config.ts`. Im Desktop-Projekt übernehmen eigene TypeScript-Skripte unter `scripts/openapi-ipc` die Generierung. Ihre Konfiguration `api/ipc-generator.config.ts` legt den Eingabevertrag und die Ausgabepfade fest. Das Prinzip der gemeinsamen Vertragsgrundlage bleibt damit erhalten, während die Umsetzung auf IPC ausgerichtet wird [39].

Der Generator liest den Vertrag mit `yaml` und erzeugt TypeScript-Typen, Zod-Laufzeitschemata, die Zuordnungstabelle `operationContracts` sowie einen transportunabhängigen API-Client unter `src/shared/api/generated`. React-Query-Hooks liegen unter `src/renderer/src/api/generated`. Der Main-Prozess verwendet die gemeinsamen Schemata zur Validierung [39].

`api:generate` erzeugt die Module; `api:check` prüft ihre Übereinstimmung mit der Generatorausgabe. Der YAML-Vertrag wird während der Entwicklung eingelesen. Zur Laufzeit werden die erzeugten Module verwendet. Die fachliche Implementierung und ihre Handler-Registrierung bleiben gesonderte Aufgaben [39].

Während der Oberflächenintegration wurde `generator-lib.ts` im Commit `f03ab7f` um React-Query-Hooks und das Exportmodul `src/renderer/src/api/index.ts` erweitert. Die Aufrufformen orientieren

sich am Web-Client: Queries erhalten Pfad- und Query-Parameter, Mutationen ein Variablenobjekt mit `data` für den bisherigen Request-Body. Auch Hilfsfunktionen für Query Keys werden erzeugt. Diese Orientierung ermöglicht die Weiterverwendung vorhandener Aufrufmuster [39], [40].

Zunächst stellte Preload den vollständigen API-Client bereit. Im Authentifizierungscommit `1c1988d` wurde seine Erzeugung in den Renderer verlagert: Die Hooks verwenden dort `createApiClient` mit `window.apiTransport.invoke`. Preload vermittelt den Transport; der Client verarbeitet Ergebnisse und rekonstruiert Fehler als `ApiClientError` [41].

5.2.5 Funktionsweise der OpenAPI-IPC-Codegenerierung

Der Generator übersetzt die Vertragsbeschreibung während der Entwicklung in TypeScript-Quelltext. Dabei sind drei Schritte zu unterscheiden: das Einlesen und Strukturieren des YAML-Dokuments, das Zusammensetzen von Quelltext-Zeichenketten und das Schreiben der erzeugten Module. Die Generierungsfunktionen erstellen somit zunächst Text; die darin beschriebenen Zod-Schemata und Client-Funktionen entstehen erst beim Laden der kompilierten Module in der Anwendung [39].

`buildGeneratedFiles` liest die konfigurierte Datei mit `readFile` und überführt sie mit `parse` aus dem Paket `yaml` in ein JavaScript-Objekt. `collectOperations` durchläuft dessen Pfade und Methoden und erzeugt für jede Operation einen internen Datensatz. Dieser enthält unter anderem Kennung, ersten Tag, Parameter, Antwortschemata und die Einordnung als Query oder Mutation. `operationInputSchema` fasst Pfad- und Query-Parameter sowie den Request-Body zu den Eingabegruppen `path`, `query` und `body` zusammen. Fehlende oder doppelte Operationskennungen werden zurückgewiesen [39].

Die rekursive Funktion `zodSchema` übersetzt anschließend einzelne Schemadefinitionen in Zod-Ausdrücke als Zeichenketten. Listing 5.3 zeigt den Aufbau für einen String mit Mindestlänge. Für das Titelfeld aus `EventInput` enthält `expression` zunächst den Text `z.string()`. Durch `minLength: 1` wird dieser um `.min(1)` ergänzt. In einem Objektschema wird daraus die Feldzeile `title: z.string().min(1)`. Fehlt ein Feld in der Liste `required`, ergänzt der Objektzweig `.optional()`, wie bei der Beschreibung des Termins [39].

```
expression = 'z.string()'
if (typeof schema.minLength === 'number')
  expression += `.min(${schema.minLength})`
// Nach Verarbeitung der weiteren Eigenschaften:
return expression
```

Listing 5.3: Aufbau eines Zod-Ausdrucks als Zeichenkette in `zodSchema`; gekürzter Auszug, Kommentar ergänzt [39].

Der Objektzweig setzt die erzeugten Feldzeilen mit `map` und `join` zu einem `z.object`-Ausdruck zusammen. `contractSource` bettet diesen mittels Template-Strings in eine exportierte Schemadeklaration ein und ergänzt den mit `z.infer` abgeleiteten TypeScript-Typ. Listing 5.4 stellt die YAML-Eingabe und das Ergebnis gegenüber. Das Beispiel ist auf Titel und Beschreibung reduziert; weitere Terminfelder sind ausgelassen. Die erzeugte Prüfung verlangt einen nicht leeren Titel und erlaubt eine fehlende Beschreibung [39].

YAML-Eingabe (reduziert)

```yaml
EventInput:
  type: object
  required: [title]
  properties:
    title:
      type: string
      minLength: 1
    description:
      type: string
```

TypeScript-Ausgabe (reduziert)

```typescript
export const EventInputSchema = z
  .object({
    title: z.string().min(1),
    description: z.string().optional()
  })
  .passthrough()
export type EventInput =
  z.infer<typeof EventInputSchema>
```

Listing 5.4: Übersetzung ausgewählter Terminfelder in ein Laufzeitschema und einen abgeleiteten Typ; reduzierte Darstellung nach [39].

Für zusammengesetzte Schemata erfolgt dieselbe Übersetzung schrittweise: Array-Elemente werden rekursiv verarbeitet, und lokale `$ref`-Verweise werden als Referenzen auf benannte Schemakonstanten ausgegeben. `orderSchemas` ordnet deren Deklarationen anhand der Abhängigkeiten. `contractSource` erzeugt außerdem die Eingabe-, Ausgabe- und Fehlerschemata der Operationen sowie `operationContracts`, das diese Schemata den Operationskennungen zuordnet [39]. Tabelle 5.3 fasst ausgewählte Zuordnungen zusammen.

OpenAPI-Eingabe	**Generatorfunktion**	**Erzeugtes Element**
`type: string` mit `minLength: 1`	`zodSchema`	`z.string().min(1)`
Feld außerhalb von `required`	`zodSchema`, Objektzweig	Ergänzung von `.optional()` am Feldausdruck.
Array mit `$ref` auf `Event` als Element	`zodSchema`	`z.array(EventSchema)`
Optionaler Integer-Parameter `month` mit `in: query`	`operationInputSchema`, anschließend `zodSchema`	Optionales Feld `query.month` mit `z.number().int()`.
Erforderlicher Request-Body mit Verweis auf `EventInput`	`operationInputSchema`, anschließend `contractSource`	Erforderliches Feld `body: EventInputSchema` im Operations-Eingabeschema.
`operationId: listEvents`, `tags: [events]`	`collectOperations`, anschließend `clientSource`	Client-Methode `api.events.listEvents(...)` und Aufruf mit der Kennung `listEvents`.
Methode `get` beziehungsweise `post`	`collectOperations`, anschließend `rendererReactQuerySource`	Query-Hook `useListEvents` beziehungsweise Mutation-Hook `useCreateEvent`.
Erste deklarierte `2xx`-Antwort und deklarierte Fehlerantworten	`collectOperations`, anschließend `contractSource`	Ausgabeschema sowie Vereinigung der deklarierten Fehlerformen.

Tabelle 5.3: Ausgewählte Übersetzungen vom OpenAPI-Vertrag in die projektspezifischen IPC-Module [39].

`clientSource` gruppiert die Operationen nach ihrem ersten Tag und erzeugt typisierte Methoden, die Operationskennung und Eingaben an den übergebenen Transport weiterreichen. `rendererReactQuerySource` baut daraus die React-Query-Anbindung: `get` wird als Query behandelt, die übrigen unterstützten Methoden als Mutationen. Die erzeugten Hooks rufen die Client-Methoden

auf; aus den HTTP-Pfaden werden dabei keine Netzwerkaufrufe erzeugt. Die Laufzeitweiterleitung über IPC wird im folgenden Abschnitt beschrieben [39].

Abschließend liefert `buildGeneratedFiles` eine Liste aus Zielpfaden und vollständigen Quelltexten. Prettier formatiert diese mit dem TypeScript-Parser. `generate.ts` legt die Zielverzeichnisse an und schreibt die Zeichenketten mit `writeFile` als UTF-8-Dateien. Neben `contract.generated.ts` und `client.generated.ts` entstehen `react-query.generated.ts` und die Exportmodule. Diese Dateien werden anschließend durch den regulären TypeScript- und Electron-Build verarbeitet. Der Modus `--check` vergleicht die erzeugten Inhalte mit den vorhandenen Dateien, ohne sie zu überschreiben. Der Generator erzeugt damit konsistente Schnittstellenbausteine; Handler, Services und Repositories werden weiterhin fachlich implementiert [39].

5.2.6 IPC-Transport, Preload-Schnittstelle und Dispatcher

Der API-Client ersetzt HTTP-Methode, URL und `customFetch` durch eine Funktion vom Typ `ApiTransport`. Diese übermittelt `operationId` und typisierte Eingabedaten über Preload. Die übernommenen Gruppen `path`, `query` und `body` bezeichnen nun Felder des IPC-Eingabeobjekts. Die Schnittstelle folgt dem Prinzip eines Remote Procedure Call (RPC): Der Renderer ruft eine benannte Operation im Main-Prozess auf und erhält deren Ergebnis. Electron-IPC bildet dafür den lokalen Transport zwischen den Prozessen [28], [39].

In `src/preload/index.ts` sendet `apiTransport.invoke` die Operationskennung, Eingaben und eine `requestId` über `ipcRenderer.invoke` auf dem festen Kanal `api:invoke`. `contextBridge.exposeInMainWorld` stellt dem Renderer `window.apiTransport` bereit; `ipcMain.handle` nimmt den Aufruf im Main-Prozess entgegen. Dies entspricht dem von Electron dokumentierten Aufrufmuster mit Rückgabewert [14], [39].

`src/preload/index.d.ts` erweitert `Window` für die Typprüfung; das Laufzeitobjekt stellt Preload bereit. Der Renderer erhält die begrenzte Transportmethode statt frei wählbarer IPC-Kanäle oder des vollständigen `ipcRenderer`-Objekts. Diese Begrenzung entspricht der Electron-Empfehlung für die Bridge [42].

Der Dispatcher in `src/main/ipc/api-dispatcher.ts` prüft Absender und Anfrageform und ordnet die `operationId` einem Vertrag sowie einem registrierten Handler zu. Das generierte Zod-Schema validiert die Eingaben. Unbekannte Operationen und ungültige Daten werden abgewiesen [39].

Nach der Handler-Ausführung validiert der Dispatcher auch Rückgabewerte und ausdrücklich deklarierte Fehlerantworten. Ein erfolgreiches `ApiResult` enthält `ok: true`, Daten und Anfragekennung; ein Fehler enthält `ok: false` und ein Fehlerobjekt. Der API-Client entnimmt die Daten beziehungsweise erzeugt `ApiClientError` [39].

Bei einer bekannten, aber nicht registrierten Operation liefert der Dispatcher `NOT_IMPLEMENTED`. Generierte Hooks allein stellen somit keine ausführbare Funktion bereit. Fachlogik, Sitzungszuordnung und Berechtigungen werden in Handlern und Services umgesetzt; die konsolidierte Härtungsprüfung folgt in Phase 3 [39].

5.2.7 Begründung und Grenzen der vertragsbasierten Generierung

Die Entscheidung berücksichtigt auch den Vorversuch aus Abschnitt 3.3. Die dort beobachteten, schwer zuzuordnenden Änderungen begründeten das Ziel, notwendige Schnittstellenanpassungen nachvollziehbarer von zusätzlichen Verhaltensänderungen zu trennen. Die gemeinsame Vertragsgrundlage unterstützt hierfür die abgestimmte Ableitung der Schnittstellenbausteine.

Alternativ hätten die von Orval erzeugten Hooks, Typen und Zod-Schemata manuell angepasst werden können. Der eigene Generator wäre entfallen, Änderungen hätten jedoch in mehreren abhängigen

Modulen abgestimmt werden müssen. Für den erweiterbaren Demonstrator wurde deshalb die zentrale Vertragsgrundlage beibehalten und ihre Übersetzung nach IPC automatisiert.

Für Erweiterungen wird der Vertrag an einer Stelle ergänzt und die Änderung durch erneute Generierung in Typen, Client, Hooks und Laufzeitschemata übertragen. Anschließend werden Handler, Fachlogik und Oberflächenabläufe implementiert. Dieses Vorgehen unterstützt die Wiederverwendung nach A3 und das Erweiterungsmuster nach A8 und A9. Der anfängliche Entwicklungs- und spätere Pflegeaufwand des Generators bleibt zu berücksichtigen; eine quantitative Aufwandsersparnis wurde nicht nachgewiesen [39].

Die gemeinsame Vertragsbindung definiert die Schnittstelle zwischen Renderer und Main-Prozess. Die fachliche Implementierung bleibt hinter dieser Grenze im Main-Prozess. Damit können Schnittstellenverarbeitung, Fachlogik und Datenzugriff getrennt geprüft werden, wie A2 verlangt.

Abbildung 5.1 stellt den ursprünglichen Aufrufweg demjenigen der lokalen Electron-Kernanwendung gegenüber. In beiden Fällen bildet ein gemeinsamer Vertrag die Grundlage für Client und Validierung. Die wesentliche Änderung liegt in der Umsetzung dieses Vertrags zur Laufzeit: HTTP-Aufrufe an einen Express-Server werden durch begrenzte Aufrufe über Preload und IPC ersetzt. Die eingezeichnete Trennung in Handler, Services und Repositories zeigt die resultierende Desktop-Architektur; ihre fachliche Ausarbeitung ist ein weiterer Migrationsschritt [28], [39].

Die Umsetzung der gemeinsamen Vertragsgrundlage im Web-Prototyp und in der Electron-Kernanwendung wird in Tabelle 5.4 gegenübergestellt.

Aspekt	**Web-Prototyp**	**Electron-Kernanwendung**
Vertragsgrundlage	OpenAPI mit Operationskennungen und Datenschemata.	Bestehende Definitionen übernommen und gezielt ergänzt.
Generierung	Orval für React Query und Zod.	Projektspezifische TypeScript-Generatoren für Client, Hooks und gemeinsame Zod-Verträge.
Aufrufweg	HTTP-Methode und URL über `customFetch`.	`operationId` und Eingabeobjekt über Preload und den Kanal `api:invoke`.
Parameter	Pfad, Query-String und HTTP-Body.	Strukturierte Eingabe mit `path`, `query` und `body`.
Ergebnis	HTTP-Antwort beziehungsweise HTTP-Fehler.	Geprüftes `ApiResult`; Fehler werden als `ApiClientError` weitergegeben.
Datumswerte	Strings im generierten Client; Datumsprüfung in Zod-Schemata.	`Date`-Werte und `z.date()` in den gemeinsamen Modulen.
Zugriffsgrenze	Serverseitige HTTP-Verarbeitung und Sitzungszuordnung.	Begrenzte Bridge, Absender- und Schemaprüfung sowie fachliche Sitzungsprüfung.

Tabelle 5.4: Wiederverwendete Vertragsgrundlage und Änderungen an ihrer Umsetzung.

OpenAPI beschreibt standardmäßig HTTP-APIs [9]. Die IPC-Nutzung übersetzt ausgewählte Vertragsbestandteile projektspezifisch: Methoden und Pfade bleiben Metadaten, die Weiterleitung verwendet die Operationskennung. Der Generator unterstützt die benötigten Schemakonstrukte und

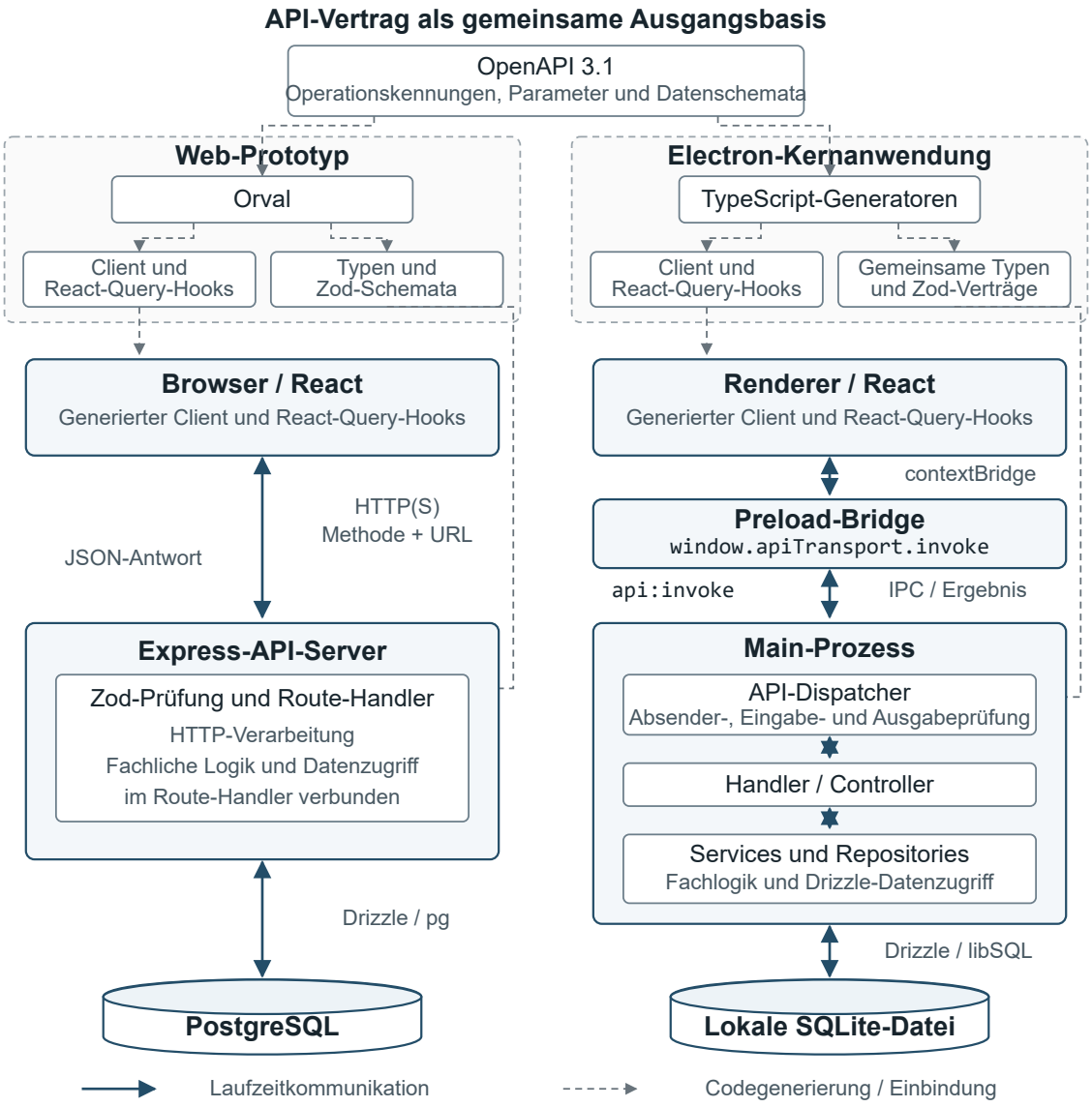


Abbildung 5.1: Vertragsbasierte Codegenerierung und Aufrufwege im Web-Prototyp und in der lokalen Electron-Kernanwendung. Gestrichelte Pfeile zeigen Generierung oder Einbindung, durchgezogene Pfeile Laufzeitkommunikation. Spätere Synchronisierungs- und Online-Pfade sind nicht dargestellt.

lokale Referenzen, weist jedoch beispielsweise rekursive Komponentenschemata zurück. Er bildet damit den verwendeten Vertragsumfang ab, ohne vollständige OpenAPI-Kompatibilität zu beanspruchen [39].

5.2.8 Übernahme der React-Oberfläche in den Renderer

Die React-Oberfläche aus `artifacts/studienplaner/` wurde im Commit `f4ea658` nach `src/renderer/src/` übernommen. Dazu gehören Layout, UI-Komponenten, Hilfsmodule unter `hooks/` und `lib/` sowie die Seiten für Dashboard, Kalender, Noten und Stundenplan [40].

`App.tsx` und `main.tsx` behielten die Einbindung von React Query über `QueryClientProvider` sowie die Anbieter für Tooltips und Benachrichtigungen bei. Die Darstellungsstruktur konnte weitgehend

weiterverwendet werden; anzupassen waren insbesondere Modulauflösung, Datenzugriffe und Navigation [28], [40].

Die übernommene `index.css` enthält Theme-Variablen und die Einbindung von Tailwind CSS, `tw-animate-css` und dem Typografie-Plugin. Die Vorlagendateien `assets/main.css` und `assets/base.css` wurden samt Beispielkomponenten und Grafiken entfernt. Der zunächst ebenfalls kopierte Google-Fonts-Import wurde im Commit `36ff5f9` entfernt. Die Schriftstapel nennen weiterhin „Plus Jakarta Sans“ und „Fraunces“ und enthalten System- beziehungsweise generische Fallbacks [40]. Damit entfällt der externe Schriftabruf; die Verfügbarkeit der ursprünglich vorgesehenen Schriftarten ist dadurch jedoch nicht nachgewiesen. Eine Prüfung ihrer lokalen Installation ist für den damaligen Stand nicht dokumentiert. Für eine verlässlich gleiche Offline-Darstellung hätten die benötigten Schriftdateien mitgeliefert werden können, beispielsweise über Fontsource-Pakete, deren CSS-Imports Vite als lokale Build-Ressourcen verarbeitet [43]. Diese Alternative wurde im Demonstrator nicht umgesetzt.

Mitkopierte Paket- und Konfigurationsdateien wurden nach dem Abgleich mit der Electron-Struktur im Commit `8b9bd66` entfernt: `package.json`, `vite.config.ts`, `tsconfig.json`, `components.json` und die zusätzliche `index.html`. Der Renderer verwendet `src/renderer/index.html` und die Paket- und Build-Konfiguration des Zielprojekts [40].

Die Replit-Plugins, `PORT`, `BASE_PATH` sowie Webserver- und Ausgabeeinstellungen der ursprünglichen Vite-Konfiguration wurden nicht übernommen [28]. electron-vite verwaltet Main, Preload und Renderer in einer gemeinsamen Konfiguration; für den Renderer verwendet es standardmäßig `src/renderer` und die dortige `index.html` [44].

Im Commit `ceccf8f` wurde der Alias `@` in `electron.vite.config.ts` unter `renderer.resolve.alias` auf `src/renderer/src` gesetzt. `tsconfig.web.json` erhielt die entsprechende Zuordnung für `@/*`; `@renderer` blieb erhalten. So konnten Imports wie `@/components/ui/badge` weiterverwendet werden [40]. Vite benötigt die Zuordnung für den Build, TypeScript für die Modul- und Typprüfung. TypeScript-`paths` allein schreibt die ausgegebenen Importpfade nicht um [45].

Für die Styles wurde `@tailwindcss/vite` mit `tailwindcss()` neben `react()` registriert, entsprechend dem dokumentierten Tailwind-Integrationsweg [46]. Diese Anpassung und die Vite-Typreferenz in `src/renderer/src/env.d.ts` gehören zur Oberflächenintegration im Commit `f03ab7f`; dessen Generatoränderungen wurden bereits in Abschnitt 5.2.4 behandelt [40].

Die Paketdefinition wurde anhand des Ausgangsartefakts um Radix UI, `clsx`, `class-variance-authority`, `tailwind-merge`, React Hook Form samt Resolver, React Query, `date-fns`, Lucide und Wouter ergänzt. Die shadcn/ui-Komponenten wurden als Quelldateien übernommen. Die Abhängigkeitsauflösung entstand im Zielprojekt neu: Das Ausgangsartefakt verwendet `pnpm-lock.yaml`, das Electron-Projekt `package-lock.json` [28], [40].

5.2.9 Anbindung der Seiten, Datumswerte und Navigation

Die bisherigen Imports aus `@workspace/api-client-react` wurden in den Seiten und im Layout durch Imports aus `@/api` ersetzt. Dieses Modul exportiert die für IPC erzeugten Hooks und zugehörigen Typen. Die Generatoranpassungen aus Abschnitt 5.2.4 ermöglichten, vorhandene Hook-Namen und typische Aufrufmuster beizubehalten. Beispielsweise blieb die Verwendung von `useListEvents(monthParams)` beziehungsweise einer Mutation mit `{ data: ... }` erhalten. Die Änderung des Imports allein genügte allerdings dort nicht, wo sich die Datendarstellung an der Schnittstelle änderte [40].

Im Commit `ce275e4` wurde beispielsweise in `pages/kalender.tsx` die Verarbeitung von Datumsstrings an die generierten `Date`-Typen angepasst. `eventsForDay` verwendet `new Date(e.startDate)` anstelle von `parseISO`; für das Enddatum gilt dieselbe Anpassung. Formulareingaben werden in `Date`-Werte

umgewandelt und Abfragegrenzen als solche übergeben. Vorhandene Werte können direkt formatiert oder über `getTime()` sortiert werden. Das Dashboard wurde entsprechend angepasst [40].

Ungenutzte Imports, Variablen und Zustandseinträge wurden ebenfalls entfernt. Die geerbte Electron-Toolkit-Konfiguration aktiviert `noUnusedLocals` und `noUnusedParameters`; das Ausgangsartefakt setzte `noUnusedLocals` auf `false` [28], [40]. Die Bereinigung war daher auch für die Typprüfung erforderlich, da TypeScript ungenutzte lokale Deklarationen bei aktivierter Prüfung als Fehler meldet [47].

Die direkten HTTP-Dateiimporte der kopierten Kalender- und Notenseiten wurden im nachfolgend beschriebenen Importschritt durch lokale Operationen ersetzt [28], [40].

Für die Navigation wurde das bereits verwendete Paket `wouter` beibehalten. Im Web-Prototyp erhielt dessen Router den aus `import.meta.env.BASE_URL` abgeleiteten Basispfad. Im Authentifizierungscommit `1c1988d` wurde stattdessen `useHashLocation` aus `wouter/use-hash-location` eingebunden und mit `<WouterRouter hook={useHashLocation}>` aktiviert [40]. Die in Abschnitt 2.4.1 erläuterte Navigation über das Adressfragment ermöglicht etwa `#/kalender`, ohne den Pfad der geladenen Renderer-Ressource zu ändern. Die vorhandenen React-Routen und Navigationskomponenten konnten auf dieser Grundlage weiterverwendet werden.

5.2.10 Lokale Authentifizierung als erster vollständiger Verarbeitungspfad

Nach der Übernahme der Oberfläche begrenzte der vorhandene Anmeldeablauf zunächst den Zugang zu den fachlichen Seiten. Das Layout ruft `useGetMe` auf und setzt einen zugeordneten Benutzer voraus. Erwogen wurden das Entfernen der Anmeldung oder eine automatische Zuordnung zu einem Standardbenutzer im Hintergrund. Diese Varianten hätten Änderungen an der bestehenden Benutzerführung und Kontozuordnung erfordert. Die Anmeldung wurde beibehalten, weil sie zum ursprünglichen Funktionsumfang gehörte und ihre Entfernung für die lokale Ausführung technisch nicht erforderlich war. Sie wurde als erste Handlergruppe implementiert, sodass der vorbereitete Aufrufweg zwischen Oberfläche, Preload, Dispatcher und lokaler Datenhaltung an einem zusammenhängenden Ablauf genutzt werden konnte. Dieser Schritt ist im Commit `1c1988d` dokumentiert [41].

`auth.handlers.ts` ordnet `register`, `login`, `logout` und `getMe` den Service-Aufrufen zu und übersetzt deklarierte Fehler. `AuthService` übernimmt Registrierung, Passwortprüfung und Benutzerzuordnung; `UsersRepository` führt die Drizzle-Abfragen aus. Nach der Datenbankinitialisierung werden die Module erzeugt und die Handler registriert. Damit ist die in A2 geforderte Trennung erstmals für eine vollständige Operationsgruppe umgesetzt [41].

Für das Hashen und Vergleichen der Passwörter wurde das bereits im Web-Backend verwendete Paket `bcryptjs` mit dem dort angegebenen Versionsbereich `^3.0.3` übernommen. Passwort-Hashes bleiben im Main-Prozess und in der lokalen Benutzertabelle; die Oberfläche erhält nur die vorgesehenen Benutzerdaten. An die Stelle der HTTP-Sitzung tritt `SessionService`, der die aktuelle Benutzerkennung im Main-Prozess hält. Diese lokale Sitzung liegt im Arbeitsspeicher und endet beim Beenden der Anwendung; die Benutzerdatensätze bleiben dagegen in der SQLite-Datenbank gespeichert. Die lokale Anmeldung ist von der später ergänzten Authentifizierung gegenüber entfernten Kollaborationsdiensten zu unterscheiden [28], [41].

Bei erfolgreicher Anmeldung oder Registrierung leert der Renderer den bisherigen React-Query-Cache, speichert die Benutzerinformation unter dem Query Key von `getMe` und navigiert zur Anwendung. `UNAUTHENTICATED` führt zur Anmeldung und wird von der Query-Wiederholungslogik berücksichtigt. Andere Ladefehler erhalten eine Anzeige mit erneuter Abfragemöglichkeit. Die Weiterleitung bei bestehender Anmeldung erfolgt über einen React-Effekt [41].

5.2.11 Integration der CRUD-Funktionen am Beispiel der Notenverwaltung

Nach der Oberflächenintegration und der lokalen Authentifizierung wurden die weiteren fachlichen Funktionen mit dem vorbereiteten Aufrufweg verbunden. Die Migration der CRUD-Operationen wird exemplarisch an der Notenverwaltung beschrieben. Anlegen, Lesen, Bearbeiten und Löschen folgen in den persönlichen Anwendungsbereichen einer wiederkehrenden Grundstruktur; eine separate Beschreibung jeder Variante würde überwiegend dieselben Transformationsschritte wiederholen. Die Notenverwaltung wurde im Commit `f97642b` integriert, zunächst ohne CSV-Import. Als fachliche Vorlage dienten die ursprünglichen Routen aus `artifacts/api-server/src/routes/grades.ts` [28], [48].

Die Übertragung bestand darin, die innerhalb der Route verbundenen Aufgaben nach ihren Verantwortlichkeiten aufzuteilen. HTTP-spezifische Zugriffe auf `req` und `res` entfielen. Die Eingabeprüfung wurde durch die bereits eingerichtete Vertragsvalidierung des Dispatchers übernommen. Die Verarbeitung der Operationen und ihrer Fehler wurde den Handlern zugeordnet, die fachlichen Regeln dem Service und die Drizzle-Abfragen dem Repository. Tabelle 5.5 zeigt diese Zuordnung am untersuchten Beispiel [48].

Schicht	**Zielmodul**	**Aufgabe im Notenbeispiel**
Schnittstelle	`grades.handlers.ts`	Operationszuordnung, Übernahme der Sitzungskennung und Übersetzung fachlicher Fehler.
Fachlogik	`GradesService`	Anlegen und Ändern fachlicher Werte, Notenstatistik und Veranlassung von Aktivitätseinträgen.
Datenzugriff	`GradesRepository`	Lesen und Schreiben der Notendatensätze über Drizzle mit benutzerbezogenen Auswahlbedingungen.

Tabelle 5.5: Aufteilung der ursprünglichen Notenrouten auf Handler, Service und Repository.

Die Module wurden in `src/main/index.ts` in den Startablauf eingebunden. Nach der Datenbankinitialisierung werden `GradesRepository` und `ActivityLogRepository` mit der Datenbankverbindung erzeugt. Beide werden an `GradesService` übergeben. `createGradesHandlers` verbindet anschließend die Service-Aufrufe mit den vorgesehenen Operationskennungen und der bereits vorhandenen `SessionService`-Instanz. Die resultierenden Handler werden in die gemeinsame Registry aufgenommen, die der IPC-Dispatcher verwendet. Die fachliche Implementierung wird damit an die zuvor erzeugten Schnittstellen angeschlossen [48].

Die Benutzerkennung stammt bei diesen Operationen aus der lokalen Sitzung im Main-Prozess. Der gemeinsame Wrapper `authenticated` stellt sie dem Handler bereit. Im Repository werden Änderungen und Löschungen anhand der Datensatzkennung und der Benutzerkennung eingeschränkt. Findet eine solche Operation keinen passenden Datensatz, meldet der Service einen fachlichen Fehler, den der Handler in `NOT_FOUND` überführt. Damit bleibt die persönliche Datenzuordnung Bestandteil des vollständigen Verarbeitungspfads [48].

Die vorhandenen fachlichen Regeln wurden als Ausgangspunkt beibehalten. Dazu gehören optionale Werte, die Erfassung von Aktivitäten, der ECTS-gewichtete Notendurchschnitt und die Auswertung nach Semester. Anpassungen ergaben sich aus der lokalen Datenrepräsentation: Note und ECTS

werden im SQLite-Schema als numerische Werte verarbeitet, sodass die bisher für PostgreSQL verwendete Umwandlung in Strings beziehungsweise deren Rückkonvertierung entfallen konnte. Die Migration übernahm somit die fachlichen Berechnungen und gliederte ihre Ausführung neu [28], [48].

Für die integrierten CRUD-Funktionen ergibt sich der Aufrufweg **Formular → Hook → Preload/ IPC → Dispatcher → Handler → Service → Repository → SQLite**. Bei der Notenerfassung übermittelt die vorhandene Seite beispielsweise die Formulardaten über `useCreateGrade` beziehungsweise `useUpdateGrade`. Der generierte Hook bildet die Eingabe auf den IPC-Vertrag ab. Nach Prüfung und Ausführung gelangt das Ergebnis über den Dispatcher und den API-Client zurück zum Hook. Die Oberfläche aktualisiert anschließend die Abfragen für Notenliste und Statistik [48].

Diese Grundstruktur kann für weitere CRUD-Bereiche erneut angewandt werden. Dabei sind jeweils die zugehörigen Vertragsdefinitionen, fachlichen Regeln, Datenzugriffe und Berechtigungen zu berücksichtigen. Die Wiederholung betrifft die Verteilung der Verantwortlichkeiten und den Kommunikationsweg; die Berechtigungsregeln gemeinsamer Gruppenressourcen sind gesondert von der persönlichen Datenzuordnung zu prüfen. Die Struktur konkretisiert damit A2 und das in A8 geforderte Erweiterungsmuster.

`scripts/grades/grades.test.ts` verbindet den generierten Client mit Dispatcher, Handlern, Service, Repositories und einer SQLite-Datenbank im Arbeitsspeicher. Geprüft werden Sitzung, persönliche Datenzuordnung, Änderungen, Aktivitäten und Statistik. Der direkte Dispatcher-Aufruf ersetzt die native IPC-Kommunikation. Der Test prüft damit die Fachlogik samt Datenzugriff; Oberfläche und Electron-Laufzeit werden gesondert bewertet [48].

Der vorhandene Gruppenbereich wurde in dieser Phase ebenfalls als lokaler CRUD-Bereich übernommen. `GroupsService` und `GroupsRepository` verwenden die Kerndatenbank; die Oberfläche ist unter `/gruppen` erreichbar. Die spätere Kollaboration ergänzt einen eigenen synchronisierten Bereich unter `/gruppen-sync`. Diese beiden Implementierungen sind im Demonstrator nebeneinander vorhanden und werden bei der Darstellung von Phase 2 unterschieden [49].

5.2.12 Lokaler Dateiimport am Beispiel iCal

Das Importmuster wird am iCal-Import aus Commit `fe08220` beschrieben. Im Web-Prototyp übermittelte die Kalenderseite die gewählte Datei als `FormData` mit `fetch` an `/api/events/import-ical`. Multer nahm den Upload entgegen; der Route-Handler interpretierte die Kalenderdaten mit `node-ical` und schrieb die Termine in die Datenbank. Bereits vorhanden waren die Übernahme von Titel, Beschreibung und Ort, die Ableitung des Termintyps, eine Standarddauer bei fehlendem Enddatum, die Mehrtageskennzeichnung und der Aktivitätseintrag. Die fachliche Grundlage des Imports stammt somit aus dem Ausgangsartefakt [28], [50].

In der Desktop-Anwendung wurde diese Upload-Verarbeitung in eine lokale Importoperation überführt. Hierfür wurde der OpenAPI-Vertrag um `importEventsIcal` und das Ergebnisschema `EventIcalImportResult` ergänzt. Anschließend wurden Client, Hook und Validierungsschemata neu erzeugt. Die Operation besitzt keine Dateipfad- oder Inhaltsparameter: Der Renderer löst über `useImportEventsIcal` die Auswahl aus, während der Main-Prozess die Datei ermittelt und verarbeitet. Damit wird die in Abschnitt 5.2.4 beschriebene zentrale Vertragspflege an einer zusätzlichen Schnittstelle angewandt [50].

Die Dateiauswahl übernimmt `NativeIcalFilePicker` im Main-Prozess. Er verwendet `dialog.showOpenDialog` mit einem Filter für ICS-Dateien und bindet den Dialog, sofern verfügbar, an das aus dem IPC-Absender ermittelte Anwendungsfenster [50]. Electron liefert aus diesem Dialog die ausgewählten Pfade beziehungsweise einen Abbruchzustand zurück [51]. Der Pfad bleibt innerhalb der nativen Auswahl und wird nicht vom Renderer als frei wählbares Argument übergeben. Die Imple-

mentierung prüft die Endung `.ics`, den regulären Dateityp und eine Größenobergrenze von 10 MiB vor sowie nach dem Einlesen. Anschließend stellt sie den Inhalt als UTF-8-Text für die Verarbeitung bereit [50].

Die Interpretation der Kalenderdaten wurde in `src/main/services/events-ical.parser.ts` ausgelagert. Das Modul verwendet weiterhin `node-ical` und die übernommenen fachlichen Abbildungen. Bereits der Web-Handler übersprang Einträge ohne Titel oder Startdatum. Ergänzt wurden die Prüfung der grundlegenden Kalenderstruktur, die Prüfung gültiger Datumswerte und die Zurückweisung eines Enddatums vor dem Startdatum. Übersprungene Einträge werden nun gezählt. Der Parser erzeugt die Termindaten ohne Benutzerkennung und ohne Datenbankzugriff. Die Eigenleistung der Migration liegt damit in der neuen lokalen Dateischnittstelle, der Aufteilung der Verarbeitung und der ergänzten Validierungs- und Ergebnisbehandlung; die Bibliothek und wesentliche fachliche Importregeln wurden wiederverwendet. Die Größenobergrenze von 10 MiB bestand bereits im ursprünglichen Multer-Upload und wurde an die lokale Dateiverarbeitung angepasst [28], [50].

`EventsService.importEventsIcal` übernimmt die fachliche Verarbeitung der Parserausgabe. Der Service ergänzt die aus der Sitzung stammende Benutzerkennung, übergibt die Datensätze an `EventsRepository.createManyEvents` und veranlasst den Aktivitätseintrag zum Import. Dateiauswahl, Formatinterpretation, fachliche Zuordnung und Datenzugriff erhalten dadurch getrennte Zuständigkeiten. Beim Start wird der native Picker als Abhängigkeit an `createEventsHandlers` übergeben. Die Schnittstelle `IcalFilePicker` ermöglicht zugleich, die Dateiauswahl in Tests durch eine geeignete Implementierung zu ersetzen [50].

Für den Import wurde eine ausdrückliche Ergebnisbehandlung eingerichtet. Ein erfolgreicher Import liefert `status: imported` sowie `importedCount` und `skippedCount`. Ein abgebrochener Dateidialog liefert `status: cancelled` mit beiden Zählwerten `0` und verändert den Datenbestand nicht. Ungültige Dateiauswahl oder Kalenderinhalte werden im Handler über die vorgesehenen Fehlerklassen in `VALIDATION_ERROR` und die deklarierte Fehlerantwort überführt. Diese Ergebnisse durchlaufen denselben Dispatcher und API-Client wie die CRUD-Operationen [50].

In der Kalenderseite ersetzt der generierte Mutation-Hook den direkten HTTP-Aufruf und das HTML-Dateifeld. Während der Operation wird der Importknopf über `isPending` deaktiviert. Nach einem erfolgreichen Import wird die Terminanfrage invalidiert; die Oberfläche zeigt die Anzahl importierter und gegebenenfalls übersprungener Einträge an. Ein Abbruch beendet den Ablauf ohne Erfolgsmeldung, während ein Fehler gesondert angezeigt wird. Die ursprüngliche Upload-Oberfläche wird damit zu einem Auslöser für die im Main-Prozess ausgeführte lokale Importfunktion [50].

Die Termintests prüfen Parserfälle, Benutzerzuordnung, Aktivitätseintrag, Abbruch und ungültige Kalenderinhalte. Ein ersetzter `IcalFilePicker` liefert vorgegebene Inhalte oder einen Abbruchzustand. So werden Handler, Parser, Service und Repository unabhängig vom Dateidialog geprüft. Native Dialoginteraktion und die Prüfungen beim Lesen realer Dateien benötigen einen gesonderten Nachweis [50].

Der CSV-Notenimport aus Commit `d37f1b5` verwendet dieselbe Aufteilung in nativen Dateipicker, Import-Handler, Service und Repository. Die Formatverarbeitung übernimmt hier der eigene Parser `grades-csv.parser.ts`, der Spalten erkennt und unter anderem Trennzeichen, zitierte Felder und Dezimalkommas verarbeitet. Die Vertragsoperation `importGradesCsv` und der erzeugte Hook liefern entsprechend den Import- oder Abbruchstatus sowie Zählwerte. CSV zeigt damit, dass das beschriebene Importmuster sowohl eine eingebundene Parserbibliothek als auch einen eigenen Parser aufnehmen kann [52].

5.2.13 Abschluss der Offline-Kernanwendung und Übergang zu Phase 2

Mit der lokalen Datenhaltung, der übernommenen Oberfläche, den generierten Aufrufhilfen und den implementierten Verarbeitungspfaden ist die erste Migrationsphase abgeschlossen. Die persönlichen Kernfunktionen werden innerhalb der Desktop-Anwendung ausgeführt: Lokale Anmeldung, Dashboard, Kalender, Noten und Stundenplan greifen über registrierte Handler, Services und Repositories auf SQLite zu; Dateiimporte werden im Main-Prozess verarbeitet. Die für Studiengänge und Stundenplanvorlagen benötigten Ausgangsdaten werden mit den Migrationen bereitgestellt. Für diese Abläufe wird kein Web-Backend benötigt [53].

Der Kontrollpunkt umfasst den Start der gebauten Anwendung ohne Netzwerk, die Kernabläufe und die Persistenz nach einem Neustart (A1 und A5). Schichtentrennung (A2) und lokale Kommunikation (A4) werden anhand ihrer Umsetzung geprüft. Die Fachlogiktests ergänzen diesen Nachweis; Laufzeit und Persistenz sind in Kapitel 6 gesondert zu bewerten. Wiederverwendung und Erweiterungsmuster unterstützen die Bewertung von A3 und A8.

Phase 2 baut auf dieser lokalen Kernanwendung auf. Sie ergänzt die Synchronisierung gemeinsamer Aufgaben sowie den Abruf externer Katalogdaten. Damit werden zusätzliche Kommunikationspfade eingeführt, während die persönlichen Daten und ihre Verarbeitung lokal bleiben.

5.3 Phase 2: Netzwerkanbindungen und Local-First-Synchronisierung

Die zweite Phase ergänzt gezielt Funktionen, die eine Verbindung zu entfernten Diensten benötigen. Die lokale Kernanwendung bleibt dabei die Grundlage. Local-First bedeutet für den hier untersuchten Aufgabenbereich, dass eine Änderung zunächst lokal verarbeitet werden kann und ihre Übermittlung als nachgelagerter Vorgang erfolgt. Die Phase verfolgt keinen Anspruch auf vollständige Umsetzung sämtlicher Local-First-Eigenschaften.

5.3.1 Auswahlkriterien und betrachtete Synchronisierungslösungen

Die Toolauswahl knüpfte an A3 an: Die in Phase 1 eingerichtete Schichtentrennung, TypeScript und Drizzle sollten möglichst weitgehend weiterverwendet werden. Zugleich verlangte A6 die Synchronisierung gemeinsamer Aufgaben nach lokaler Verarbeitung. Eine Eigenentwicklung hätte zusätzlich Verfahren für ausstehende Änderungen, Wiederholungen nach Verbindungsunterbrechungen, den Abgleich der Datenbestände und Konfliktbehandlung erfordert. Um den Umfang der Arbeit auf die Migration und ihre Muster zu begrenzen, wurde deshalb nach einer vorhandenen Synchronisierungslösung gesucht.

Für die Vorauswahl waren insbesondere lokale SQLite-Verarbeitung, eine Drizzle-Anbindung, die Ausführung im Electron-Main-Prozess und die Anbindung eines entfernten Backends relevant. In die engere Auswahl kamen PowerSync und Turso. PowerSync stellt mit `@powersync/drizzle-driver` eine Anbindung für typisierte Drizzle-Abfragen bereit [54]. Für Turso besteht eine Drizzle-Anbindung über den bereits verwendeten libSQL-Treiber [55]. Beide Ansätze boten damit Anknüpfungspunkte an die lokale Datenhaltung aus Phase 1.

Die Kandidaten übernehmen unterschiedliche Aufgaben. PowerSync bildet eine Synchronisierungsschicht zwischen einer Backend-Datenbank und lokalem SQLite; für die Node.js-Laufzeit steht ein eigenes SDK bereit [56]. Turso verbindet eine SQLite-kompatible Datenbank mit einem Cloud-Dienst und Replikationsmöglichkeiten. Die dokumentierten Embedded Replicas unterstützen lokale Datenhaltung und einen Abgleich mit Turso Cloud [57]. Tabelle 5.6 fasst die für die Vorauswahl relevanten Anknüpfungspunkte zusammen.

Aspekt	PowerSync	Turso/libSQL
Rolle	Synchronisierungsschicht vor einer Backend-Datenbank.	Datenbank und Cloud-Dienst mit lokalen Replikaten.
Drizzle-Anbindung	Wrapper und Ableitung des lokalen Schemas über `@powersync/drizzle-driver`.	Datenzugriff über `drizzle-orm/libsql` und den libSQL-Client.
Entfernte Datenhaltung	Anbindung von Supabase/ PostgreSQL im gewählten Entwurf.	Abgleich mit der Datenbank in Turso Cloud.

Tabelle 5.6: Dokumentationsbasierte Gegenüberstellung der für die Vorauswahl betrachteten Integrationsansätze [54], [55], [57], [58].

5.3.2 Entscheidung für PowerSync und Supabase

Für den Demonstrator wurde eine Architektur mit entferntem Backend gewählt, das den gemeinsamen Datenbestand der Gruppen hält. Ein Peer-to-Peer-Ansatz wäre grundsätzlich eine andere Entwurfsrichtung; er wurde in dieser Arbeit nicht weiterverfolgt. Auch für das Backend sollten vorhandene Dienste verwendet werden, um die Infrastrukturentwicklung innerhalb des festgelegten Untersuchungsumfangs zu halten.

Die Wahl fiel auf Supabase, das eine PostgreSQL-Datenbank sowie Authentifizierung mit SDKs und API-Endpunkten bereitstellt [59], [60]. Ausschlaggebend war seine dokumentierte Integration mit PowerSync: Ausgewählte PostgreSQL-Daten werden in lokale SQLite-Datenbanken übertragen; lokale Änderungen werden gespeichert, in eine Upload-Warteschlange aufgenommen und bei verfügbarer Verbindung über einen Connector an das Backend übermittelt. Sync Streams legen fest, welche Daten an die jeweiligen Clients übertragen werden [58].

Diese Kombination passte zum ausgewählten Gruppenfeature: Unterschiedliche Benutzer sollen auf gemeinsame Aufgaben zugreifen und Änderungen zunächst lokal ausführen können. PowerSync verband dafür die lokale SQLite-Verarbeitung und die Drizzle-Anbindung mit Supabase als entferntem Backend. Die Anwendung musste weiterhin ihre Datenschemata, den Upload-Connector und die fachlichen Zugriffsregeln implementieren. Der Einsatz der Dienste begrenzte damit die zusätzlich zu entwickelnde Synchronisierungsinfrastruktur.

Die Entscheidung gegen Turso beruht auf dieser projektbezogenen Passung. Auch Turso ermöglicht Synchronisierung und Offline-Schreibzugriffe; die Turso-Sync-Dokumentation beschreibt hierfür lokale Änderungen mit späterem `push()` und den Empfang entfernter Änderungen über `pull()` [61]. Für den gewählten Supabase-Entwurf bot PowerSync eine unmittelbar dokumentierte Verbindung von Authentifizierung, Backend-Datenbank und gezielter Datenverteilung [58]. Die Auswahl begründet daher die verwendete Kombination, ohne eine allgemeine Überlegenheit gegenüber Turso zu behaupten.

5.3.3 Vorprüfung der Integration und Grenzen der Wiederverwendung

Die praktische Integrationsprüfung ist für PowerSync in zwei begrenzten Prototypen dokumentiert. Der erste prüfte die Node-SDK-Ausführung im Electron-Main-Prozess, lokale Persistenz samt Upload-Warteschlange, Schemaerweiterung, Geräteabgleich und Paketierung. Der zweite ergänzte den Drizzle-Wrapper und prüfte typisierte CRUD-Abfragen sowie die Abbildung von Datentypen. Die Projektberichte dokumentieren erfolgreiche lokale Prüfungen und einen Abgleich über Supabase und PowerSync. Sie stützten die Entscheidung, zunächst einen ausgewählten Kollaborationsbereich zu

integrieren. Diese Vorprüfung untersucht die gewählte Kombination; sie ist kein praktisch durchgeführter Vergleich beider Kandidaten [62].

Die Drizzle-Anbindung ermöglicht Wiederverwendung auf der Ebene der Typdefinitionen, Abfragen und Repository-Muster. Im Prototyp wird das lokale PowerSync-Schema mit `DrizzleAppSchema` aus geeigneten Drizzle-Definitionen abgeleitet; `wrapPowerSyncWithDrizzle` stellt den typisierten Datenzugriff bereit [62].

Für synchronisierte Tabellen übernimmt PowerSync die Verwaltung der lokalen Datenbankstruktur. Drizzle Kit bleibt für die persönliche Datenbank aus Phase 1 zuständig; serverseitige PostgreSQL-Migrationen und die Sync-Stream-Konfiguration werden gesondert gepflegt [62]. Damit konnte Drizzle als typisierte Datenzugriffsschicht erhalten bleiben, während die Verwaltung des synchronisierten Datenbestands und seiner Identitäten gezielt angepasst wurde. Dieser Unterschied bestimmt die nachfolgende Umsetzung der Gruppenfunktionen.

5.3.4 Architektur mit zwei lokalen Datenhaltungsbereichen

Mit PowerSync und Supabase erweitert sich die lokale Architektur um entfernte Dienste. Der Renderer verwendet weiterhin die begrenzte Preload-/IPC-Schnittstelle; Datenzugriff und Dienstanbindung werden im Main-Prozess koordiniert. Persönliche Funktionen und ursprüngliche lokale Gruppen verbleiben in der SQLite-Datenbank aus Phase 1. Der neue Kollaborationsbereich verwendet eine weitere lokale SQLite-Datenbank, deren Struktur und Synchronisierung PowerSync verwaltet. Für jede entfernte Benutzeridentität wird eine eigene Datei geöffnet. Es bestehen damit zwei lokale Datenhaltungsbereiche; hinzu kommt der gemeinsame PostgreSQL-Datenbestand bei Supabase [63].

Abbildung 5.2 zeigt die getrennten Kommunikationswege. Aufgabenänderungen werden zunächst lokal gespeichert und über den Upload-Connector an die Supabase-API übermittelt. PowerSync überträgt die anhand der Sync Streams ausgewählten Backend-Daten zurück in den lokalen Kollaborationsbestand. Der Upload führt somit über die Anwendungs-API, während der Download durch den PowerSync-Dienst erfolgt [58], [63].

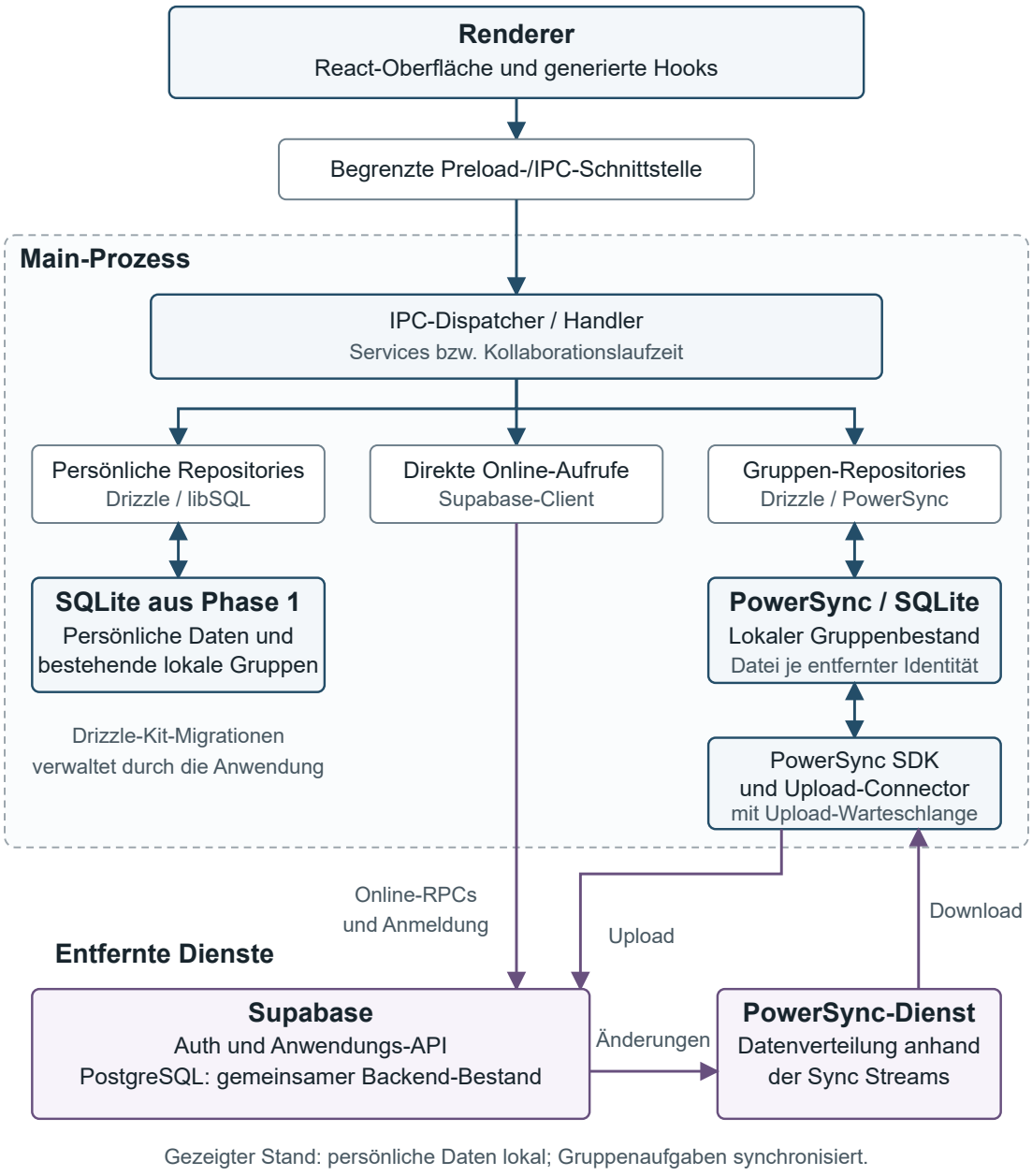


Abbildung 5.2: Architektur nach Ergänzung des synchronisierten Gruppenbereichs. Dargestellt sind die beiden lokalen Datenhaltungsbereiche sowie direkte Online-Aufrufe, Upload und Download. Der separate Katalogabruf ist hier nicht dargestellt. Eigene Darstellung nach [63] und [58].

Gruppenanlage, Profilsuche und Einladungen verwenden unmittelbare Online-Aufrufe. Die entfernte Sitzung wird im Demonstrator nur im Arbeitsspeicher gehalten; nach einem Neustart ist eine erneute Anmeldung erforderlich. Die lokale Speicherung von Aufgaben bedeutet deshalb keine uneingeschränkte Offline-Verfügbarkeit aller Gruppenoperationen [63].

5.3.5 Technische Integration der Kollaborationsmodule

Die Integration begann mit der Abstimmung des Datenmodells zwischen Supabase und lokalem PowerSync. In Supabase wurden die fünf Tabellen des Gruppenbereichs durch versionierte SQL-Migrationen angelegt. Datensatz- und Benutzerkennungen werden dort als UUIDs geführt; das lokale Drizzle-Schema bildet sie als Textfelder ab. Neue Aufgaben erhalten ihre UUID bereits beim lokalen Einfügen. Zeitangaben werden lokal als ISO-Strings und boolesche Werte als SQLite-Integer repräsentiert. `schema.ts` definiert diesen lokalen Tabellenbestand und leitet daraus mit `DrizzleAppSchema` das PowerSync-Schema ab. Die neuen Kennungen unterscheiden sich damit von den numerischen Kennungen des Ausgangsmodells [64].

Anschließend wurden die Tabellen in die PostgreSQL-Publication `powersync` aufgenommen und für deren Replikationsrolle lesbar gemacht. Die Verbindung des PowerSync-Dienstes zu Supabase umfasst die Datenbankanbindung und die Anerkennung der Supabase-Anmeldetoken [58]. Die Datenverteilung wurde in `sync-streams-v4.yaml` beschrieben und als Dienstkonfiguration bereitgestellt. Die Abfragen berücksichtigen Gruppenmitgliedschaften sowie Absender und Empfänger von Einladungen. Diese Datei steuert den Download; die für Schreibzugriffe geltenden Berechtigungen sind zusätzlich im Backend definiert [64]. Die entfernten SQL-Migrationen und Sync Streams ergänzen damit das lokale Anwendungsschema und werden getrennt von der Desktop-Paketierung gepflegt.

Der Ausschnitt in Quellcode 5.5 zeigt die Auswahl gemeinsamer Aufgaben. `auth.user_id()` bezeichnet die gegenüber PowerSync authentifizierte Supabase-Identität. Die Unterabfrage `user_groups` bestimmt ihre Gruppenmitgliedschaften; die anschließende Abfrage beschränkt die Aufgaben auf diese Gruppen. Weitere Abfragen desselben Streams verteilen Gruppen, Mitglieder und zugehörige Profile. Ein eigener Stream behandelt Einladungen anhand von Absender und Empfänger [64].

```yaml
config:
  edition: 3

streams:
  collaboration_groups:
    auto_subscribe: true
    with:
      user_groups: |
        SELECT group_id
        FROM group_members
        WHERE user_id = auth.user_id()
    queries:
      - |
        SELECT id, group_id, title, description, due_date,
               assignee_id, status, created_by, updated_by,
               created_at, updated_at
        FROM group_tasks
        WHERE group_id IN user_groups
```

Quellcode 5.5: Auswahl gemeinsamer Aufgaben in `sync-streams-v4.yaml`; die übrigen Abfragen wurden weggelassen.

Die YAML-Datei wird im PowerSync-Dienst ausgewertet. Dieser übernimmt Änderungen aus der angebundenen Supabase-Datenbank und verteilt die durch den Stream ausgewählten Daten an die zugehörigen Clients. `auto_subscribe: true` aktiviert den Stream beim Verbinden automatisch. Das lokale SDK speichert die empfangenen Daten in SQLite; die Repositories lesen sie über Drizzle [65]. Der Renderer führt die Stream-Abfragen somit nicht selbst aus. In Gegenrichtung übermittelt der Connector lokale Aufgabenänderungen über die Supabase-API. Die Streams ersetzen weder diesen Upload noch die dortigen Berechtigungsprüfungen [64].

Ergänztes Paket	Aufgabe im Demonstrator
`@powersync/node`	Lokale PowerSync-Datenbank, Warteschlange und Synchronisierungsverbindung.
`@powersync/drizzle-driver`	Ableitung des Client-Schemas und typisierter Drizzle-Zugriff.
`@supabase/supabase-js`	Anmeldung, direkte RPC-Aufrufe und Upload an die Supabase-API.
`better-sqlite3`	Nativer SQLite-Treiber für die gewählte PowerSync-Implementierung.

Tabelle 5.7: Für den Kollaborationsbereich ergänzte Laufzeitabhängigkeiten und ihre Aufgaben [64].

Für die neue Datenbank wurde `better-sqlite3` gewählt, während die Datenbank aus Phase 1 weiterhin über libSQL angebunden bleibt. Das PowerSync-Node-SDK unterstützt verschiedene SQLite-Treiber; der Wechsel beschreibt somit die konkrete Implementierung des Demonstrators [56]. `database.ts` öffnet die PowerSync-Datenbank und stellt mit `wrapPowerSyncWithDrizzle` den Zugriff für die Repositories bereit. Die Ausführung verwendet einen eigenen Worker, der über `powersync.worker?nodeWorker` in den Electron-Build eingebunden wird [64].

Auch die Paketierung wurde angepasst: In `electron-builder.yml` werden `node_modules/@powersync/node/**` und `node_modules/better-sqlite3/**` unter `asarUnpack` aufgeführt. Der Worker ermittelt die native PowerSync-Erweiterung im Verzeichnis `app.asar.unpacked`. Diese Anpassung adressiert das im Prototyp beobachtete Laden der SQLite-Erweiterung unter Windows. Das vorhandene `postinstall`-Skript mit `electron-builder install-app-deps` dient dem Aufbau nativer Abhängigkeiten für die Electron-Laufzeit [62], [64]. Die Electron-Dokumentation beschreibt die erforderliche Entpackung bei APIs, die reale Dateipfade für native Module benötigen [66].

Die Anwendungsmodule erhielten getrennte Zuständigkeiten. `CollaborationRuntime` koordiniert Sitzung, Lebenszyklus und fachliche Operationen. Bei der Anmeldung öffnet sie eine zur entfernten Benutzerkennung gehörende Datenbank und verbindet diese mit dem Connector. Der gekürzte Auszug in Listing 5.6 zeigt diesen Schritt. Die Repositories lesen den lokalen Gruppenbestand und schreiben Aufgabenänderungen. `online-gateway.ts` führt die unmittelbar netzwerkabhängigen Gruppen- und Einladungsoperationen aus. Der Connector liefert über `fetchCredentials` das Anmeldetoken für PowerSync und übermittelt mit `uploadData` die ausstehenden Aufgabenänderungen an Supabase [64].

```
const userId = await connection.connector.getUserId()
const directory = join(this.userDataDirectory, 'collaboration')
database = await openCollaborationDatabase(
  directory,
  `collaboration-${userId}.db`
)
await database.powerSync.connect(connection.connector)
```

Listing 5.6: Öffnen und Verbinden der benutzerbezogenen PowerSync-Datenbank; gekürzter Auszug aus `CollaborationRuntime.signIn` [64].

Die Preload-Ebene stellt nun zwei begrenzte Aufrufwege bereit. Über `window.apiTransport.invoke` verwendet die Kernanwendung weiterhin den Kanal `api:invoke`. Für die Kollaboration wurde `window.collaborationTransport.invoke` mit dem Kanal `collaboration:invoke` ergänzt. Der neue

Modulvertrag liegt in `src/shared/collaboration/ipc.ts` und wird im betrachteten Stand eigenständig typisiert, statt durch den OpenAPI-Generator erzeugt zu werden. Der zugehörige Main-Handler prüft Absender, Operation und Eingaben mit Zod und berücksichtigt die lokale Sitzung [64]. Die beiden IPC-Pfade bilden die getrennte Integration der Funktionsbereiche ab. Zwei Datenbanktreiber erzwingen diese Aufteilung nicht: Die Schnittstellen bieten fachliche Operationen an, deren Datenzugriff hinter den Repositories liegt. Eine Zusammenführung der Datenbanken würde daher nicht automatisch die IPC-Verträge vereinheitlichen.

Abbildung 5.3 zeigt die Module und ihre Verbindungen. Sie ergänzt das Architekturbild aus Abbildung 5.2 um die konkrete Aufteilung der Verarbeitung und die beiden IPC-Pfade.

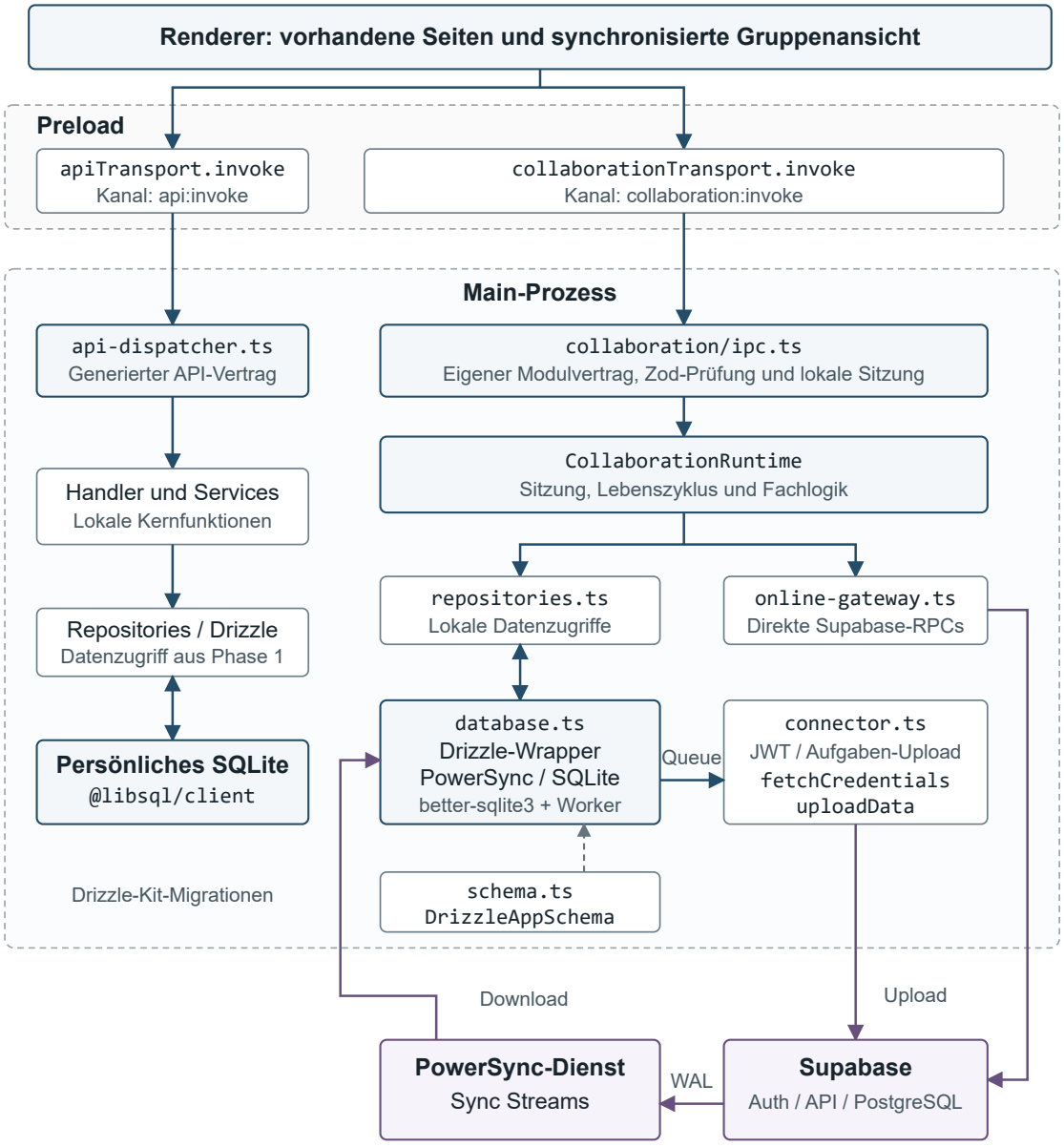


Abbildung 5.3: Modulaufteilung der Kollaborationsintegration neben der lokalen Kernanwendung. Blaue Pfeile zeigen lokale Aufruf- und Datenzugriffswege, violette Pfeile Dienstkommunikation; der gestrichelte Pfeil zeigt die Schemaeinbindung. Eigene Darstellung nach [64].

5.3.6 Anpassung des Einladungsablaufs und vorausgesetzte Konten

Der Ausgangsprompt benannte Gruppeneinladungen über Link oder E-Mail. Im Web-Prototyp erzeugte die Gruppenroute einen Einladungstoken. Die Oberfläche bildete daraus eine URL und kopierte sie in die Zwischenablage; die Weitergabe erfolgte außerhalb der Anwendung. Bereits vorhanden waren außerdem die Seite `/einladung/:token` zum Anzeigen und Annehmen der Einladung sowie Backend-Routen zum Lesen des Tokens und zum Gruppenbeitritt [67]. Der alternative Ablauf lässt sich daher nicht mit vollständig fehlender Beitrittslogik begründen.

Die statische Prüfung zeigt allerdings eine Lücke im Übergang zur Anmeldung: Die Einladungsseite übergibt mit `redirect` eine Rücksprungadresse, die Login-Seite navigiert nach erfolgreicher Anmeldung jedoch zu `/`, ohne diesen Parameter auszuwerten. Damit ist keine automatische Rückkehr zur Einladung umgesetzt [67]. Eine Laufzeitprüfung der ursprünglichen Replit-Bereitstellung wurde nicht durchgeführt; aus diesem Befund folgt kein Nachweis, dass der gesamte Einladungsablauf dort grundsätzlich unbrauchbar war.

Für den synchronisierten Gruppenbereich wurde ein anderer Einladungsablauf implementiert. Supabase Auth verwaltet die entfernten Benutzerkonten; die anwendungsbezogenen Namen und Kennungen liegen in einer zugeordneten `profiles`-Tabelle. Die Trennung zwischen Auth-Daten und über die Anwendungs-API zugänglichen Profilen entspricht dem von Supabase dokumentierten Ansatz [68]. Neu entwickelt wurde eine Suche nach Benutzer- oder Anzeigenamen. Die Funktion `search_invitable_profiles` steht dem angemeldeten Gruppeneigentümer zur Verfügung und liefert bis zu 20 passende, als auffindbar markierte Profile. Eigene Identität, vorhandene Mitglieder und Personen mit bereits ausstehender Einladung werden ausgeschlossen. Als Ergebnis werden Kennung, Benutzername und Anzeigename übermittelt [63].

Dieser Entwurf setzt für den Demonstrator voraus, dass Gruppeneigentümer die Existenz und die angegebenen Namen auffindbarer Profile kennen dürfen. Die Annahme vereinfacht die Auswahl innerhalb der Anwendung und ist eine fachliche Entscheidung der Fallstudie. Sie bedeutet keine Freigabe sämtlicher Auth-Daten und ersetzt keine Festlegung der gewünschten Auffindbarkeit in einer produktiven Anwendung.

Die Auswahl eines Profils führt zu einer personenbezogenen Einladung über `send_group_invitation`. Die eingeladene Person erhält den Eintrag durch PowerSync und kann ihn mit `respond_group_invitation` annehmen oder ablehnen. Nach der Annahme entsteht die Mitgliedschaft, anhand derer die Gruppendaten übertragen werden [63]. Die Benutzersuche und der zugehörige Einladungsablauf verändern die ursprüngliche Benutzerführung. Damit konnte die Interaktion mit dem entfernten Backend innerhalb der Anwendung demonstriert werden, ohne zusätzlich die externe Linkweitergabe und den Wiedereinstieg in die Desktop-Anwendung auszuarbeiten. Der Ablauf ist eine fachliche Anpassung des Untersuchungsumfangs und keine technisch zwingende Folge des Transportwechsels.

Für die Fallstudie wurden die benötigten entfernten Testkonten vorab über das Supabase-Dashboard angelegt. Ihre E-Mail-Adressen und Passwörter mussten somit bereits vorliegen. Die Desktop-Anwendung enthält eine Anmeldung für diese Konten: Im Main-Prozess ruft der Connector `signInWithPassword` mit den eingegebenen Zugangsdaten auf [63]. Die Methode authentifiziert bestehende Konten und übernimmt keine Registrierung [69]. Die lokale Kontoverwaltung aus Phase 1 bleibt davon getrennt.

Eine Registrierung für das entfernte Backend, E-Mail-Bestätigung und Passwortzurücksetzung wurden in der Anwendungsoberfläche nicht implementiert. Der Untersuchungsaufbau setzt eingerichtete Konten voraus, um authentifizierte Gruppenzugriffe und die Synchronisierung gemeinsamer Aufgaben zu demonstrieren. Die Ausarbeitung eines vollständigen Kontolebenszyklus würde zusätzliche Produktfunktionen und Anforderungen einführen. Für diesen begrenzten Nachweis wurden sie daher nicht entwickelt; bei einer produktiven Anwendung wären sie anhand des vorgesehenen Benutzerzugangs gesondert festzulegen. Die Fallstudie bewertet damit die Migration unter dieser Voraussetzung und weist keine vollständige entfernte Benutzerverwaltung nach.

5.3.7 Schrittweise Überführung weiterer Funktionen

Die parallele Datenhaltung ermöglicht eine Migration nach Funktionsbereichen: Ein vorhandener Bereich bleibt zunächst bestehen, während ein ausgewählter Bereich mit dem neuen Datenzugriff

umgesetzt wird. Dieses Vorgehen folgt dem Grundgedanken des Strangler-Fig-Patterns, bestehende Funktionen schrittweise durch neue Implementierungen abzulösen [24]. Im Demonstrator bleibt der in Phase 1 lokal übernommene Gruppenbereich unter `/gruppen` bestehen. Der in Phase 2 ergänzte Bereich unter `/gruppen-sync` verwendet die neue Kollaborationsdatenbank. Die Oberfläche unterscheidet beide als „Gruppen“ und „Gruppen (Sync)“. Die Koexistenz ist ein Übergangsstand der Umsetzung; bestehende lokale Gruppendaten werden nicht automatisch in den synchronisierten Bereich übertragen [49].

Das Terminbeispiel aus der Vorprüfung konkretisiert die erforderlichen Anpassungen. Die ursprüngliche `events`-Tabelle besitzt numerische, automatisch erzeugte Kennungen und lokale Benutzerkennungen. Für die separate Prototyptabelle `spike_events` wurden clientseitige UUIDs, eine Supabase-Benutzerzuordnung und geeignete Datentypabbildungen verwendet. Geprüft wurden typisierte CRUD-Zugriffe, Persistenz ausstehender Änderungen und der Abgleich zwischen Replikaten. Damit liegt ein Beispiel für die technische Überführung eines weiteren Datenbereichs vor; die Übernahme vorhandener Termine einschließlich ihrer Referenzen wurde damit noch nicht durchgeführt [62]. Weitere Bereiche würden dieselben Grundschritte durchlaufen: Schema und Identitäten anpassen, Daten und Referenzen zuordnen, Repositories anbinden und das Verhalten prüfen. Die Umsetzung jedes weiteren CRUD-Pfads würde das bereits gezeigte Muster wiederholen und wird daher nicht vollständig ausgearbeitet.

Quellcode 5.7 und Quellcode 5.8 zeigen ausgewählte Felder vor und nach dieser Anpassung. Der Vergleich beginnt beim bereits auf SQLite übertragenen Schema aus Phase 1. Im Prototyp ersetzt eine Textkennung den automatisch erzeugten Integer; `ownerId` verweist auf die entfernte Supabase-Identität. Datumswerte werden als Text gespeichert: Das Repository erzeugt beim Schreiben ISO-Strings und wandelt sie beim Lesen wieder in `Date`-Werte um. Aus den Drizzle-Tabellen wird zusätzlich das Schema für PowerSync abgeleitet. Die Ausschnitte zeigen somit eine Schema- und Repository-Anpassung, keine Migration vorhandener Benutzerdaten [70].

```
export const eventsTable = sqliteTable('events', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  title: text('title').notNull(),
  startDate: integer('start_date', { mode: 'timestamp' })
    .notNull(),
  endDate: integer('end_date', { mode: 'timestamp' })
    .notNull(),
  isMultiDay: integer('is_multi_day', { mode: 'boolean' })
    .notNull().default(false),
  userId: integer('user_id').notNull()
  // Weitere Felder weggelassen.
})
```

Quellcode 5.7: Ausgewählte Felder des lokalen Terminschemas aus Phase 1; weitere Felder weggelassen.

```ts
export const spikeEventsTable = sqliteTable('spike_events', {
  id: text('id').primaryKey().notNull(),
  ownerId: text('owner_id').notNull(),
  title: text('title').notNull(),
  startDate: text('start_date').notNull(),
  endDate: text('end_date').notNull(),
  isMultiDay: integer('is_multi_day', { mode: 'boolean' })
    .notNull()
  // Weitere Felder und Indizes weggelassen.
})

export const powerSyncDrizzleSchemaDefinition = {
  spikeEventsTable
}
export const powerSyncDrizzleSchema =
  new DrizzleAppSchema(powerSyncDrizzleSchemaDefinition)
```

Quellcode 5.8: Entsprechende Felder der separaten PowerSync-Prototyptabelle und Schemaableitung; weitere Felder und Indizes weggelassen.

Eine spätere Zusammenführung in einer PowerSync-verwalteten lokalen Datenbank wäre grundsätzlich möglich, ohne sämtliche Inhalte zu teilen. Persönliche Tabellen müssten dafür ausdrücklich als lokale Tabellen konfiguriert werden, etwa mit `localOnly: true`; ihre Änderungen gelangen dann nicht in die Upload-Warteschlange [54], [71]. Das bloße Weglassen eines Sync Streams reicht hierfür nicht aus, da Streams den Download steuern. Benutzerzuordnung und Zugriff ohne entfernte Anmeldung wären bei einer solchen Zusammenführung weiterhin zu erhalten. Erst nach Ablösung aller bisherigen libSQL-Zugriffe könnte `@libsql/client` entfallen. Dazu wären die betroffenen Repositories, Datenbankinitialisierung und Schemaverwaltung zu überführen. Im gewählten Entwurf bliebe `better-sqlite3` als lokaler Treiber erhalten. Eine solche vollständige Ablösung wurde nicht geprüft; sie beschreibt eine mögliche Weiterentwicklung, nicht den erreichten Projektstand.

5.3.8 Verantwortung für Schemaänderungen und Updates

Die Einführung der Synchronisierung verändert auch die Verantwortung für spätere Schemaänderungen. In Phase 1 verwaltet die Desktop-Anwendung ihre Datenbank durch mitgelieferte Drizzle-Migrationen. Für synchronisierte Funktionen sind nun das entfernte PostgreSQL-Schema, die PowerSync-Datenverteilung und das lokale Anwendungsschema aufeinander abzustimmen. Die Supabase-Datenbank erhält eigene versionierte SQL-Migrationen [72]. Das Desktop-Update ist damit Bestandteil einer koordinierten Bereitstellung über mehrere Komponenten.

Auf Clientseite wird das mit `DrizzleAppSchema` abgeleitete Schema beim Öffnen der PowerSync-Datenbank an das SDK übergeben [64]. Im verwendeten Standardmodell speichert PowerSync Daten als JSON und stellt sie über SQLite-Views entsprechend dem lokalen Schema bereit. Eine aktualisierte Schemadefinition wird beim Start der neuen Anwendungsversion angewandt; für diese Views werden keine zusätzlichen Drizzle-SQL-Migrationsdateien benötigt [73]. Die Drizzle-Migrationen aus Phase 1 bleiben für die separate persönliche Datenbank zuständig.

Für eine additive Erweiterung, etwa eine neue synchronisierte Tabelle, ergibt sich folgender Bereitstellungsablauf:

1. Zunächst wird das Supabase-Schema erweitert. Dazu gehören die erforderlichen Zugriffsregeln und gegebenenfalls Anpassungen der Backend-Operationen.
2. Anschließend wird die Tabelle in die PostgreSQL-Publication aufgenommen und die Sync-Stream-Konfiguration angepasst. Die Verarbeitung der aktualisierten Konfiguration muss abgeschlossen sein, bevor die neue Funktion freigegeben wird.

3. Danach wird die Desktop-Version mit dem erweiterten Client-Schema und den zugehörigen Anwendungsfunktionen veröffentlicht.

Diese Reihenfolge entspricht dem von PowerSync beschriebenen Vorgehen für additive Änderungen. Umbenennungen, Löschungen oder Typänderungen können andere Übergangsschritte erfordern [74].

Die Abstimmung muss auch ältere Clients berücksichtigen, die ein Update noch nicht installiert haben oder nach einer Offline-Phase ausstehende Änderungen übertragen. Bestehende Lese- und Schreibformate sind während des Übergangs kompatibel zu halten. PowerSync beschreibt dafür unter anderem versionierte Streams und Datentransformationen; deren konkrete Verwendung hängt von der Schemaänderung ab [73]. Ein vollständiger Ablauf mit unterschiedlichen Anwendungsversionen, Backend-Änderung und anschließender Desktop-Aktualisierung wurde im Demonstrator nicht geprüft. Dieser Abschnitt benennt die daraus abgeleitete Anforderung an spätere Updates und erweitert den bisherigen Implementierungsnachweis nicht.

5.3.9 Grenzen der untersuchten Synchronisierung

Die Untersuchung umfasst die lokale Verarbeitung gemeinsamer Aufgaben und deren späteren Abgleich. Eine vollständige Übernahme persönlicher Kontodaten auf andere Geräte oder gemeinsame Echtzeitbearbeitung von Dokumenten gehört gemäß Kapitel 4 nicht dazu. Der Replikatabgleich der Vorprüfung prüft die technische Datenübertragung, ersetzt jedoch keine vollständige Kontomigration [62].

Der Web-Prototyp verarbeitete Änderungen unmittelbar über sein autoritatives Backend. Eine fachliche Zusammenführung konkurrierender Offline-Änderungen war dort nicht implementiert [28]. Local-First ergänzt diese Entwurfsfrage: Regeln für konkurrierende Bearbeitungen, Löschkonflikte oder manuelle Konfliktentscheidungen müssen aus den Anforderungen der jeweiligen Anwendung abgeleitet werden. Ihre umfassende Entwicklung würde neue Fachfunktionalität schaffen und geht über die hier demonstrierte Migration hinaus.

Auch der Demonstrator besitzt deshalb keine eigens entwickelte Konfliktzusammenführung. Der Connector des ursprünglichen Integrationsstands übermittelte Aufgabenänderungen mit `upsert`, `update` und `delete` an Supabase, ohne eine eigene Versionsprüfung [63]. Die spätere Upload-Nachbesserung ergänzt eine Backend-Operation mit Wiederholungsschutz und ein lokales Ablehnungsjournal; ihre Ergebnisse werden in Abschnitt 6.4 bewertet [75]. Eine als Last Write Wins bezeichnete Regel müsste ausdrücklich festlegen, welche Reihenfolge maßgeblich ist; ihre fachliche Eignung ist hier nicht evaluiert. Die Begrenzung betrifft erweiterte Konfliktstrategien. Warteschlange, Wiederholungen und der Umgang mit abgelehnten Änderungen bleiben für die Migration relevant. PowerSync verwendet ein serverautoritatives Modell; aus einem lokalen Schreibabschluss folgt noch keine Bestätigung durch das Backend [20].

5.3.10 Online-Abruf des Studienkatalogs

Der Studienkatalog ergänzt die Synchronisierung um einen einfacheren Netzwerkablauf: Fachliche Referenzdaten werden online bereitgestellt, in die lokale Datenbank übernommen und anschließend offline gelesen. Damit können Kataloginhalte unabhängig von einer neuen Anwendungsversion aktualisiert werden. Der Demonstrator ergänzt hierzu den in Phase 1 ausgelieferten Seed-Bestand um online abrufbare Beispieldaten; der vorhandene Grundbestand bleibt erhalten [76].

Im Commit `9320911` wurde dafür eine Supabase Edge Function ergänzt, die Studiengänge und zugehörige Lehrveranstaltungen aus einer JSON-Datei über einen öffentlichen HTTP-Endpunkt bereitstellt. Dieser Endpunkt benötigt weder ein Benutzerkonto noch einen Zugriff auf die entfernte Datenbank. Änderungen am angebotenen Katalog erfordern eine erneute Bereitstellung dieser Datei mit der Funktion, jedoch keine neue Version der Desktop-Anwendung. Der bestehende OpenAPI-

Vertrag wurde um `refreshStudyCatalog` und dessen Ergebnis mit den Anzahlen übernommener Studiengänge und Lehrveranstaltungen erweitert. Daraus wurden die IPC-Vertragsbestandteile und der React-Query-Hook erneut generiert [76].

Beim Öffnen der Studienseite liest die Oberfläche zunächst den lokalen Bestand und löst zusätzlich die Aktualisierung aus. Der Main-Prozess lädt die JSON-Daten, validiert sie anhand der vorhandenen Feldschemata und übernimmt beide Datenbereiche gemeinsam in einer Transaktion. Anhand stabiler Kennungen werden neue Einträge eingefügt und vorhandene aktualisiert, sodass wiederholte Abrufe keine Duplikate erzeugen. In der Antwort fehlende Einträge werden nicht gelöscht. Nach erfolgreicher Übernahme werden die betroffenen Abfragen der Oberfläche erneut ausgeführt [76].

Scheitern Abruf, Validierung oder Datenbankübernahme, bleibt der bisherige lokale Bestand nutzbar. Die Oberfläche zeigt den Fehler an und bietet einen erneuten Versuch an. Bereits übernommene Katalogdaten bleiben auch nach einem Neustart lokal verfügbar. Der untersuchte Stand ruft den Katalog bei jedem Seitenaufruf erneut ab; eine Versionsprüfung zur Erkennung neuer Katalogstände wurde nicht implementiert. Er zeigt damit das Muster eines Downloads mit dauerhafter lokaler Datenhaltung. Die Desktop-Anwendung lädt dabei keine Katalogänderungen zum entfernten Dienst hoch [76].

Der Kontrollpunkt dieser Phase betrifft insbesondere A6 und A8. Für Aufgaben sind der lokale Abschluss einer Änderung und ihre spätere Übernahme im Kollaborationsablauf getrennt zu beobachten. Bei unmittelbar netzwerkabhängigen Operationen sind Fehlermeldung und Verhalten ohne Verbindung auszuweisen. Bereits lokal verfügbare Funktionen dürfen durch den Ausfall einer zusätzlichen Online-Anbindung nicht unnötig blockiert werden.

5.4 Phase 3: Prüfung und Härtung der vervollständigten Anwendung

5.4.1 Prüfrahmen und Sicherheitsmatrix

Nach der funktionalen Umsetzung werden die Sicherheitsgrenzen der entstandenen Anwendung zusammenhängend geprüft. **Prüfung** bezeichnet hier den Abgleich einer Anforderung mit der tatsächlichen Implementierung und den verfügbaren Belegen. **Härtung** bezeichnet die daraus abgeleiteten Änderungen, mit denen unnötige Befugnisse eingeschränkt oder festgestellte Abweichungen behoben werden. Die bereits in den ersten beiden Phasen eingerichtete Prozess- und Schichtentrennung bleibt dabei die Grundlage; Sicherheitsmaßnahmen entstehen somit auch während der Migration und werden in dieser Phase konsolidiert.

Den Bezugsrahmen bildet A7: Sämtliche Empfehlungen der Electron-Sicherheitscheckliste sind auf Anwendbarkeit, Umsetzung und Nachweistiefe zu prüfen [23]. Hinzu kommen die in Abschnitt 4.4 und der Abgrenzung festgelegte Eingabevalidierung, anwendungsinterne Zugriffskontrolle und Verarbeitung externer Daten. A2 und A4 bestimmen die Verantwortlichkeiten entlang des Aufrufwegs; A5, A6 und A8 legen die betrachteten lokalen Funktionen, Kollaborationsabläufe und Interaktionsformen fest. Diese Anforderungen bestimmen die Prüforte, ohne selbst bereits einen Sicherheitsnachweis zu liefern.

Für den Abgleich wurden ein Prüfplan, eine Ergebnismatrix sowie Protokolle der gezielten Prüfungen erstellt. Jede Empfehlung erhält eine begründete Anwendbarkeitsentscheidung, einen Verweis auf die betroffenen Dateien, eine Bewertung und die verwendete Nachweisebene. **Nicht anwendbar** ist ein Punkt nur, wenn der betreffende Mechanismus im Demonstrator fehlt. Eine fehlende Prüfung oder eine noch ausstehende Umsetzung wird gesondert ausgewiesen. Die konsolidierte Matrix verbindet die Einzelbewertungen mit den Quellcode-, Test-, Paket- und Laufzeitbelegen [77], [78].

5.4.2 Zuordnung der Anforderungen zu den Prüforten

Die Prüfung folgt den Grenzen, an denen Daten oder Befugnisse übergeben werden: vom Renderer an den Preload und Main-Prozess, von ausgewählten Dateien an die Parser sowie von entfernten Diensten an die lokale Anwendung. Hinzu kommen Fenster- und Sessionkonfiguration sowie Einstellungen, die erst beim Paketieren wirksam werden. Tabelle 5.8 ordnet diesen Stellen die konkrete Umsetzung und das jeweilige Prüfverfahren zu. Die Nummern beziehen sich auf alle 20 Electron-Empfehlungen; verwandte Punkte sind für die Darstellung zusammengefasst. A7 gilt für den gesamten Richtlinienabgleich. Die Einzelbewertungen verbleiben in der Projektmatrix [78].

Bezug	**Prüfstelle und Umsetzung**	**Prüfverfahren und Nachweisgrenze**
Nr. 1; A6, A8	**Netzwerkanbindung im Main:** HTTPS-Adressen ohne eingebettete Zugangsdaten für Katalog, Supabase und PowerSync prüfen. Katalog- und Supabase-Abrufe lehnen Weiterleitungen ab.	Code- und Konfigurationstests; Redirectversuch mit tatsächlichem Node-Fetch. Die SDK-interne PowerSync-Redirectbehandlung ist nicht praktisch nachgewiesen.
Nr. 2–4	**Fenster und Preload:** `nodeIntegration: false`, `contextIsolation: true`, `sandbox: true`; privilegierte Zugriffe bleiben im Main.	Codeinspektion von Fensterkonfiguration und Startpfaden. Keine zusätzliche Laufzeitabfrage der Einstellungen oder eigenständige Betriebssystem-Sandbox-Analyse.
Nr. 5	**Session-Berechtigungen:** Check- und Request-Handler erlauben ausschließlich `clipboard-sanitized-write` für das zugelassene Fenster und Dokument.	Codeinspektion der gemeinsamen Fenster- und Dokumentprüfung; keine gesonderte Berechtigungsdialog-Testreihe.
Nr. 6, 8–10	**Fenster- und Startoptionen:** Web-Sicherheit bleibt aktiv; unsichere Inhalte, experimentelle Funktionen und zusätzliche Blink-Funktionen werden nicht freigegeben.	Code- und Startpfadprüfung mit den Standardwerten der dokumentierten Electron-Version. Die Abwesenheit eines Overrides wird von einer expliziten Einstellung unterschieden.
Nr. 7	**Renderer-HTML:** CSP mit `default-src 'none'`, `script-src 'self'`, gesperrten Objekten und Frames.	Codeabgleich und P3: beobachtete Blockierung einer fremden Skriptquelle. Inline-Styles bleiben erlaubt; der Einzelversuch belegt keine allgemeine XSS-Freiheit.
Nr. 11–12	**Webviews:** Keine Nutzung und keine Aktivierung von `webviewTag`.	Quellcodesuche und versionsbezogener Standardwert. Beide Empfehlungen sind wegen des fehlenden Mechanismus begründet nicht anwendbar.
Nr. 13–15	**Navigation, Fenster und Shell:** Dokumentpolicy begrenzt Navigation; neue Fenster werden abgelehnt; `shell.openExternal` wurde entfernt.	Codeinspektion, Policy-Tests und ausgewählte Paketversuche: fremde Navigation und `window.open`. Die Beobachtung gilt für diese Versuche.
Nr. 17, 20; A2, A4	**Beide IPC-Eingänge:** Feste Preload-Kanäle ohne Electron-Rohobjekte. Gemeinsame Policy prüft registriertes Fenster, Hauptframe und Dokumentadresse; Operationskennung und Eingabe werden im Main validiert.	Codeabgleich, sechs Sender-Unit-Tests und P1 an beiden realen Bridges. Kein zusätzlicher Fenster-/Frame-Harness; ein kompromittierter erlaubter Hauptframe bleibt erlaubter Sender.
Nr. 18	**Lokale Ressourcen:** `app://bundle/index.html` mit fester Rendererwurzel, GET/HEAD, Pfad- und Realpathprüfung sowie passenden Inhaltstypen; keine CSP-Umgehung.	Resolver- und Handler-Tests einschließlich eines Junction-Ausbruchs; Assets und betrachtete Abläufe im gebauten Paket. Kein allgemeiner Dateisystemaudit.
Nr. 16; A1	**Laufzeitversion:** Electron 44.5.1 exakt in Paketdefinition und Lockfile festgelegt. Native Datenbankmodule müssen weiterhin funktionieren.	Buildprotokoll, nativer Datenbankcheck und Paket-User-Agent. Versionsaktualität ist eine zeitpunktbezogene Aussage des dokumentierten Prüfstands.

Bezug	Prüfstelle und Umsetzung	Prüfverfahren und Nachweisgrenze
Nr. 19; A1	**Paketkonfiguration:** Vier unbenötigte Fuse-Optionen deaktiviert; Laden aus ASAR und eingebettete ASAR-Integritätsprüfung aktiviert.	Fuse-Werte aus der tatsächlichen EXE lesen, eingebetteten mit tatsächlichem ASAR-Headerhash vergleichen; Paket-, Datenbank- und Workerfunktion prüfen. Keine Signierungs- oder Distributionsbewertung.
Ergänzend; A5, A8	**Datei- und Datenübernahme:** CSV-/ICS-Auswahl im Main; Dateityp und Größe begrenzen. Eingaben und externe Katalogdaten vor fachlicher Übernahme validieren; fremde Textinhalte als Text darstellen.	Codeabgleich der Importpfade aus Phase 1, vorhandene Validierungs- und Katalogtests sowie P2 am Aufgabentitel. Keine zusätzliche umfassende Parser- oder XSS-Testfamilie.

Tabelle 5.8: Prüforte, Härtungsmaßnahmen und Nachweise im Demonstrator; Zuordnung nach [78] und [79], ergänzt um die lokalen Importpfade nach [80].

Senderprüfung, Eingabevalidierung und fachliche Berechtigung bilden unterschiedliche Kontrollen. Die Senderpolicy begrenzt, welches Fenster und welcher Frame die Schnittstelle verwenden dürfen. Die Schemaprüfung begrenzt die zulässige Form der Anfrage. Ob die betreffende Person auf die angeforderten Daten zugreifen darf, muss weiterhin durch die Sitzungs- und fachliche Zugriffslogik entschieden werden. Auch ein formal gültiger Aufruf eines erlaubten Renderers benötigt diese Kontrolle. Die Schichtentrennung macht die Verantwortlichkeiten und ihre Prüfung nachvollziehbar [80].

5.4.3 Umgesetzte Nachbesserungen

Der erste dokumentierte Richtlinienabgleich vom 3. Oktober 2026 identifizierte neben bereits vorhandenen Schutzmaßnahmen konkrete Abweichungen. Die Anwendung verwendete noch das `file://`-Protokoll und Electron 39.8.10. Das externe Öffnen war lediglich auf HTTPS begrenzt, und mehrere unbenötigte Fuse-Optionen waren in der untersuchten Binary aktiviert. Diese Befunde wurden in einen Nachbesserungsplan überführt und am 4. Oktober durch Änderungen sowie eine gezielte Nachprüfung bearbeitet. Die Matrix vom 6. Oktober konsolidiert diesen Verlauf [77], [78], [79].

Für lokale Ressourcen wurde das Protokoll `app:` eingerichtet. Sein Handler liefert ausschließlich Dateien innerhalb der gebauten Rendererwurzel aus. Neben der syntaktischen Pfadprüfung wird der tatsächlich aufgelöste Dateipfad geprüft, damit auch Verknüpfungen keinen Zugriff außerhalb dieser Wurzel eröffnen. Die Dokumentpolicy erlaubt im Paket ausschließlich das vorgesehene Dokument unter `app://bundle/index.html`; dessen Hash-Routen bleiben nutzbar. Navigation, Berechtigungsprüfung und IPC-Senderprüfung beziehen sich auf diese Dokumentpolicy. Assetadressen erhalten dadurch keine Freigabe als IPC-Dokument. Die bisherige Möglichkeit zum externen Öffnen wurde entfernt [80].

Beide IPC-Eingänge behielten ihre gemeinsame Senderprüfung. Diese wurde in `sender-policy.ts` ausgelagert und um die ausdrückliche Ablehnung eines fehlenden `senderFrame` ergänzt. Dadurch lässt sich die tatsächlich verwendete Policy unabhängig vom Dispatcher prüfen. Die Preload-Bridges exponieren weiterhin lediglich die beiden festen Invoke-Kanäle. Im Main werden bekannte Operationen und ihre Zod-Eingabeschemata vor der fachlichen Verarbeitung geprüft [79], [80].

Für externe Anbindungen wurde ein gemeinsamer HTTPS-Prüfer eingeführt. Er weist ungeeignete Protokolle und eingebettete Zugangsdaten zurück, ohne den Konfigurationswert in Fehlermeldungen auszugeben. Katalog und Supabase verwenden zusätzlich `redirect: 'error'`. Der Prüfer erhält die Schreibweise einer gültigen Adresse: Eine zwischenzeitliche Normalisierung ergänzte am PowerSync-Endpunkt einen Slash und verhinderte die Verbindung. Die Korrektur und ein Regressionstest sichern

dieses Detail ab. PowerSync verwendet weiterhin seinen eigenen Transport; dessen interne Weiterleitungen sind von dieser Absicherung nicht vollständig erfasst [79].

Electron wurde auf 44.5.1 aktualisiert. Damit mussten auch die nativen Datenbankabhängigkeiten und ihr Paketbetrieb geprüft werden; die erforderlichen Anpassungen betrafen `better-sqlite3`, das PowerSync-Node-SDK und den nativen Prüfschritt. Die Paketauswahl wurde auf gebaute Anwendungsteile und Produktionsabhängigkeiten begrenzt, während Drizzle-Migrationen weiterhin über `extraResources` ausgeliefert werden. Prüfdokumente und Umgebungsdateien gehören nicht zum Anwendungspaket [79].

Die Fuse-Konfiguration deaktiviert `RunAsNode`, die Auswertung von `NODE_OPTIONS`, Node-Inspektionsargumente und zusätzliche Dateiprotokollprivilegien. Aktiviert wurden `OnlyLoadAppFromAsar` und `EnableEmbeddedAsarIntegrityValidation`; Cookie- und Snapshot-Fuses blieben nach Bedarfsprüfung unverändert. Da diese Einstellungen die ausgelieferte Binary betreffen, genügt der Eintrag in `electron-builder.yml` nicht als Beleg. Die Werte wurden deshalb am gebauten Windows-Paket ausgelesen und die eingebetteten Integritätsmetadaten mit dem tatsächlichen ASAR-Header verglichen [79].

5.4.4 Nachweisverfahren und dokumentierte Grenzen

Die Nachweise wurden auf vier Ebenen erhoben. **Codeinspektion** ordnet Einstellungen und Kontrollen den tatsächlichen Einstiegspunkten zu. **Automatisierte Tests** prüfen ausgewählte Entscheidungen und Fehlerpfade; hierfür wurden vorhandene Dispatcher-, Service-, Upload- und Katalogtests sowie eine kleine Security-Suite verwendet. **Paketprüfungen** kontrollieren Buildabhängigkeiten, Fuses und Integritätsmetadaten an der erzeugten Anwendung. **Laufzeitbeobachtungen** zeigen die Wirkung ausgewählter Maßnahmen im gestarteten Paket. Diese Ebenen ergänzen sich, ersetzen einander jedoch nicht [79].

Die Security-Suite prüft unter anderem sechs Senderfälle: einen erlaubten Hauptframe, ein fremdes Fenster, einen Unterframe, einen fehlenden Frame, eine falsche Dokumentadresse und ein zerstörtes Fenster. Hinzu kommen Pfad- und Handlerfälle sowie negative HTTPS-Konfigurationen. Ein Redirecttest verwendet das tatsächliche Node-Fetch-Verhalten; der dafür injizierte lokale HTTP-Transport ist ausschließlich Testaufbau und begründet keine HTTP-Ausnahme im Anwendungsbetrieb. Die Tests der Senderpolicy verwenden nachgebildete Electron-Objekte und belegen die Policy-Entscheidungen, nicht einen Angriff aus einem zusätzlich gestarteten Fenster [79], [80].

Die drei praktischen Fälle P1–P3 wurden am korrigierten Windows-Paket über dessen bestehenden Renderer ausgeführt. **P1** übergibt je eine ungültige Eingabe an die API- und Kollaborationsbridge und prüft die Fehlerantworten `VALIDATION_ERROR` beziehungsweise `INVALID`. Die Reihenfolge der Schemaprüfung vor der fachlichen Verarbeitung wird ergänzend im Code nachvollzogen. Dieser Fall prüft die beiden ausgewählten Eingaben; er ist keine vollständige Operations- oder Berechtigungsmatrix [77], [79].

P2 speichert über die reguläre Aufgabenoberfläche einen synthetischen HTML-artigen Titel mit einem SVG-Element und einem harmlosen Ausführungsmarker. Die Darstellung wird unmittelbar und nach erneutem Öffnen der Aufgabenansicht untersucht. Erwartet werden ausschließlich Textknoten, kein aus dem Titel erzeugtes SVG-Element und kein gesetzter Marker. Das Protokoll ergänzt die DOM-Beobachtung um das Lesen der gespeicherten Aufgabe und den zu diesem Zeitpunkt verbundenen Sync-Zustand ohne ausstehende Uploads. Dieser Ablauf belegt weder Mehrgeräte-Synchronisierung noch allgemeine XSS-Freiheit oder einen dauerhaft störungsfreien Onlinebetrieb [79].

P3 versucht, eine von der CSP nicht zugelassene fremde Skriptquelle zu laden. Bewertet wird das konkrete `securitypolicyviolation`-Ereignis einschließlich der verletzten Skriptrichtlinie; ein DNS-Fehler allein wäre kein ausreichender Nachweis. Ergänzend wurden fremde Navigation und das Öffnen

eines zusätzlichen Fensters versucht. Die Ausführung von Prüfcode in den DevTools ist dabei keine nachgewiesene Schwachstelle: Sie dient zur gezielten Auslösung und Beobachtung der vorhandenen Kontrolle [77], [79].

Dateihashes binden die finalen Paket- und Laufzeitprotokolle an dieselbe EXE und dasselbe ASAR-Archiv. Frühere, während der Nachbesserung fehlgeschlagene Builds oder Prüfungen werden dadurch nicht nachträglich als erfolgreiche Nachweise des korrigierten Pakets ausgegeben. Die technischen Paket- und Laufzeitbelege stammen vom 4. Oktober 2026; der für diese Darstellung eingesehene HEAD `17434b2` enthält den zugehörigen Sicherheitscode unverändert. Die spätere Konsolidierung der Dokumentation ist kein neuer Testdurchlauf [78], [79].

Die wesentliche verbleibende Transportgrenze betrifft die SDK-interne Redirectbehandlung von PowerSync. Dort wurde die HTTPS-Startkonfiguration geprüft, aber kein praktischer Redirect- oder Downgradeversuch am Sync-Stream durchgeführt. Aus der begrenzten Bewertung der Matrix folgt daher kein umfassender Transportnachweis. Ebenso verbleiben die in Kapitel 4 ausgeschlossene Härtung entfernter Dienste und der Schutz der lokalen Datenbank gegenüber anderen Betriebssystemprozessen außerhalb der Untersuchung. Die Bewertungen und ihre Bedeutung für A7 werden in Kapitel 6 ausgewiesen; dieses Kapitel dokumentiert die Maßnahmen und den Weg zu den Belegen [78].

5.5 Handlungsempfehlungen des Migrationsleitfadens

Aus dem dokumentierten Prozess werden acht Handlungsempfehlungen M1–M8 abgeleitet. Sie beschreiben die erforderlichen Veränderungen zwischen Ausgangsartefakt und Zielarchitektur sowie Kontrollpunkte für deren Umsetzung. Die Reihenfolge folgt den technischen Abhängigkeiten: Lokale Datenhaltung und begrenzte Schnittstellen bilden die Grundlage vollständiger Offline-Abläufe; darauf bauen externe Datenübernahme und Synchronisierung auf. Die Sicherheitsanforderungen begleiten alle Schritte und werden am Ende zusammenhängend geprüft.

Die Empfehlungen beziehen sich auf Verantwortlichkeiten und Interaktionsformen. React, Vite, Drizzle, PowerSync und Supabase sind die konkreten Ausprägungen der Fallstudie. Ihre Auswahl ist bei einer anderen Anwendung anhand der vorhandenen Komponenten zu prüfen. Framework-agnostisch bedeutet hier, dass die Transformation über fachliche Operationen, Datenhaltung und Vertrauensgrenzen beschrieben wird und kein bestimmtes Web-Framework voraussetzt. Die beschriebenen Prozessgrenzen und Härtungsmaßnahmen beziehen sich auf die gewählte Electron-Zielumgebung.

5.5.1 Voraussetzungen und Herstellung der lokalen Datenhaltung

M1: Ausgangsartefakt und Zielumgebung abgleichen. Vor der Übernahme sind Oberfläche, Schnittstellen, Fachlogik, Datenhaltung und externe Abhängigkeiten zu erfassen. Für jede Kernfunktion ist festzulegen, ob sie vollständig lokal ausgeführt werden kann, externe Daten herunterlädt oder gemeinsame Änderungen synchronisiert. Vorhandener Code dokumentiert dabei zunächst eine Implementierung; seine tatsächliche Nutzbarkeit muss anhand eines vollständigen Ablaufs geprüft werden. Der Untersuchungsumfang sollte mindestens je ein Beispiel der benötigten Interaktionsformen enthalten. So entsteht eine begründete Auswahl, ohne jede CRUD-Variante separat migrieren zu müssen.

Auf dieser Grundlage ist eine kompatible Desktop-Laufzeit einzurichten. Vorhandene Sprachen, Bibliotheken und Buildverfahren sollen weiterverwendet werden, soweit sie mit der Zielarchitektur vereinbar sind. Die Wahl von Electron und electron-vite in Abschnitt 5.2 veranschaulicht dieses Vorgehen, belegt jedoch keinen allgemein geringsten Aufwand. Ein kleiner lauffähiger Paketstand sollte früh prüfen, ob die vorgesehenen Datenbanktreiber und nativen Abhängigkeiten auch außerhalb des Entwicklungsmodus funktionieren. Die Verantwortung für Dateisystem, Datenbank und Dienstzugriffe ist dem Main-Prozess zuzuordnen; der Renderer erhält dafür begrenzte fachliche Schnittstellen.

M2: Lokale Datenhaltung und Identitätszuordnung herstellen. Ein entferntes Datenbankschema ist anhand seiner Bedeutung zu überführen. Datentypen, Schlüssel, Beziehungen, Standardwerte und Prüfbedingungen müssen einzeln abgeglichen werden. Die SQLite-Übertragung in Phase 1 zeigt, dass eine erhaltene TypeScript-Typisierung keine entfallene Datenbankprüfung ersetzt. Erforderliche Regeln sind deshalb im Zielsystem ausdrücklich abzubilden und an den Eingangsstellen zu validieren [34].

Programmdateien und veränderliche Benutzerdaten sind getrennt zu verwalten. Die Anwendung soll ihre Datenbank an einem geeigneten Benutzerdatenpfad öffnen und das Schema vor der Nutzung vorbereiten. Schemaänderungen sind versioniert auszuliefern, soweit die Anwendung das Schema selbst verwaltet. Erforderliche Referenzdaten müssen auch beim ersten Offline-Start verfügbar sein. Die lokale Identität ist gesondert festzulegen: Anmeldung, Sitzungsdauer und Zuordnung persönlicher Datensätze müssen zusammenpassen. Eine frühere HTTP-Sitzung lässt sich nicht unverändert als lokale Sitzung behandeln. Im Demonstrator übernehmen hierfür die Datenbankinitialisierung und die getrennte lokale Sitzungsverwaltung die Verantwortung [34], [41].

5.5.2 Schnittstellen und Offline-Kernfunktionen überführen

M3: Fachlichen Vertrag und Transport trennen. Ausgangspunkt sollen die benötigten Operationen mit Eingaben, Ergebnissen und Fehlerfällen sein. Ein vorhandener API-Vertrag kann hierfür weiterverwendet werden, sofern seine Bedeutungen erhalten bleiben. HTTP-Pfade und Serverobjekte sind dagegen durch einen begrenzten lokalen Transport zu ersetzen. Der Renderer soll fachliche Operationen aufrufen und keine frei wählbaren IPC-Kanäle oder privilegierten Rohschnittstellen erhalten.

Die Vertragsdefinitionen sollen gemeinsam gepflegt werden. Wo bereits generierte Clients und Laufzeitschemata vorliegen, ist ihre Ableitung an den neuen Transport anzupassen. Abschnitt 5.2.5 zeigt hierfür den projektspezifischen OpenAPI-IPC-Generator. Änderungen am Vertrag müssen in den erzeugten Modulen nachvollziehbar ankommen; Darstellung und Fachlogik sind nur bei tatsächlich veränderter Semantik anzupassen. Statische Typen und Laufzeitvalidierung bleiben unterschiedliche Aufgaben. Bei mehreren Modulverträgen gelten dieselben Regeln für jeden Eingang; die Zahl der Datenbanken bestimmt nicht die Zahl der IPC-Schnittstellen [39], [64].

M4: Einen vollständigen Kernablauf herstellen und dessen Muster fortführen. Nach der Einrichtung von Datenhaltung und Transport ist zunächst ein zusammenhängender Verarbeitungspfad zu integrieren. Dabei sind Oberflächenkomponenten und ihre Abhängigkeiten mit den Build-, Import- und Routingbedingungen der Desktop-Umgebung abzugleichen. Der erfolgreiche Build allein zeigt noch nicht, dass eine Seite ihre fachlichen Operationen ausführen kann. Im Demonstrator erschließt die lokale Authentifizierung den ersten vollständigen Ablauf; die Notenverwaltung konkretisiert anschließend das CRUD-Muster aus Abschnitt 5.2.11 [40], [41].

Gemischte Webrouten sind nach Verantwortlichkeiten aufzuteilen: Handler übernehmen Operationszuordnung und Fehlerübersetzung, Services die fachlichen Regeln und Repositories den Datenzugriff. Der Dispatcher prüft Absender und Eingaben vor dem Handler. Persönliche Datenzuordnung und Berechtigungen bleiben Teil des Ablaufs und dürfen beim Entfernen des HTTP-Transports nicht entfallen. Für weitere Funktionen ist dieses Muster mit den jeweils passenden Vertragsdefinitionen und fachlichen Regeln fortzuführen. Datenzugriffe und externe Dienste sollen in Servicetests durch Testimplementierungen ersetzbar sein. Lokaler Schreibabschluss, erneutes Lesen und Fehlerbehandlung bilden den Kontrollpunkt eines übernommenen Kernablaufs [48].

M5: Dateiimporte als begrenzte native Operationen gestalten. Ein Webupload ist in Dateiauswahl, Lesen, Parsen, Validieren und Übernehmen aufzuteilen. Die Oberfläche soll den Import auslösen; der Main-Prozess übernimmt die native Auswahl und den Zugriff auf die ausgewählte Datei.

Eine frei verwendbare Schnittstelle zum Lesen beliebiger Renderer-Pfade ist dafür nicht erforderlich. Dateityp und Größe sind zu begrenzen, der Abbruch ist als eigener Zustand zu behandeln und Parserergebnisse sind vor der fachlichen Speicherung zu prüfen.

Das Ergebnis muss angeben, welche Daten übernommen, übersprungen oder abgewiesen wurden. Die Oberfläche aktualisiert danach die betroffenen Abfragen. Die Beispiele iCal und CSV in Abschnitt 5.2.12 zeigen, dass dieses Verantwortungsmuster unabhängig von der eingesetzten Parserbibliothek ist. Ein zusätzlicher Dateityp benötigt einen geeigneten Parser und fachliche Prüfungen; die grundlegende Trennung von Oberfläche, Dateizugriff und Verarbeitung bleibt erhalten [50], [52].

5.5.3 Externe Daten und Kollaboration schrittweise ergänzen

M6: Downloads mit dauerhafter lokaler Übernahme verbinden. Für externe Referenzdaten ist zunächst zu klären, ob eine Aktualisierung in einer Richtung ausreicht. Der Abruf soll hinter einer im Main-Prozess verwalteten Dienstanbindung liegen. Empfangene Daten sind vor der Speicherung zu validieren und zusammengehörige Änderungen konsistent zu übernehmen. Stabile Kennungen und ein festgelegtes Aktualisierungsverhalten müssen verhindern, dass wiederholte Abrufe unkontrolliert Duplikate oder Datenverluste erzeugen.

Die Oberfläche soll den vorhandenen lokalen Bestand weiterhin lesen können, auch wenn die zusätzliche Aktualisierung scheitert. Erst ein erfolgreich übernommener Bestand steht für spätere Offline-Aufrufe zur Verfügung; ein leerer lokaler Datenbestand lässt sich durch Fehlerbehandlung allein nicht ersetzen. Der Katalogabruf aus Abschnitt 5.3.10 veranschaulicht dieses Muster mit Schemaprüfung, Transaktion und erneutem Ladeversuch. Eine solche Datenübernahme ist von der Synchronisierung lokaler Änderungen zu unterscheiden [76].

M7: Synchronisierung an einem ausgewählten Funktionsbereich integrieren. Vor der breiten Überführung ist die vorgesehene Kombination aus lokaler Datenbank, Synchronisierung und entferntem Backend in einem begrenzten Ablauf zu prüfen. Die Vorprüfung soll Schemaabbildung, Identitäten, lokale Persistenz, ausstehende Änderungen und Paketbetrieb umfassen. Eine vorhandene ORM-Anbindung unterstützt Wiederverwendung, ersetzt jedoch nicht die Prüfung von Datentypen und dem Verhalten des neuen Datenzugriffs. Die Integrationsprototypen des Demonstrators bilden hierfür die Grundlage [62].

Anschließend sind die Verantwortung für lokale Änderungen, Upload und Download sowie unmittelbar netzwerkabhängige Operationen festzulegen. Clientseitig erzeugbare Kennungen und eine nachvollziehbare Zuordnung lokaler und entfernter Identitäten müssen Datensätze und Beziehungen konsistent halten. Der lokale Schreibabschluss ist von der Annahme durch das Backend zu unterscheiden; ausstehende, fehlgeschlagene und abgewiesene Änderungen benötigen ein festgelegtes Verhalten. Im betrachteten Gruppenfeature bleiben Aufgabenänderungen lokal ausführbar, während Gruppenerstellung und Einladungen unmittelbare Online-Operationen sind. Vorausgesetzte entfernte Konten und Änderungen des ursprünglichen Einladungsablaufs sind als Voraussetzungen beziehungsweise fachliche Anpassungen auszuweisen [63], [64].

Eine zeitweise parallele Datenhaltung kann die Überführung nach Funktionsbereichen begrenzen. Für jeden Bereich ist eindeutig festzulegen, welcher lokale Bestand und welche Repository-Implementierung verwendet werden. Die Übernahme bestehender Daten einschließlich ihrer Referenzen ist eine zusätzliche Aufgabe; das Einrichten einer neuen synchronisierten Tabelle führt sie nicht automatisch aus. Auch Downloadfilter ersetzen keine Schreibberechtigungen oder die ausdrückliche Festlegung rein lokaler Tabellen. Abschnitt 5.3.4 erläutert diese Trennung im Demonstrator.

Mit der Synchronisierung ist außerdem die Verantwortung für Schemaänderungen neu zu ordnen. Entferntes Schema, Datenverteilung und Client-Schema müssen kompatibel bereitgestellt werden.

Welche Änderungen weiterhin lokale SQL-Migrationen benötigen und welche das SDK verwaltet, hängt vom gewählten Datenhaltungsmodell ab. Abschnitt 5.3.8 beschreibt den daraus abgeleiteten Bereitstellungsablauf. Dieser ist bei einer Erweiterung mit bestehenden Clients und noch ausstehenden Änderungen abzugleichen; er wurde in der Fallstudie nicht als vollständiger Versionsrollout erprobt.

5.5.4 Kontrollpunkte, Härtung und Fortführung des Leitfadens

M8: Härtung konsolidieren und den Paketstand prüfen. Abschließend sind die tatsächlich entstandenen Fenster, Preload-Bridges, IPC-Eingänge, Ressourcen- und Netzwerkpfade gemeinsam zu prüfen. Jede Electron-Empfehlung erhält eine begründete Anwendbarkeitsentscheidung, Umsetzung und Beleg. Fehlende Mechanismen, fehlende Implementierungen und fehlende Prüfungen sind getrennt zu bewerten. Die Sicherheitsmatrix aus Abschnitt 5.4 zeigt das dazu verwendete Dokumentationsmuster [78].

Das Prüfverfahren ist an den Gegenstand anzupassen: Codeinspektion belegt die vorhandene Konfiguration, Unit-Tests prüfen ausgewählte Entscheidungen, und Laufzeitfälle beobachten die Wirkung konkreter Kontrollen. Buildabhängige Aussagen wie Fuse-Werte, ASAR-Integrität und native Modulkompatibilität sind am erzeugten Paket zu prüfen. Nach Änderungen an Version, Treibern, Protokoll oder Paketkonfiguration sind die betroffenen Nachweise zu erneuern und an den geprüften Stand zu binden. Die Nachbesserungen in Phase 3 begründen diese Empfehlung; insbesondere dürfen Ergebnisse einer früheren Binary nicht einem veränderten Paket zugeschrieben werden [79].

Tabelle 5.9 fasst die zu prüfenden Zustände zusammen. Ihre Zuordnung zu A1–A8 macht den Zweck der einzelnen Schritte sichtbar; die dokumentierte Verbindung von Voraussetzung, Transformation und Kontrollpunkt konkretisiert A9. Die Tabelle formuliert Prüfaufgaben für die Anwendung des Leitfadens. Welche Ergebnisse der Demonstrator hierfür tatsächlich liefert, wird in Kapitel 6 bewertet.

Schritt / Bezug	**Ergebnis des Schritts**	**Kontrollpunkt**
M1; A1, A3, A8	Erfasster Kernumfang und begründete, kompatible Zielumgebung.	Interaktionsformen und externe Abhängigkeiten benennen; kleinen Desktop-Paketstand einschließlich vorgesehener nativer Module starten.
M2; A5	Lokal verwalteter Datenbestand mit Schema, Referenzdaten und Identitätszuordnung.	Lokale Initialisierung und Zuordnung prüfen; Daten ohne Netzwerk schreiben und nach Neustart lesen. Erforderliche Änderungen an bestehenden Daten gesondert prüfen.
M3; A4, A7	Fachlicher Vertrag mit begrenztem Preload-/IPC-Transport.	Vertrag und erzeugte Module abgleichen; unbekannte Operationen und ungültige Eingaben abweisen. Prüfen, dass lokale Aufrufe keinen HTTP-Server benötigen.
M4; A2, A5, A8	Vollständiger Kernablauf mit getrennten Verantwortlichkeiten.	Aufruf von Oberfläche bis Repository nachvollziehen; persönliche Datenzuordnung, CRUD-Verhalten und Fehler prüfen. Fachliche Tests von nativer IPC- und Oberflächenprüfung unterscheiden.
M5; A7, A8	Native Dateiauswahl mit begrenztem Zugriff und validierter Verarbeitung.	Abbruch, ungeeignete Eingaben und gültigen Import prüfen; fachliches Ergebnis und Aktualisierung der Oberfläche nachvollziehen.
M6; A5, A8	Validierte Aktualisierung mit dauerhaft lokalem Bestand.	Wiederholten Abruf und fehlerhafte Übernahme prüfen; verfügbaren Bestand bei Netzausfall und nach Neustart weiterverwenden.
M7; A6, A8	Abgegrenzter synchronisierter Bereich mit geregelter Identität und Dienstanbindung.	Lokalen Schreibabschluss, Persistenz ausstehender Änderungen und Übermittlung getrennt prüfen; Online-Voraussetzungen und Verhalten bei Fehlern ausweisen.
M8; A1, A7	Konsolidierte Sicherheitsmatrix mit versionsbezogenen Belegen.	Alle Electron-Empfehlungen zuordnen; ausgewählte Eingabe- und Fremdinhaltfälle prüfen; Paketwerte und Funktion belegen. Verbleibende Nachweisgrenzen dokumentieren.

Tabelle 5.9: Handlungsempfehlungen und Kontrollpunkte, abgeleitet aus dem dokumentierten Migrationsprozess; die Zuordnung konkretisiert A9.

Für eine weitere Funktion ist zunächst ihre Interaktionsform zu bestimmen. Ein zusätzlicher CRUD-Bereich kann das Vertrags-, Handler-, Service- und Repository-Muster fortführen; ein weiterer Import ergänzt Dateiformat und fachliche Validierung; eine weitere Online-Anbindung benötigt einen geregelten Datenübernahme- oder Synchronisierungsablauf. Neue privilegierte Fähigkeiten, Identitätswechsel oder Datenflüsse erfordern einen erneuten Architektur- und Sicherheitsabgleich. Der Leitfaden bietet damit ein begründetes Vorgehensmodell für die untersuchten Aufgaben. Seine Übertragung auf andere Ausgangsartefakte bleibt anhand dieser Voraussetzungen zu prüfen; eine unabhängige Anwendung auf weitere Projekte wurde nicht durchgeführt.

Kapitel 6

Evaluation und Demonstration

Dieses Kapitel bewertet den Demonstrator und den daraus abgeleiteten Leitfaden anhand von A1–A9 aus Abschnitt 4.4. Im Mittelpunkt steht, welche Veränderungen und Zielzustände durch die vorliegenden Belege nachvollziehbar sind. Die Beschreibung einer Implementierung, ein erfolgreicher Test und ein beobachteter Ablauf liefern unterschiedliche Aussagen. Die Bewertung verbindet diese Nachweise mit ihrem jeweiligen Prüfstand und weist verbleibende Lücken aus.

6.1 Prüfgegenstand und Belegbasis

Gegenstand der Quellcodebewertung ist der StudyPlan-Stand `17434b2`, eingesehen am 10. Oktober 2026. Dieser Stand ist im öffentlichen GitHub-Repository `lysmz/studyplan` zugänglich. Für Paket- und Laufzeitaussagen werden die dokumentierten Windows-Prüfungen vom 4. Oktober herangezogen. Sie beziehen sich auf die korrigierte Anwendung nach der Härtung mit Electron 44.5.1 unter Windows x64. Der native Prüfschritt bestätigt SQLite-Schreiben und -Lesen sowie den libSQL-Zugriff in dieser Electron-Laufzeit. Die Paketidentität ist durch EXE- und ASAR-Hashes festgehalten; die betreffenden Sicherheitsquellen sind gegenüber dem zugehörigen Commit `fc25822` unverändert [79], [81].

Die Belegbasis umfasst Code- und Architekturinspektionen, automatisierte Prüfungen sowie ausgewählte manuelle Paketabläufe. Für die Synchronisierung liegen zusätzlich Berichte der Integrationsprototypen und der Upload-Nachbesserung vom 26. September vor. Diese dokumentieren damalige lokale und entfernte Prüfszenarien; sie sind keine erneute Prüfung der heutigen Dienstkonfiguration. Ein unabhängiger Migrationsdurchlauf, Frontend-Unit-Tests und automatisierte End-to-End-Tests gehören gemäß Kapitel 3 und 4 nicht zum Evaluationsumfang [62], [75].

Am 10. Oktober wurden ergänzend die fünf vorhandenen lokalen Service-Suites erneut ausgeführt. Das zugehörige Ausführungsprotokoll wird als separates lokales Artefakt außerhalb des öffentlichen Repositorys geführt. Die Suites verwenden Node.js 24.18.1 unter Windows x64 und SQLite-Datenbanken im Arbeitsspeicher. Ihre Testschemata werden innerhalb der Fixtures erzeugt. Der Aufruf des Dispatchers ersetzt native IPC; die Dateiimporte verwenden nachgebildete Dateiauswahlergebnisse. Damit prüfen diese Suites Fachlogik, Datenzuordnung und ausgewählte Schnittstellenfehler, jedoch weder die Oberfläche noch native Dateidialoge, produktive Datenbankmigrationen oder Neustartpersistenz. Tabelle 6.1 zeigt die Ergebnisse [82].

Suite	Bestanden	Geprüfte Ausschnitte
`auth:test`	8/8	Passwort-Hashing, Anmeldung und Abmeldung, fehlende Sitzungen, Benutzerzuordnung und strukturierte Fehler.
`dashboard:test`	4/4	Leere Datenbestände, persönliche Statistik, Fristen und benutzerbezogene Aktivitäten.
`events:test`	9/9	Persönliche Terminoperationen, Filter, ICS-Parsing und -Import, Abbruch sowie ungültige Inhalte.
`grades:test`	8/8	Numerische Notenwerte, ECTS-gewichtete Statistik, Datenzuordnung, CSV-Parsing, Import und Abbruch.
`timetable:test`	6/6	Eigene Einträge, vorbereitete Pläne, Entfernen und Leeren, ungültige Werte und fremde Datensätze.
Gesamt	**35/35**	Keine fehlgeschlagenen oder übersprungenen Tests in diesen fünf Suites.

Tabelle 6.1: Erneute Ausführung vorhandener lokaler Service-Tests am 10. Oktober 2026; Protokoll und Prüfstand nach [82].

Davon getrennt dokumentiert der Nachbesserungsbericht vom 4. Oktober 53 erfolgreiche Prüfungen: Security-Suite (13), IPC-/Generator-Suite (10), lokale Gruppenressourcen (5), Katalogsuite (8) und Kollaborations-Service-/Uploadtests (17). Im Windows-Paket bestanden außerdem drei isolierte PowerSync-Prüfungen zu Neustartpersistenz, Client-Schemaerweiterung und Repository-CRUD. Die unterschiedlichen Zahlen beschreiben jeweils ausgewählte Testmengen und keine prozentuale Abdeckung der Anwendung [79].

6.2 Architektur und Wiederverwendung

Die Zuordnung der Module bestätigt die in A2 geforderte Trennung von Darstellung, fachlicher Verarbeitung und Datenzugriff. Die Oberfläche ruft fachliche Operationen über Preload und IPC auf. Im Main-Prozess vermitteln Handler zwischen Dispatcher und Services; Repositories führen die Datenbankabfragen aus. Das Notenbeispiel aus Abschnitt 5.2.11 lässt sich durchgehend von `useCreateGrade` bis `GradesRepository` nachvollziehen. Die erneut ausgeführten Tests verbinden Client, Dispatcher, Handler, Service und Repository mit einer Testdatenbank. Sie prüfen damit einen vollständigen fachlichen Verarbeitungspfad ohne React-Oberfläche [81], [82].

Electron-Prozesse und fachliche Schichten bleiben unterschiedliche Strukturmerkmale. Fachlogik und Datenzugriff liegen beide im Main-Prozess; ihre Verantwortlichkeiten werden durch die Module und deren Abhängigkeiten getrennt. Testimplementierungen für Dateiauswahl und externe Transporte zeigen, dass diese Abhängigkeiten an den betrachteten Stellen austauschbar sind. Daraus folgt eine strukturelle Grundlage für weitere Tests, jedoch keine bereits vollständige Testabdeckung sämtlicher Services [81].

Für A4 ist der lokale Aufrufweg maßgeblich: Die Kernoperationen werden über die beiden begrenzten IPC-Bridges registriert und ausgeführt. Im betrachteten Main-Startablauf wird hierfür kein Express-Server gestartet. Der Vite-Entwicklungsserver dient der Bereitstellung des Renderers im Entwicklungsmodus; er ist kein HTTP-Wrapper der lokalen Fachlogik. Im Paket werden die Renderer-Ressourcen über `app://bundle/index.html` geladen. Externe HTTPS-Anfragen bleiben auf die dafür vorgesehenen Anbindungen beschränkt [80], [81].

Die Wiederverwendung gemäß A3 ist qualitativ nachvollziehbar. React-Seiten, Oberflächenkomponenten, Gestaltung und wesentliche fachliche Regeln bilden weiterhin die Grundlage der Anwendung.

Der OpenAPI-Vertrag und Drizzle bleiben erhalten, während Transport, Laufzeitschemata, Datenbankdialekt und Treiber angepasst wurden. Die ursprünglichen HTTP-Routen wurden auf Handler, Services und Repositories verteilt. Die Synchronisierung ergänzt einen eigenen Datenhaltungsbereich und neue Dienstanbindungen. Kapitel 5 benennt diese Eingriffe und ihre Gründe. Gemessene Entwicklungszeiten, ein Codeanteil in Prozent oder ein vergleichender Nachweis minimalen Aufwands liegen nicht vor und werden für A3 auch nicht vorausgesetzt.

Im Hinblick auf den Vorversuch aus Abschnitt 3.3 sind die Konsistenz der Schnittstellenbausteine und der Erhalt des fachlichen Verhaltens getrennt zu bewerten. Die gemeinsame Vertragsableitung macht Typen, Aufrufhilfen und Laufzeitschemata auf dieselbe Definition zurückführbar. Die fachlichen Tests prüfen demgegenüber die Ergebnisse ausgewählter Verarbeitungspfade. Weder die Vertragsbindung noch ein erfolgreicher Build belegen für sich die Übereinstimmung aller Anwendungsabläufe mit dem Ausgangsartefakt. Eine geringere Häufigkeit unbeabsichtigter Verhaltensänderungen gegenüber dem Vorversuch wurde nicht vergleichend gemessen.

6.3 Kernfunktionen und lokale Nutzbarkeit

Die Paketprotokolle belegen den Start der gebauten Windows-Anwendung, die Darstellung der Anmeldung und der lokalen Assets sowie die Nutzung der Gruppenroute mit Aufgaben. Nach der Aktualisierung auf Electron 44.5.1 funktionieren die geprüften nativen Datenbank- und Workerpfade. Damit ist A1 für diesen Paketstand und die dokumentierten Abläufe gestützt. Eine entsprechende Aussage über macOS, Linux oder eine signierte Installation lässt sich daraus nicht ableiten [79].

Die persönlichen Funktionen wurden mit ihrem fachlichen Kontext übernommen. Tabelle 6.2 ordnet die Ausgangsfunktionen den Veränderungen und Nachweisen zu. Die Gegenüberstellung bewertet insbesondere die in A8 geforderten Interaktionsformen. Sie behauptet keine separat geprüfte Übereinstimmung jedes visuellen Details der ursprünglichen Oberfläche.

Bereich	Ausgangsfunktion	Migrationsergebnis und Beleg
Dashboard	Persönliche Kennzahlen, Fristen und Aktivitäten über Webrouten.	Lokale Services und benutzerbezogene SQLite-Abfragen; vier erfolgreiche Tests für ausgewählte Kennzahlen und Auswahlbedingungen.
Kalender	Persönliche Termine, Ansichten und ICS-Webupload.	Lokale Terminoperationen und nativer Importaufruf; neun erfolgreiche Tests für CRUD, Filter und ausgewählte ICS-Fälle. Kalenderdarstellung und Farbcodierung wurden nicht als vollständige GUI-Testmatrix geprüft.
Noten	Semesterbezogene Noten, Statistik und CSV-Webupload.	Service-/Repository-Pfad mit numerischen Werten sowie lokaler CSV-Verarbeitung; acht erfolgreiche Tests. Die fachliche Statistik bleibt erhalten.
Stundenplan	Persönliche Einträge und vorbereitete Hochschulpläne.	Lokale Auswahl und Verwaltung mit Sitzungszuordnung; sechs erfolgreiche Tests. Rasterdarstellung und visuelle Unterscheidungen sind übernommene Oberflächenbestandteile.
Identität	Webanmeldung mit Session-Cookies.	Lokale Sitzung im Main und getrennte Anmeldung für bestehende entfernte Konten; acht lokale Auth-Tests und dokumentierte Paket-Anmeldung. Keine Registrierung entfernter Konten in der App.
Studiengruppen	Gruppenerstellung, Einladungslink und gemeinsame Aufgaben.	Neue personenbezogene Einladung und lokaler Aufgabenbestand mit Synchronisierung. Gruppenverwaltung benötigt eine Verbindung; Kommentare und weitere gemeinsame Inhalte sind nicht Untersuchungsgegenstand.
Katalog	Bereitgestellter Referenzdatenbestand.	Ergänzender Download mit Schemaprüfung und transaktionaler lokaler Übernahme. Acht Katalogtests im Nachbesserungsbericht; keine Rückübertragung lokaler Änderungen.

Tabelle 6.2: Funktionale Gegenüberstellung für den Kernumfang nach [28], [81] und den Prüfprotokollen [82], [79] und [75].

Für A5 ergibt sich aus der Codeinspektion ein lokaler Verarbeitungspfad der persönlichen Funktionen. Die Datenbank liegt im Benutzerdatenverzeichnis; Schema und anfängliche Referenzdaten werden vor der Nutzung eingerichtet. Die lokalen Service-Tests bestätigen die betrachteten Operationen ohne entfernten Transport. Weil sie im Arbeitsspeicher ausgeführt werden, belegen sie deren Ergebnisse jedoch nicht nach einem Anwendungsneustart. Die paketierten PowerSync-Persistenztests betreffen einen separaten Prototyp und dürfen nicht als Neustartnachweis aller persönlichen Tabellen gewertet werden [79], [81], [82].

Die Offline-Kernarchitektur ist damit nachvollziehbar und ausgewählte lokale Abläufe sind geprüft. Ein vollständiger Paketdurchlauf sämtlicher persönlicher Kernfunktionen bei unterbrochener Verbindung einschließlich Neustart und erneutem Lesen liegt in der ausgewerteten Belegbasis nicht vor. A5 wird deshalb nur teilweise als belegt bewertet. Dies bezeichnet eine Nachweislücke; aus ihr folgt weder eine nachgewiesene Fehlfunktion noch die pauschale Erfüllung der Anforderung.

Auch die Dateiimporte besitzen unterschiedliche Nachweisebenen. ICS- und CSV-Parser sowie ihre fachliche Übernahme werden in den lokalen Tests ausgeführt. Die native Dialoganbindung, Dateityp- und Größenbegrenzung sind durch Codeinspektion nachvollziehbar. Die Test-Dateiauswahl ersetzt den echten Dialog. Es wird daher kein vollständig beobachteter nativer Importablauf für jede Variante behauptet. Der Katalogtest ergänzt die Datenübernahme um einen injizierten Transport mit Import-,

Aktualisierungs-, Fehler- und Rollbackfällen; er ist kein neuer Live-Abruf des entfernten Dienstes [79], [82].

6.4 Synchronisierung und Fehlerverhalten

Die Integrationsprototypen belegen die technische Verbindung von PowerSync, lokalem SQLite und Drizzle. Im Terminprototyp blieben ein Datensatz und vier ausstehende Änderungen nach dem Schließen und erneuten Öffnen der Datenbank erhalten. Der dokumentierte Replikatabgleich übertrug denselben Datensatz an eine weitere lokale Replik. Die geprüfte Schemaerweiterung ergänzt eine Client-View; sie ist kein vollständiger koordinierter Versionsrollout von Backend, Datenverteilung und Anwendung. Die Übernahme bestehender persönlicher Termine einschließlich ihrer Referenzen wurde ebenfalls nicht demonstriert [62].

Für den Gruppenbereich dokumentiert die Upload-Nachprüfung vom 26. September neun erfolgreiche Prüfungen mit Electron, PowerSync und SQLite sowie zwölf erfolgreiche Cloud-Prüfpunkte. Zu den Cloud-Fällen gehören Benutzersuche und Einladungszustellung, Annahme mit synchronisierter Mitgliedschaft, Offline-Aufgabenänderung und anschließender Abgleich zwischen Mitgliedern. Die zwölf Prüfpunkte umfassen auch die Bereinigung der erzeugten Testdaten. Diese Ergebnisse beziehen sich auf getrennte Testreplikate und den damaligen Dienststand, nicht auf einen vollständig geprüften Kontolebenszyklus oder eine produktive Mehrgeräteanwendung [75].

Die Nachprüfung enthält auch negative Ergebnisse des vorherigen Verhaltens. Fünf Regressionen zeigten, dass eine ohne Wirkung gebliebene Änderung aus der Warteschlange verschwinden konnte, ein abgewiesener Auftrag unabhängige Änderungen blockierte und wiederholte Uploads neuere Bearbeitungen überschrieben. Der erfolgreiche lokale Standardablauf hatte diese Fehler nicht erkannt. Nach der Änderung werden bekannte Ablehnungen vor dem Bestätigen der Warteschlangentransaktion lokal gesichert. Wiederholte Operationen erhalten anhand ihrer Identität den bereits erfassten Ausgang. Die Protokolle belegen die ausgewählten Fälle einschließlich verlorener Antworten, Prozessabbruch und konkurrierender doppelter Anfragen [75].

Der aktuelle Connector verwendet hierfür `upload_group_task` und ein lokales Ablehnungsjournal. Die erneute Anwendung derselben Operation ist von einer neuen konkurrierenden Bearbeitung zu unterscheiden: Die protokollierten Fälle belegen Schutz vor den untersuchten Wiederholungsfehlern, aber keine allgemeine Zusammenführung konkurrierender Änderungen. Auch eine lokale Transaktion mit mehreren Aufgabenoperationen wird dadurch nicht zu einer einzigen atomaren entfernten Transaktion. Diese Grenzen konkretisieren das in Kapitel 5 beschriebene Fehlerverhalten der Synchronisierung [75], [81].

Im korrigierten Windows-Paket wurde ergänzend eine synthetische Aufgabe über die Oberfläche gespeichert, erneut geöffnet und aus dem lokalen Bestand gelesen. Zum dokumentierten Zeitpunkt meldete die Synchronisierung `connected: true` und `pendingUploads: 0`. Der spätere Weiterbetrieb erzeugte jedoch Streamabbrüche im Startprotokoll. Der erfolgreiche Einzelablauf ist deshalb kein Beleg eines dauerhaft störungsfreien Onlinebetriebs. Zusammen mit den Prototypen stützt er A6 für die ausgewählten Aufgaben-, Gruppen- und Einladungsabläufe. Unmittelbare Online-Voraussetzungen und der lokale Abschluss bleiben dabei getrennt [79].

6.5 Ergebnis der Sicherheitsprüfung

Die konsolidierte Matrix bewertet alle 20 Electron-Empfehlungen. Sie weist 17 auf der jeweils dokumentierten Nachweisebene als erfüllt aus. Zwei Empfehlungen zu Webviews sind wegen deren fehlender Nutzung begründet nicht anwendbar. Punkt 1 wird innerhalb des ausdrücklich begrenzten Prüfumfangs als erfüllt bewertet: HTTPS-Konfiguration und Redirect-Abweisung sind für Katalog

und Supabase belegt, die SDK-interne PowerSync-Redirectbehandlung ist dagegen nicht praktisch nachgewiesen. Diese Grenze bleibt Teil der Bewertung von A7 [78].

Für Renderer-Isolation, Sandbox, Berechtigungen und bestimmte Standardwerte stützt sich die Bewertung überwiegend auf Code- und versionsbezogene Konfigurationsprüfung. Die Senderpolicy wird dagegen zusätzlich durch sechs Unit-Fälle geprüft. Fuses und ASAR-Integrität wurden an der tatsächlichen EXE und am Archiv kontrolliert. Keine dieser Nachweisebenen darf als umfassender Laufzeitangriff gegen sämtliche Empfehlungen ausgegeben werden. Abschnitt 5.4 dokumentiert die Zuordnung und Umsetzung; Tabelle 6.3 fasst die tatsächlich beobachteten Fälle zusammen [79].

Fall	**Beobachtetes Ergebnis**	**Reichweite**
P1	Ungültige Eingabe an beiden realen Bridges zurückgewiesen: API mit `VALIDATION_ERROR`, Kollaboration mit `INVALID`.	Zwei ausgewählte Eingaben aus dem erlaubten Renderer; keine vollständige Operations- oder Rollenmatrix und kein Angriff aus einem fremden Fenster.
P2	Gespeicherter HTML-artiger Aufgabentitel bleibt auch nach Wiederöffnung Text: ein passender Absatz, Textknoten, kein SVG-Element und kein Ausführungsmarker.	Darstellung und erneutes Lesen dieser Aufgabe im Paket; keine allgemeine XSS-Freiheit oder Mehrgeräteprüfung.
P3	Fremde Skriptquelle durch die CSP blockiert; beobachtetes Ereignis mit `script-src-elem`, kein gesetzter Ausführungsmarker.	Konkreter Ressourcenladefall; die DevTools-Ausführung des Prüfaufbaus ist selbst kein XSS-Nachweis.
Navigation / Fenster	Fremde Navigation ändert die App-Adresse nicht; `window.open` liefert `null`, kein zusätzliches App-Fenster oder beobachtetes Browseröffnen.	Ausgewählte Paketversuche ergänzen die Codeprüfung der Dokument- und Fensterpolicy.

Tabelle 6.3: Beobachtete negative Laufzeitfälle am korrigierten Windows-Paket vom 4. Oktober 2026 nach [79].

Der Nachbesserungsverlauf zeigt außerdem die Bedeutung der Prüfung am erzeugten Artefakt. Die erste URL-Normalisierung verhinderte die PowerSync-Verbindung; ein Resolverproblem beeinträchtigte die Formularvalidierung. Beide Befunde wurden korrigiert und die finalen Paketnachweise separat gebunden. Frühere erfolgreiche Teilprüfungen des fehlerhaften Zwischenstands wurden nicht als Funktionsnachweise der korrigierten Binary übernommen. Die Sicherheitsbewertung bezieht sich daher auf den ausgewiesenen finalen Stand [79].

A7 ist damit im festgelegten Richtlinien- und Nachweisrahmen erfüllt. Die Bewertung umfasst weder die Härtung der entfernten Infrastruktur noch Datenbankverschlüsselung gegenüber anderen Betriebssystemprozessen oder eine Distributionsfreigabe. Sie enthält insbesondere keinen zusätzlichen PowerSync-Redirectnachweis. Aus dem dokumentierten Ergebnis folgt keine Aussage über die Abwesenheit sämtlicher Schwachstellen [78].

6.6 Bewertung der Anforderungen und des Leitfadens

Die Ergebnisse werden in Tabelle 6.4 auf den Anforderungskatalog zurückgeführt. **Belegt** bezeichnet dort einen durch die genannten Verfahren gestützten Befund innerhalb des ausgewiesenen Umfangs. **Teilweise belegt** weist aus, dass für einen erforderlichen Zielzustand nur Teilnachweise vorliegen. Die Anforderungsbewertung wird somit von einer pauschalen Aussage über die gesamte Anwendung getrennt.

Anf.	Bewertung	Beleg und Einordnung
A1	Für den geprüften Windows-Paketstand belegt.	Start, Assets, Anmeldung, Gruppenroute und Aufgabenablauf dokumentiert; native Datenbank- und Workerchecks erfolgreich. Keine entsprechende Plattform- oder Installerbewertung.
A2	Architektonisch und exemplarisch belegt.	Verantwortlichkeiten der Module und vollständiger Notenpfad nachvollzogen; lokale fachliche Tests ohne Oberfläche. Erweiterte Testabdeckung ist davon zu unterscheiden.
A3	Qualitativ belegt.	Wiederverwendete Komponenten und erforderliche Änderungen dokumentiert. Kein Vergleich des Entwicklungsaufwands mit alternativen Werkzeugen.
A4	Belegt.	Lokale Fachoperationen über Preload und IPC statt eines lokalen HTTP-Backends; externe HTTPS-Anbindungen bleiben gesondert.
A5	Teilweise belegt.	Lokale Datenhaltung und geprüfte persönliche Operationen vorhanden. Vollständiger Offline-Paketdurchlauf mit Neustartnachweis aller persönlichen Kernfunktionen nicht dokumentiert.
A6	Für die untersuchten Kollaborationsfälle belegt.	Lokale Persistenz, Warteschlange, Einladung, Mitgliedschaft und Abgleich in ausgewählten Prototypfällen; Aufgabenablauf im finalen Paket. Kein dauerhaft störungsfreier Betrieb nachgewiesen.
A7	Im festgelegten Prüfumfang erfüllt.	Alle 20 Empfehlungen eingeordnet; Code-, Unit-, Paket- und Laufzeitbelege ausgewiesen. PowerSync-Redirectbehandlung bleibt dokumentierte Nachweisgrenze.
A8	Für die repräsentativen Verarbeitungspfade belegt.	CRUD, Dateiimporte, Download und Kollaboration mit wiederkehrenden Architekturmustern abgedeckt. Oberflächeneinzelheiten nicht vollständig systematisch geprüft.
A9	Durch Prozess- und Strukturabgleich belegt.	Voraussetzungen, Phasen, Änderungen und Kontrollpunkte in Kapitel 5 dokumentiert; M1–M8 zugeordnet. Keine unabhängige Anwendung des Leitfadens auf ein weiteres Artefakt.

Tabelle 6.4: Anforderungsbezogene Bewertung auf Grundlage der in diesem Kapitel ausgewiesenen Belege und Nachweisgrenzen.

Der Leitfaden aus Abschnitt 5.5 lässt sich dem Demonstrator zuordnen: M1–M2 betreffen Laufzeit, Datenhaltung und Identität; M3–M5 die Vertrags-, CRUD- und Importpfade; M6–M7 die externe Datenübernahme und Kollaboration; M8 die konsolidierte Härtung. Die Beispiele zeigen wiederkehrende Verantwortlichkeiten bei unterschiedlichen Interaktionsformen. Dadurch ist die Ableitung der Handlungsempfehlungen aus dem Fall nachvollziehbar. Dass eine weitere Person damit ein anderes Artefakt ohne wesentliche Lücken migriert, wurde nicht unabhängig geprüft.

Die nachgewiesene Wiederverwendung und die geprüften Verarbeitungspfade stützen den fallbezogenen Ansatz. Die wichtigste offene funktionale Nachweisaufgabe ist der vollständige Offline- und Neustartdurchlauf für A5. Darüber hinaus bleiben die genannten Transport- und Laufzeitgrenzen bestehen. Ein früheres Upload-Prüfprotokoll enthält auch einen fehlgeschlagenen Gesamtlint mit überwiegend bestehenden Formatierungsbefunden; diese Prüfung wurde hier nicht erneut ausgeführt. Erfolgreiche fachliche Tests und Paketabläufe sind daher kein pauschaler Nachweis technischer Schuldenfreiheit. Die Bedeutung dieser Ergebnisse für Übertragbarkeit und weitere Anwendung wird in Kapitel 7 diskutiert [75].

Kapitel 7

Diskussion

Die Evaluation in Kapitel 6 zeigt, welche Eigenschaften des Demonstrators durch die vorhandenen Belege gestützt sind. Dieses Kapitel ordnet ihre Bedeutung für die Forschungsfrage und den Migrationsleitfaden ein. Im Mittelpunkt stehen die Verlagerung von Verantwortlichkeiten, die Abwägung zwischen Wiederverwendung und Anpassung sowie die Grenzen der untersuchten Architektur.

7.1 Beitrag zur Forschungsfrage

Die Forschungsfrage aus Kapitel 1 zielt auf eine systematische Überführung KI-generierter Webarchitekturen in sichere Local-First-Desktop-Anwendungen. Der untersuchte Fall liefert dafür ein Vorgehen, das die vorhandenen Komponenten nach ihren Aufgaben einordnet und ihre Zuständigkeiten in der Zielumgebung neu festlegt. Die Oberfläche bleibt weitgehend erhalten, während Datenhaltung, Schnittstellenverarbeitung und Dienstanbindungen angepasst werden. Die Migration lässt sich damit als Transformation von Verantwortlichkeiten beschreiben: Aufgaben, die zuvor das entfernte Backend übernahm, werden teilweise in die lokale Anwendung verlagert; gemeinsame Daten benötigen weiterhin einen geregelten entfernten Abgleich.

Die Phasenstruktur macht diese Transformation nachvollziehbar. Eine lokale Kernarchitektur schafft zunächst die Grundlage für Datenverarbeitung und Persistenz. Darauf aufbauend werden externe Datenübernahme und Kollaboration ergänzt. Die konsolidierte Härtung prüft die tatsächlich entstandenen Prozess- und Schnittstellengrenzen. Die Schritte besitzen somit fachliche Abhängigkeiten, auch wenn Sicherheitsentscheidungen bereits während der gesamten Entwicklung erforderlich sind. Die Beispiele für CRUD, Dateiimporte und Netzwerkanbindungen zeigen, wie dieses Vorgehen auf unterschiedliche Interaktionsformen angewendet wurde.

Der Beitrag zur Forschungsfrage besteht in einem begründeten, am Demonstrator nachvollziehbaren Migrationsmodell. Die in Abschnitt 5.5 formulierten Handlungsempfehlungen verbinden Voraussetzungen, Änderungen und Kontrollpunkte. Im Rahmen des gestaltungsorientierten Vorgehens werden Entwicklung, Demonstration und Evaluation eines solchen Artefakts miteinander verbunden [25]. Die Aussagekraft dieses Ergebnisses hängt an den ausgewiesenen Nachweisen; die Fallstudie begründet keine allgemeine Gültigkeit für beliebige Webarchitekturen.

7.2 Wiederverwendung und architektonische Anpassung

Das Ziel eines begrenzten Refactoring-Aufwands erwies sich als sinnvolle Orientierung für die Auswahl und Übernahme der Komponenten. Die Kombination aus React, TypeScript und Vite sowie die Node.js-Laufzeit des Electron-Main-Prozesses passten zu wesentlichen Bestandteilen des Ausgangsartefakts. Abschnitt 5.2 und Abschnitt 5.2.8 zeigen die dadurch mögliche Weiterverwendung. Diese technische Anschlussfähigkeit erklärt die Werkzeugwahl im konkreten Projekt. Eine Überlegenheit von

Electron oder electron-vite gegenüber den betrachteten Alternativen hinsichtlich Aufwand, Wartbarkeit oder Sicherheit wurde jedoch nicht vergleichend untersucht.

Wiederverwendung betrifft dabei unterschiedliche Ebenen. Oberflächenkomponenten und fachliche Regeln konnten als Grundlage erhalten bleiben. PostgreSQL-spezifische Schemadefinitionen, HTTP-Sitzungen und Route-Handler mussten dagegen an die lokale Laufzeit und ihre Verantwortlichkeiten angepasst werden. Abschnitt 6.2 stützt deshalb eine qualitative Aussage über erhaltene Bestandteile und nachvollziehbare Eingriffe. Für A3 ist diese Unterscheidung entscheidend: Die Begrenzung des Änderungsumfangs bleibt den Architektur- und Sicherheitsanforderungen untergeordnet. Die Eignung einer übernommenen Komponente richtet sich weiterhin nach den Anforderungen der Zielumgebung.

Der Vorversuch aus Abschnitt 3.3 verdeutlicht eine weitere Bedeutung der Wiederverwendung: Werden während der Übertragung zugleich Formulare, Datendarstellungen und Schnittstellen verändert, wird die Zuordnung einzelner Eingriffe schwieriger. A3 dient deshalb auch der Kontrolle des Änderungsumfangs. Vorhandenes Verhalten bildet einen Bezugspunkt, gegenüber dem notwendige Veränderungen begründet werden können. Für den Leitfaden folgt daraus, Anpassungen an Transport und Persistenz nachvollziehbar zuzuordnen und zusätzliche Änderungen der Fachlogik oder Oberfläche ausdrücklich auszuweisen.

Der OpenAPI-IPC-Generator veranschaulicht diese Abwägung besonders deutlich. Ein gemeinsamer Vertrag ermöglicht die abgestimmte Ableitung von Client, Hooks, Typen und Laufzeitschemata. Dafür wurde ein eigener Generator entwickelt, der selbst verstanden, geprüft und gepflegt werden muss. Die Entscheidung verlagert wiederkehrende Anpassungen in eine zentrale Übersetzung und ist im erweiterbaren Demonstrator nachvollziehbar. Wie Abschnitt 5.2.4 zeigt, bleiben fachliche Implementierung, Berechtigungsentscheidungen und Fehlerbehandlung eigenständige Aufgaben. Der Vertrag beschreibt die Schnittstelle; ihre korrekte und sichere Verwendung muss in den beteiligten Verarbeitungspfaden gewährleistet werden.

Die Übernahme dieses Vertrags setzt außerdem eine fachliche Prüfung des Ausgangsartefakts voraus. Ein Generator kann Typen und Validierungsschemata konsistent aus einer Beschreibung ableiten, aber nicht feststellen, ob diese Beschreibung die beabsichtigte Domäne richtig modelliert. Ein falscher Zusammenhang zwischen Entitäten oder eine unzutreffende Pflichtfeldregel kann dadurch ebenso konsequent in Schnittstellen und Zod-Schemata übernommen werden. In der Fallstudie wurden ausgewählte Regeln der Studienplanung nachvollzogen; daraus folgt kein Nachweis einer allgemeinen fachlichen Zuverlässigkeit des erzeugenden Sprachmodells. Bei anderen Domänen kann zusätzliches Fachwissen zur Prüfung erforderlich sein. Die gemeinsame Vertragsgrundlage verringert somit widersprüchliche Schnittstellendefinitionen, ersetzt aber keine fachliche Validierung.

Der OpenAPI-Vertrag blieb zudem nicht die einzige Quelle für alle IPC-Schnittstellen der gesamten Migration. Der Kollaborationsbereich erhielt separat definierte TypeScript-Verträge und Zod-Eingabeschemata, während die Kernanwendung den Generator weiterverwendet [83]. Die in Abschnitt 5.3.5 beschriebene Aufteilung begrenzt den Geltungsbereich der zentralen Vertragsquelle auf den generierten Bereich. Dass fachliche Implementierungen zusätzlich geschrieben werden mussten, widerspricht diesem Prinzip für sich genommen nicht: Ein Schnittstellenvertrag beschreibt Aufrufe und Daten, nicht die vollständige Geschäftslogik. Auch die Einführung einer zweiten Datenbank erzwingt keine zweite Vertragsquelle. Die Aufteilung entstand aus der konkreten Integration und müsste für einen durchgehend gemeinsamen Vertrag gesondert bereinigt werden.

Die Trennung in Handler, Services und Repositories unterstützt diese weitere Bearbeitung. Fachliche Tests können ohne React-Oberfläche ausgeführt und externe Abhängigkeiten an den betrachteten Stellen ersetzt werden. Die Struktur schafft damit eine Erweiterungsgrundlage. Ihre vollständige Nutzung durch zusätzliche Tests und weitere Funktionen bleibt eine Entwicklungsaufgabe. Der in

Abschnitt 6.6 festgehaltene frühere Lint-Befund zeigt zudem, dass strukturelle Verbesserungen und erfolgreiche fachliche Tests nicht sämtliche bestehenden Qualitätsprobleme beseitigen.

7.3 Einordnung der Local-First-Architektur

Kleppmann et al. verbinden Local-First mit lokaler Nutzbarkeit, Zusammenarbeit, langfristiger Verfügbarkeit sowie Kontrolle über die eigenen Daten. Lokale Kopien besitzen in diesem Zielbild eine zentrale Rolle; Server unterstützen die Verteilung [19]. Der Demonstrator greift daraus gezielt lokale Datenverarbeitung und ergänzende Synchronisierung auf. Seine Einordnung muss deshalb zwischen diesen umgesetzten Eigenschaften und dem weitergehenden Anspruch auf Unabhängigkeit und Datenkontrolle unterscheiden.

Für die gemeinsamen Aufgaben bleibt das Backend maßgeblich für die Annahme von Änderungen und den anschließend verteilten Datenbestand. PowerSync beschreibt dieses Vorgehen als serverautoritative Architektur [20]. Die Anwendung ermöglicht zunächst einen lokalen Abschluss, während die entfernte Verarbeitung später erfolgen kann. Die Zusammenarbeit bleibt an Konten, Berechtigungen und die gewählten Dienste gebunden. Damit werden ausgewählte Local-First-Eigenschaften demonstriert; die vollständige Umsetzung des von Kleppmann et al. beschriebenen Zielbilds wird nicht beansprucht.

Die zwei lokalen Datenhaltungsbereiche erlaubten, Kollaboration neben den persönlichen Funktionen einzuführen. Diese schrittweise Überführung lässt sich mit dem von Fowler beschriebenen Strangler-Fig-Vorgehen einordnen, bei dem abgegrenzte Teile einer bestehenden Anwendung nach und nach ersetzt werden. Auch Übergangsstrukturen gehören zu diesem Ansatz [24]. Im Demonstrator entstehen dadurch zusätzliche Zuständigkeiten für Datenbanktreiber, Identitäten und Schemaverwaltung.

Eine vollständige Fortführung müsste zwei unabhängige Entwurfsaufgaben behandeln. Für eine gemeinsame Vertragsquelle könnten die Kollaborationsoperationen in den zentralen Vertrag aufgenommen und ihre Typen, Aufrufhilfen und Validierungsschemata daraus erzeugt werden. Die Repositories könnten dabei zunächst weiterhin auf verschiedene Datenbanken zugreifen. Eine zusätzliche Vereinheitlichung der Datenhaltung müsste dagegen bestehende Datensätze und Referenzen, Kennungen sowie lokale und entfernte Benutzerzuordnungen überführen. Persönliche Tabellen wären ausdrücklich lokal zu konfigurieren; ihr Zugriff müsste auch ohne entfernte Anmeldung möglich bleiben. Erst nach Ablösung aller bisherigen Datenzugriffe und ihrer Initialisierung könnte der libSQL-Treiber entfallen. Die Schema-Vorprüfung aus Phase 2 zeigt einzelne Anpassungen, weist jedoch weder eine solche vollständige Datenmigration noch einen vereinheitlichten Gesamtvertrag nach. Diese Schritte sind mögliche Weiterentwicklungen des Entwurfs.

Die Synchronisierung verändert außerdem die Verantwortung für Updates. Das ausgelieferte Desktop-Paket verwaltet nur einen Teil der beteiligten Strukturen. Änderungen an synchronisierten Daten betreffen auch das entfernte Schema und die Datenverteilung. Die in Abschnitt 5.3.8 hergeleitete Abstimmung ist deshalb eine betriebliche Konsequenz der gewählten Architektur. Bei einer praktischen Anwendung müssten unterschiedliche Client-Versionen und noch ausstehende Änderungen berücksichtigt werden.

Die Upload-Nachprüfung aus Abschnitt 6.4 verdeutlicht schließlich, welche zusätzlichen Anforderungen aus der zeitlich getrennten Verarbeitung entstehen. Fehler bei Ablehnung und Wiederholung wurden durch erfolgreiche Standardabläufe zunächst nicht erkannt. Daraus folgt für den Leitfaden, dass der lokale Schreibabschluss, die Annahme durch das Backend und das Verhalten bei Wiederholung eigene Kontrollpunkte benötigen. Fachliche Regeln für neue konkurrierende Bearbeitungen bleiben anwendungsspezifisch. Der untersuchte Wiederholungsschutz und das Ablehnungsjournal behandeln dagegen konkrete Zuverlässigkeitsprobleme des übertragenen Verarbeitungspfads. Diese

Aufgaben müssen bei der Migration ausdrücklich eingeordnet werden, auch wenn eine umfassende Konfliktzusammenführung außerhalb des Untersuchungsumfangs liegt.

7.4 Bedeutung der Härtung und des Paketbetriebs

Die Härtung betrifft das Zusammenwirken von Prozessisolation, begrenzten Fähigkeiten und Prüfungen an den Eingangsstellen. Die Architektur begrenzt die Funktionen des Renderers; die Schnittstellenvalidierung prüft übertragene Daten, während fachliche Zugriffskontrollen deren zulässige Verwendung bestimmen. Die Nachweise aus Abschnitt 6.5 sind deshalb als Ergänzung dieser Struktur zu verstehen. Eine einzelne Maßnahme oder ein bestandener negativer Fall trägt jeweils nur die zugehörige begrenzte Aussage.

Die dokumentierten Nachbesserungen zeigen die Bedeutung des erzeugten Pakets als Prüfgegenstand. Native Datenbankmodule, Renderer-Ressourcen und buildabhängige Sicherheitskonfigurationen müssen in ihrer tatsächlichen Auslieferungsform funktionieren. Änderungen am Ressourcenprotokoll und an der URL-Verarbeitung beeinflussten im untersuchten Verlauf auch Formulare und Synchronisierung [79]. Funktionale und sicherheitsbezogene Prüfungen sind damit praktisch miteinander verbunden: Nach einer Härtungsänderung müssen die betroffenen Abläufe am neuen Paket erneut nachvollzogen werden.

Die Sicherheitsmatrix schafft hierfür einen nachvollziehbaren Bewertungsrahmen. Sie macht sichtbar, welche Empfehlungen anwendbar sind und welche Aussage durch Codeinspektion, Unit-Test oder Laufzeitversuch gestützt wird. Die dokumentierte Grenze bei der SDK-internen PowerSync-Redirectbehandlung bleibt dabei erhalten. Ebenso sind die ausgeschlossenen Schutzaufgaben für entfernte Dienste und die lokale Datenbank innerhalb des Betriebssystems bei einer späteren Nutzung gesondert zu bestimmen. Die Härtungsbewertung dieser Arbeit dient der begrenzten Prüfung des Demonstrators; sie ist keine allgemeine Freigabe für den produktiven Betrieb.

7.5 Übertragbarkeit und Grenzen der Ergebnisse

Potenzial für eine Übertragung besitzen vor allem die beschriebenen Verantwortlichkeiten: die Trennung von fachlichem Vertrag und Transport, die Aufteilung von Schnittstellenverarbeitung und Fachlogik, die lokale Persistenz sowie die Einordnung externer Datenflüsse. Die konkreten Skripte, Bibliotheken und Konfigurationen sind dagegen an den untersuchten Technologiestack gebunden. Der frameworkagnostische Anspruch liegt somit auf der Ebene dieser Prinzipien und Prüffragen. Eine praktische Übertragbarkeit auf weitere Frameworks wurde nicht unabhängig nachgewiesen.

Die Voraussetzungen des Ausgangsartefakts begrenzen diesen Anspruch zusätzlich. Im Replit-Prototyp lagen Frontend, serverseitige Fachlogik und Datenbankschema als analysierbarer Quellcode vor. Bei der in Kapitel 4 eingeordneten dienstgebundenen Ausgangslage wären auch die vom entfernten Dienst übernommenen Aufgaben zu erfassen und gegebenenfalls lokal nachzubilden. Der Leitfaden bietet hierfür Fragen zur Zuständigkeit und Datenverarbeitung, aber keinen erprobten vollständigen Migrationsablauf.

Auch die Herkunft des Prototyps ist entsprechend einzuordnen. Die Fallstudie untersucht ein anhand alltagssprachlicher Prompts erzeugtes Artefakt. Die beobachtete Kopplung und die erforderlichen Anpassungen werden an dessen konkreter Architektur nachvollzogen. Daraus lässt sich kein allgemeiner Vergleich zwischen KI-generiertem und menschlich entwickeltem Code ableiten. Für die Anwendung des Leitfadens ist daher die Prüfung des tatsächlich vorliegenden Artefakts maßgeblich. Ebenso begründet die rückblickende Einordnung des Vorversuchs die Verfahrensänderung, ohne eine allgemeine Aussage über die Zuverlässigkeit LLM-gestützter Migrationen zu tragen. Ob das überarbeitete Vorgehen systematisch weniger unbeabsichtigte Änderungen erzeugt, wurde nicht unter vergleichbaren Bedingungen untersucht.

Die Evaluation verbindet Nachweise unterschiedlicher Ebenen und Prüfstände. Lokale Service-Tests, Integrationsprototypen und ausgewählte Windows-Paketabläufe ergänzen sich, ersetzen aber keinen vollständigen gemeinsamen Durchlauf. Die wichtigste funktionale Nachweislücke bleibt A5: Die lokalen Verarbeitungspfade sind nachvollziehbar, der vollständige Offline- und Neustartablauf aller persönlichen Kernfunktionen ist jedoch nicht dokumentiert. Dieser offene Nachweis beschränkt die Aussage über den erreichten Gesamtzustand der Anwendung. Der Leitfaden formuliert den entsprechenden Kontrollpunkt bereits und macht damit auch sichtbar, welche Prüfung noch erforderlich ist.

Entwicklung und Bewertung erfolgten innerhalb derselben Fallstudie. Die Zuordnung der Schritte zum Demonstrator stützt die Herleitung des Leitfadens; seine Verständlichkeit und Anwendbarkeit durch eine weitere Person wurden nicht gesondert erprobt. Die Erweiterbarkeit ist folglich durch die vorhandenen Muster begründet. Sie ist keine empirische Aussage darüber, dass eine unerfahrene Person zusätzliche Funktionen ohne wesentliche Fehler umsetzen kann. Ebenso bleiben technische Kenntnisse zur Durchführung und Bewertung der Migration erforderlich.

Für die weitere Verwendung sind daraus zwei vorrangige Aufgaben abzuleiten: den fehlenden Offline- und Persistenznachweis am festgelegten Paketstand zu vervollständigen und den Leitfaden an einem weiteren Ausgangsartefakt unabhängig anzuwenden. Die erste Aufgabe stärkt die Aussage über den Demonstrator; die zweite prüft den Geltungsbereich der Handlungsempfehlungen. Auf dieser Grundlage können die in der Fallstudie begründeten Prinzipien schrittweise zu einem breiter erprobten Migrationsverfahren weiterentwickelt werden.

Kapitel 8

Fazit und Ausblick

8.1 Fazit

Diese Arbeit hat einen Migrationsleitfaden für die Überführung eines KI-generierten Webprototyps in eine Electron-Anwendung mit lokaler Datenhaltung und ergänzender Synchronisierung entwickelt. Ausgangspunkt war der auf Replit erzeugte Studienplaner. Seine Funktionen und Architektur wurden analysiert, Anforderungen an die Zielanwendung abgeleitet und die erforderlichen Veränderungen anhand eines repräsentativen Kernumfangs umgesetzt.

Die Forschungsfrage aus Kapitel 1 lässt sich für den untersuchten Fall durch die systematische Neuordnung von Verantwortlichkeiten beantworten. Vorhandene Komponenten werden zunächst nach Darstellung, Schnittstellenverarbeitung, Fachlogik, Datenhaltung und externen Abhängigkeiten eingeordnet. Darauf aufbauend entsteht eine lokal verarbeitende Kernarchitektur mit begrenzten Prozessschnittstellen. Externe Datenübernahme und die Synchronisierung gemeinsamer Aufgaben ergänzen diese Grundlage. Die Härtung begleitet die Entwurfsentscheidungen und wird abschließend anhand der tatsächlich entstandenen Anwendung konsolidiert.

Die Umsetzung zeigt, auf welchen Ebenen Wiederverwendung möglich ist. React-Oberfläche, fachliche Regeln, Drizzle und der OpenAPI-Vertrag konnten als Grundlagen erhalten bleiben. Notwendige Änderungen betrafen insbesondere Datenbankdialekt und Schemaverwaltung, Identitätszuordnung sowie den Wechsel von HTTP-Aufrufen zu Preload und IPC. Die Trennung in Handler, Services und Repositories machte diese Eingriffe nachvollziehbar und schuf eine Grundlage für fachliche Tests. Dateiimporte und Katalogabruf ergänzten das CRUD-Muster um begrenzte Datei- und Netzwerkoperationen. Die Kollaboration erforderte zusätzlich eine eigenständige Behandlung lokaler Änderungen, ihrer entfernten Annahme und möglicher Ablehnungen oder Wiederholungen.

Das zentrale Ergebnis ist der in Abschnitt 5.5 formulierte Leitfaden. Seine Handlungsempfehlungen verbinden Voraussetzungen, Transformationsschritte und Kontrollpunkte mit dem Demonstrator. Wiederkehrende Aufgaben werden dadurch als fortführbare Muster beschrieben. Der frameworkagnostische Anteil liegt auf der Ebene dieser Verantwortlichkeiten und Prüffragen; Generator, Treiber und Konfigurationen bleiben an den konkreten Technologiestack gebunden.

Die Evaluation stützt die Architektur, ausgewählte funktionale Verarbeitungspfade und die dokumentierte Härtung innerhalb des festgelegten Umfangs. Die wichtigste offene funktionale Nachweisaufgabe betrifft A5: Ein vollständiger Offline-Paketdurchlauf aller persönlichen Kernfunktionen einschließlich Neustartpersistenz ist nicht dokumentiert. Die Sicherheitsbewertung bleibt an ihre ausgewiesenen Nachweisebenen und Grenzen gebunden. Eine unabhängige Übertragung des Leitfadens auf weitere

Artefakte und ein vergleichender Nachweis geringsten Migrationsaufwands wurden nicht durchgeführt. Abschnitt 6.6 und Abschnitt 7.5 ordnen diese Grenzen ein.

Der Beitrag der Arbeit besteht damit in einer fallbezogen entwickelten und begründeten Vorgehensweise. Sie verbindet die Weiterverwendung eines vorhandenen Prototyps mit architektonischer Neuordnung, lokaler Nutzbarkeit und expliziten Prüfaufgaben. Die Zuordnung von Entscheidung, Umsetzung und Nachweis ermöglicht es, sowohl erreichte Eigenschaften als auch noch erforderliche Arbeiten nachvollziehbar zu benennen. Die Erfahrung aus dem Vorversuch unterstreicht dabei die Bedeutung, unbeabsichtigte Verhaltensänderungen zu begrenzen und notwendige Anpassungen ausdrücklich zu begründen.

8.2 Ausblick

Als nächster Schritt ist der fehlende Nachweis der Offline-Kernfunktionen am festgelegten Windows-Paketstand zu vervollständigen. Dazu gehören die Anmeldung und persönliche Datenzuordnung, die ausgewählten Bearbeitungs- und Importabläufe bei unterbrochener Verbindung sowie das erneute Lesen gespeicherter Änderungen nach einem Neustart. Für katalogabhängige Abläufe ist der vorausgesetzte lokale Datenbestand festzuhalten. Die Prüfungen sollten die gebaute Anwendung, ihre Datenbankinitialisierung und die nativen Dateidialoge gemeinsam erfassen, damit die vorhandenen Service-Tests durch Beobachtungen des vollständigen Verarbeitungspfads ergänzt werden.

Ergänzend sollte das Verhalten ausgewählter Abläufe zwischen Ausgangs- und Zielanwendung systematisch verglichen werden. Gleiche fachliche Eingaben und erwartete Ergebnisse können dabei als Grundlage für Regressionstests dienen. Dazu gehören insbesondere Formulareingaben, Datumsverarbeitung und Validierungsfälle. Beabsichtigte Änderungen durch die Zielarchitektur sind gesondert festzuhalten, damit sie von unbeabsichtigten Abweichungen unterschieden werden können.

Für die weitere Erprobung des Leitfadens bietet sich eine unabhängige Migration eines zweiten Ausgangsartefakts an. Eine andere Frontend-Technologie könnte zeigen, welche Empfehlungen auf der beschriebenen Abstraktionsebene fortgeführt werden können. Ein dienstgebundenes Backend würde zusätzlich die in Kapitel 4 eingegrenzte Frage aufgreifen, wie bislang entfernte Verantwortlichkeiten lokal nachgebildet werden müssen. Zu erfassen wären insbesondere erforderliche Anpassungen, unklare Anleitungsschritte und fehlende Kontrollpunkte. So ließe sich die Übertragbarkeit des Leitfadens über den bisherigen Prozess- und Strukturabgleich hinaus bewerten.

Eine Fortführung des Demonstrators kann anschließend die Datenhaltungs- und Updatearchitektur vertiefen. Eine Zusammenführung der lokalen Datenbestände müsste bestehende Datensätze und Beziehungen erhalten. Für synchronisierte Bereiche wäre ein koordinierter Schemaübergang mit unterschiedlichen Client-Versionen und ausstehenden Änderungen zu erproben. Weitergehende Konfliktregeln wären aus konkreten fachlichen Anforderungen abzuleiten. Diese Erweiterungen setzen die untersuchten Muster fort, benötigen aber eigene Implementierungs- und Funktionsnachweise.

Die Sicherheitsprüfung sollte bei solchen Änderungen an den neuen Paketstand angepasst werden. Ein offener Prüfpunkt bleibt insbesondere die SDK-interne Redirectbehandlung des Synchronisierungstransports. Neue privilegierte Fähigkeiten oder Datenflüsse erfordern zudem einen erneuten Abgleich der Prozess- und Schnittstellengrenzen. Eine spätere praktische Nutzung müsste ihren erweiterten Funktions-, Betriebs- und Sicherheitsumfang ausdrücklich festlegen.

Diese weiteren Schritte können die technische Belegbasis vervollständigen und den Geltungsbereich der Handlungsempfehlungen erweitern. Der entwickelte Leitfaden bildet dafür eine nachvollziehbare Grundlage, die anhand zusätzlicher Fälle und ihrer Ergebnisse weiter geprüft und präzisiert werden kann.

Abbildungsverzeichnis

4.1 Ursprüngliche Webarchitektur des Studienplaners mit Laufzeitkommunikation und OpenAPI-basierter Codegenerierung; eigene Darstellung auf Grundlage des Ausgangsartefakts. 16
5.1 Vertragsbasierte Codegenerierung und Aufrufwege im Web-Prototyp und in der lokalen Electron-Kernanwendung. Gestrichelte Pfeile zeigen Generierung oder Einbindung, durchgezogene Pfeile Laufzeitkommunikation. Spätere Synchronisierungs- und Online-Pfade sind nicht dargestellt. 29
5.2 Architektur nach Ergänzung des synchronisierten Gruppenbereichs. Dargestellt sind die beiden lokalen Datenhaltungsbereiche sowie direkte Online-Aufrufe, Upload und Download. Der separate Katalogabruf ist hier nicht dargestellt. Eigene Darstellung nach [63] und [58]. 39
5.3 Modulaufteilung der Kollaborationsintegration neben der lokalen Kernanwendung. Blaue Pfeile zeigen lokale Aufruf- und Datenzugriffswege, violette Pfeile Dienstkommunikation; der gestrichelte Pfeil zeigt die Schemaeinbindung. Eigene Darstellung nach [64]. 43

Quellcodeverzeichnis

4.1 Exemplarischer Express-Route-Handler für die Abfrage anstehender Termine im Ausgangsartefakt. 17
5.1 Drizzle-Kit-Konfiguration zur Erzeugung der SQLite-Migrationen. 24
5.2 Auslieferung der Migrationsdateien über zusätzliche Paketressourcen (Ausschnitt). 25
5.3 Aufbau eines Zod-Ausdrucks als Zeichenkette in `zodSchema`; gekürzter Auszug, Kommentar ergänzt [39]. 26
5.4 Übersetzung ausgewählter Terminfelder in ein Laufzeitschema und einen abgeleiteten Typ; reduzierte Darstellung nach [39]. 27
5.5 Auswahl gemeinsamer Aufgaben in `sync-streams-v4.yaml`; die übrigen Abfragen wurden weggelassen. 40
5.6 Öffnen und Verbinden der benutzerbezogenen PowerSync-Datenbank; gekürzter Auszug aus `CollaborationRuntime.signIn` [64]. 41
5.7 Ausgewählte Felder des lokalen Terminschemas aus Phase 1; weitere Felder weggelassen. 45
5.8 Entsprechende Felder der separaten PowerSync-Prototyptabelle und Schemaableitung; weitere Felder und Indizes weggelassen. 46

Tabellenverzeichnis

4.1 Funktionaler Untersuchungsumfang auf Grundlage der ursprünglichen Erstellungs-Prompts. . 14
5.1 Phasen und Kontrollpunkte des fallbezogenen Migrationsleitfadens. 22
5.2 Ausgewählte Abbildungen der PostgreSQL-Schemadefinitionen auf SQLite im Demonstrator. 23
5.3 Ausgewählte Übersetzungen vom OpenAPI-Vertrag in die projektspezifischen IPC-Module [39]. 27
5.4 Wiederverwendete Vertragsgrundlage und Änderungen an ihrer Umsetzung. 29
5.5 Aufteilung der ursprünglichen Notenrouten auf Handler, Service und Repository. 33
5.6 Dokumentationsbasierte Gegenüberstellung der für die Vorauswahl betrachteten Integrationsansätze [54], [55], [57], [58]. 37
5.7 Für den Kollaborationsbereich ergänzte Laufzeitabhängigkeiten und ihre Aufgaben [64]. 41
5.8 Prüforte, Härtungsmaßnahmen und Nachweise im Demonstrator; Zuordnung nach [78] und [79], ergänzt um die lokalen Importpfade nach [80]. 50
5.9 Handlungsempfehlungen und Kontrollpunkte, abgeleitet aus dem dokumentierten Migrationsprozess; die Zuordnung konkretisiert A9. 57
6.1 Erneute Ausführung vorhandener lokaler Service-Tests am 10. Oktober 2026; Protokoll und Prüfstand nach [82]. 59
6.2 Funktionale Gegenüberstellung für den Kernumfang nach [28], [81] und den Prüfprotokollen [82], [79] und [75]. 61
6.3 Beobachtete negative Laufzeitfälle am korrigierten Windows-Paket vom 4. Oktober 2026 nach [79]. 63
6.4 Anforderungsbezogene Bewertung auf Grundlage der in diesem Kapitel ausgewiesenen Belege und Nachweisgrenzen. 64

Glossar

Referenzen

[1] Lovable, „App & Website Builder: Build Software in Minutes“. Zugegriffen: 6. Oktober 2026. [Online]. Verfügbar unter: https://lovable.dev/

[2] N. Perry, M. Srivastava, D. Kumar, und D. Boneh, „Do Users Write More Insecure Code with AI Assistants?“, in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, S. 2785–2799. doi: 10.1145/3576915.3623157.

[3] IREB, „CPRE Online Glossar“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://cpre.ireb.org/de/downloads-and-resources/glossary

[4] ISO/IEC, „ISO/IEC 25010:2023: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://www.iso.org/standard/78176.html

[5] MDN contributors, „Client-server overview“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview

[6] Microsoft, „N-tier architecture style“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier

[7] R. Stafford, „Service Layer“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://martinfowler.com/eaaCatalog/serviceLayer.html

[8] E. Hieatt und R. Mee, „Repository“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://martinfowler.com/eaaCatalog/repository.html

[9] OpenAPI Initiative, „OpenAPI Specification v3.1.0“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://spec.openapis.org/oas/v3.1.0.html

[10] TypeScript Team, „The Basics“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://www.typescriptlang.org/docs/handbook/2/basic-types.html

[11] Zod, „Basic usage“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://zod.dev/basics

[12] Electron, „Process Model“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/process-model

[13] Electron, „Context Isolation“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/context-isolation

[14] Electron, „Inter-Process Communication“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/ipc

[15] MDN contributors, „URI fragment“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://developer.mozilla.org/en-US/docs/Web/URI/Reference/Fragment

[16] Wouter Contributors, „Wouter: Customizing the location hook“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://github.com/molefrog/wouter#customizing-the-location-hook

[17] SQLite, „About SQLite“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://www.sqlite.org/about.html

[18] Drizzle Team, „Drizzle Kit: generate“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/drizzle-kit-generate

[19] M. Kleppmann, A. Wiggins, P. van Hardenberg, und M. McGranaghan, „Local-first software: You own your data, in spite of the cloud“, in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019, S. 154–178. doi: 10.1145/3359591.3359737.

[20] PowerSync, „Writing Client Changes“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/handling-writes/writing-client-changes

[21] R. T. Fielding, M. Nottingham, und J. Reschke, „HTTP Semantics“, Request for Comments 9110, 2022. doi: 10.17487/RFC9110.

[22] J. H. Saltzer und M. D. Schroeder, „The Protection of Information in Computer Systems“, *Proceedings of the IEEE*, Bd. 63, Nr. 9, S. 1278–1308, 1975, Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://web.mit.edu/Saltzer/www/publications/protection/Basic.html

[23] Electron, „Security“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/security

[24] M. Fowler, „Strangler Fig“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://martinfowler.com/bliki/StranglerFigApplication.html

[25] K. Peffers, T. Tuunanen, M. A. Rothenberger, und S. Chatterjee, „A Design Science Research Methodology for Information Systems Research“, *Journal of Management Information Systems*, Bd. 24, Nr. 3, S. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[26] Replit, „Web Apps“. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://docs.replit.com/features/artifact-types/web-apps

[27] Lovable, „Connect to Supabase“. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://docs.lovable.dev/integrations/supabase

[28] „Studienplaner: Replit-Ausgangsartefakt“. 2026.

[29] Tauri, „Process Model“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://v2.tauri.app/concept/process-model/

[30] Tauri, „Node.js as a sidecar“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://v2.tauri.app/learn/sidecar-nodejs/

[31] electron-vite, „Getting Started“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://electron-vite.org/guide/

[32] Electron Forge, „React with TypeScript“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronforge.io/guides/framework-integration/react-with-typescript

[33] Electron Forge, „Getting Started“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronforge.io/

[34] „StudyPlan: Lokale Datenhaltung und Schemamigrationen“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/17434b2c81b0dc4300f5e71651484a7f1ad86222

[35] Electron, „app: getPath“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/api/app#appgetpathname

[36] Drizzle Team, „SQLite column types“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/sqlite/column-types

[37] Drizzle Team, „Drizzle Kit: push“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/drizzle-kit-push

[38] SQLite, „Pragma statements supported by SQLite“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.sqlite.org/pragma.html

[39] „StudyPlan: Vertragsbasierte API-Generierung und IPC-Schnittstelle“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/17434b2c81b0dc4300f5e71651484a7f1ad86222/scripts/openapi-ipc

[40] „StudyPlan: Übernahme und Anbindung der React-Oberfläche“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/17434b2c81b0dc4300f5e71651484a7f1ad86222/src/renderer

[41] „StudyPlan: Lokaler Authentifizierungsablauf“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/commit/1c1988dd7d6b25756ef6d807fbe6dacfa47fe4f5

[42] Electron, „contextBridge“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/api/context-bridge

[43] Fontsource, „Vite“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://fontsource.org/docs/guides/vite

[44] electron-vite, „Configuring electron-vite“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://electron-vite.org/config/

[45] Microsoft, „TSConfig Reference: paths“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.typescriptlang.org/tsconfig/paths.html

[46] Tailwind Labs, „Installing Tailwind CSS with Vite“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://tailwindcss.com/docs/installation/using-vite

[47] Microsoft, „TSConfig Reference: noUnusedLocals“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.typescriptlang.org/tsconfig/noUnusedLocals.html

[48] „StudyPlan: Migration der CRUD-Operationen für Noten“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/commit/f97642bc3a7f4bb1844ebe6ee23d97393e4d6477

[49] „StudyPlan: Lokaler und synchronisierter Gruppenbereich“. 2026.

[50] „StudyPlan: Lokaler iCal-Import und seine Ergebnisbehandlung“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/commit/fe082207f46329ae3f6b1ef7afbf78469781d462

[51] Electron, „dialog: showOpenDialog“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/api/dialog#dialogshowopendialogwindow-options

[52] „StudyPlan: Lokaler CSV-Notenimport mit eigenem Parser“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/commit/d37f1b54ffb456d46e51ac6158bb036effa5327e

[53] „StudyPlan: Lokale Kernanwendung vor der Synchronisierungsintegration“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/ae4ca65fbfbcdefdac400cba721171daf22423b3

[54] PowerSync, „PowerSync Drizzle Driver“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://www.npmjs.com/package/@powersync/drizzle-driver

[55] Drizzle Team, „Turso Cloud“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/sqlite/connect-turso

[56] PowerSync, „Node.js Client SDK“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/client-sdks/reference/node

[57] Turso, „Embedded Replicas“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.turso.tech/features/embedded-replicas/introduction

[58] PowerSync, „Supabase + PowerSync“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/integrations/supabase/guide

[59] Supabase, „Database“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/guides/database/overview

[60] Supabase, „Auth“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/guides/auth

[61] Turso, „Turso Sync: Usage“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.turso.tech/sync/usage

[62] „StudyPlan: Vorprüfung von PowerSync und der Drizzle-Anbindung“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/7886f2ba852b55fcfd711b280a90325f7134e8ca/docs/spikes

[63] „StudyPlan: Architektur und Umsetzung des synchronisierten Gruppenbereichs“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/blob/e5d54e08ceaedf128ffecedce70d8e12c359219e/docs/spikes/powersync-groups/implementation.md

[64] „StudyPlan: Technische Integration von PowerSync und Supabase“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/e5d54e08ceaedf128ffecedce70d8e12c359219e

[65] PowerSync, „Sync Streams“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/sync/streams/overview

[66] Electron, „ASAR Archives“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/asar-archives

[67] „Studienplaner: Statische Prüfung des ursprünglichen Einladungsablaufs“. 2026.

[68] Supabase, „User Management“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/guides/auth/managing-user-data

[69] Supabase, „JavaScript: signInWithPassword“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/reference/javascript/auth-signinwithpassword

[70] „StudyPlan: Vergleich des lokalen Terminschemas mit dem PowerSync-Prototyp“. 2026.

[71] PowerSync, „Local-Only Usage“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/client-sdks/advanced/local-only-usage

[72] Supabase, „Database Migrations“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/guides/deployment/database-migrations

[73] PowerSync, „Implementing Schema Changes“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/maintenance-ops/implementing-schema-changes

[74] PowerSync, „Deploying Schema Changes“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/maintenance-ops/deploying-schema-changes

[75] „Kollaborations-Uploads: Nachbesserung und Prüfbelege“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/blob/17434b2c81b0dc4300f5e71651484a7f1ad86222/docs/spikes/powersync-groups/p1-hardening-results.md

[76] „Online-Abruf und lokale Übernahme des Studienkatalogs“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/blob/9320911beb401517d1499e93af6c165412a22284/docs/study-catalog.md

[77] „Planung und erster Abgleich der Electron-Sicherheitsprüfung“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/17434b2c81b0dc4300f5e71651484a7f1ad86222/docs

[78] „Konsolidierte Electron-Sicherheitsergebnismatrix“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/blob/17434b2c81b0dc4300f5e71651484a7f1ad86222/docs/electron-security-matrix.md

[79] „Nachbesserung und Verifikation der Electron-Sicherheitsprüfung“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/blob/17434b2c81b0dc4300f5e71651484a7f1ad86222/docs/electron-security-remediation-results.md

[80] „StudyPlan: Implementierung der Sicherheitsgrenzen“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/17434b2c81b0dc4300f5e71651484a7f1ad86222/src/main

[81] „StudyPlan: Quellcodeabgleich für die Evaluation“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/17434b2c81b0dc4300f5e71651484a7f1ad86222

[82] „StudyPlan: Erneute lokale Service-Prüfung“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://github.com/lysmz/studyplan/tree/17434b2c81b0dc4300f5e71651484a7f1ad86222/scripts

[83] „StudyPlan: Generierte und separat definierte IPC-Verträge“. 2026.

Anhang A

Ursprüngliche Erstellungs-Prompts

Dieser Anhang dokumentiert die beiden ursprünglichen Prompts, mit denen der Replit-Webprototyp Studienplaner erstellt und um Studiengruppen erweitert wurde. Die Texte werden im Wortlaut der vom Verfasser bereitgestellten Datei wiedergegeben. Rechtschreibung und Formulierungen bleiben erhalten; Überschriften, Absatzdarstellung und die Trennung der beiden Prompts wurden für die Dokumentation ergänzt.

Die Prompts belegen die Beschreibung von Funktionen und Nutzungsszenarien ohne Festlegung eines UI-Frameworks oder einer konkreten technischen Architektur. Sie enthalten zugleich Anforderungen an Anmeldung, Zugriffsschutz und die Trennung persönlicher und gemeinsamer Inhalte. Der ursprünglich gewünschte Funktionsumfang ist von dem in Kapitel 4 festgelegten Untersuchungsumfang zu unterscheiden; insbesondere begründet der im zweiten Prompt genannte Gerätewechsel keine Erweiterung des Migrations- oder Evaluationsumfangs dieser Arbeit.

Quelle ist die bereitgestellte Textdatei `Pasted-Hi-Ich-m-chte-eine-Web-App-f-r-mein-Studium-bauen-damit_1783021865134.txt`. Eine unveränderte Kopie ist im Thesis-Projekt unter `content/sources/original-creation-prompts.txt` abgelegt. Die Kennzeichnung „Prompt 2“ in der Quelldatei dient hier zur Aufteilung in die folgenden Abschnitte.

A.1 Prompt 1: Persönliche Studienorganisation

> Hi! Ich möchte eine Web-App für mein Studium bauen, damit ich endlich mal mein ganzes Chaos im Griff habe. Ich habe mir das so vorgestellt:
>
> Am Anfang soll man auf ein Dashboard kommen. Das ist quasi die Startseite, wo ich sofort sehe, wie mein aktueller Notenschnitt (GPA) ist, wie viele ECTS-Punkte ich insgesamt schon gesammelt habe und welche Fristen als Nächstes ablaufen. Da sollen auch ein paar schnelle Buttons sein, womit ich sofort eine neue Note oder einen neuen Termin eintragen kann, ohne erst mühsam in die Untermenüs zu klicken. Und eine kleine Liste mit den letzten Aktivitäten und den nächsten Events wäre super.
>
> Damit ich meine Termine im Blick habe, brauche ich einen Kalender. Ganz klassisch als Monatsübersicht. Wenn ich auf einen Tag klickt, soll sich so ein kleines Infofenster (Popover) öffnen, das mir alle Termine für den Tag anzeigt. Und wenn ich einen Termin anklicke, will ich die Details sehen und ihn auch löschen können. Die Termine müssen farblich sortiert sein, damit man sofort sieht, was Sache ist: Prüfungen und Deadlines in Rot, Anmeldefristen in Gelb, normale Vorlesungen in Blau und alles andere in Grün. Ganz wichtig: Ich will meine Termine von der Uni als iCal- oder ICS-Datei hochladen können, damit ich nicht alles einzeln abtippen

muss! Manchmal gehen Events auch über mehrere Tage, das muss der Kalender anzeigen können. Und eine normale Listenansicht, wo ich nach Wichtigkeit filtern kann, wäre auch cool.

Dann brauche ich einen Bereich für meine Noten. Das soll eine schöne Tabelle sein, wo alle meine Kurse mit den Noten und ECTS-Punkten drinstehen, am besten nach Semestern sortiert. Ich will Noten hinzufügen, bearbeiten und löschen können. Weil es nervt, alles einzutippen, wäre eine Funktion genial, wo ich eine CSV-Datei mit meinen Noten hochlade und die App schlau genug ist, zu erkennen, was in welcher Spalte steht. Der Notenschnitt muss sich dann automatisch anhand der ECTS-Gewichtung ausrechnen. Und die App muss aufpassen, dass ich nur Noten für Kurse eintragen kann, die auch wirklich zu meinem Studiengang gehören.

Für den Stundenplan stelle ich mir ein Raster vor: 5 Wochentage (Montag bis Freitag) und 6 feste Zeitblöcke am Tag. Die Zeiten an unserer Uni sind immer: 8:00-9:30, 9:45-11:15, 11:30-13:00, 14:00-15:30, 15:45-17:15 und 17:30-19:00. Man soll über ein Dropdown-Menü fertige Pläne von der Uni (die irgendwie aus den Daten der Uni-Website gezogen wurden) durchsuchen und auswählen können. Mit einem einfachen Plus- oder Minus-Button will ich dann Kurse zu meinem persönlichen Plan hinzufügen oder entfernen können. Man soll optisch sofort den Unterschied sehen zwischen den vorgeschlagenen Uni-Kursen und meinen selbst hinzugefügten. Und ein Button, um den ganzen Plan auf einmal zu löschen (nachdem man nochmal kurz gefragt wurde, ob man sich sicher ist), wäre top.

Ach ja, das Ganze muss natürlich sicher sein! Man muss sich registrieren und einloggen können, damit jeder nur seine eigenen Noten und Termine sieht. Und einen Ausloggen-Button brauche ich natürlich auch.

Ich habe leider überhaupt keine Ahnung von Programmieren oder wie man sowas baut, aber ich hoffe, du kannst mir daraus eine schicke, einfach zu bedienende App machen! Danke schonmal!

A.2 Prompt 2: Studiengruppen und Zusammenarbeit

Mir ist gerade noch etwas eingefallen: Ich würde die App gerne auch mit anderen aus meinem Semester benutzen können.

Vielleicht könnte man sogenannte Studiengruppen anlegen. Zum Beispiel für ein bestimmtes Modul, eine Lerngruppe oder die Bachelorarbeit. Eine Gruppe sollte einen Namen bekommen und man sollte andere Leute über einen Einladungslink oder per E-Mail hinzufügen können.

In einer Studiengruppe wäre es praktisch, wenn man gemeinsame Termine und Aufgaben verwalten könnte. Zum Beispiel:

Prüfungstermine
Abgabefristen
Lerntreffen
gemeinsame To-dos
wichtige Links oder Notizen
Bei Aufgaben sollte man ungefähr sehen können, worum es geht, wann sie fällig sind und wer sich darum kümmert. Man sollte Aufgaben auch als offen oder erledigt markieren können.

Zu gemeinsamen Terminen und Aufgaben sollten die Mitglieder Kommentare schreiben können. Außerdem wäre es hilfreich, wenn man sehen kann, wer etwas erstellt oder zuletzt geändert hat.

Die gemeinsamen Inhalte sollen nur für die Mitglieder der jeweiligen Gruppe sichtbar sein. Meine persönlichen Noten und privaten Termine sollen nicht automatisch mit anderen geteilt werden. Wenn ich etwas teilen möchte, sollte ich das selbst auswählen können.

Wenn jemand aus der Gruppe einen Termin oder eine Aufgabe ändert, sollten die anderen Mitglieder die Änderung sehen, wenn sie die App öffnen. Ich möchte die App außerdem mit meinem Benutzerkonto auch an einem anderen Gerät verwenden können, ohne dass meine Gruppen und Einträge dort fehlen.

Die Studiengruppen sollten sich möglichst einfach bedienen lassen und sich optisch gut in die bestehende Kalender- und Aufgabenansicht einfügen. Es wäre schön, wenn persönliche und gemeinsame Termine klar voneinander zu unterscheiden wären.